

Curso: Transición al Backend con Java y Spring Boot

Para Desarrolladores Node.js/TypeScript que buscan dominar el ecosistema Java

Información del Curso

Audiencia Objetivo

Este curso está diseñado para desarrolladores con experiencia sólida en:

- **JavaScript/TypeScript** - Programación funcional y orientada a objetos
- **Node.js Backend** - Express, NestJS, frameworks modernos
- **Bases de Datos** - PostgreSQL, MongoDB, ORMs como TypeORM
- **API Development** - REST APIs, autenticación JWT, microservicios
- **DevOps Básico** - Docker, CI/CD, deployment básico

Prerrequisitos

- ☒ **Programación:** Experiencia sólida en JavaScript/TypeScript
- ☒ **Backend:** Desarrollo de APIs REST con Node.js
- ☒ **Bases de Datos:** Conocimiento de SQL y ORMs
- ☒ **Arquitectura:** Conceptos de microservicios y patrones de diseño
- ☒ **Herramientas:** Git, Docker básico, testing frameworks

Objetivos de Aprendizaje

Al finalizar este curso, serás capaz de:

Core Java & JVM

-  Dominar Java como lenguaje compilado y la arquitectura JVM
-  Aplicar programación orientada a objetos avanzada en Java
-  Manejar colecciones, streams y APIs modernas de Java

Spring Boot Ecosystem

-  Construir aplicaciones enterprise con Spring Framework
-  Desarrollar APIs REST robustas y escalables con Spring Boot
-  Implementar seguridad completa con JWT y Spring Security
-  Gestionar persistencia con Spring Data JPA + PostgreSQL

Arquitectura & DevOps

-  Diseñar y desarrollar microservicios con Spring Boot
-  Implementar testing completo (Unit, Integration, E2E)

-  Dockerizar y desplegar aplicaciones Java en producción
-  Configurar monitoring y observabilidad enterprise

Herramientas & Productividad

-  Gestionar proyectos y dependencias con Maven
-  Documentar APIs automáticamente con Swagger/OpenAPI
-  Desarrollar aplicaciones reactivas con WebFlux
-  Aprovechar IA para acelerar desarrollo Java

Mapa de Aprendizaje

Fase 1: Fundamentos (Módulos 1-2)

JavaScript/TypeScript → Java Fundamentals → Spring Boot Basics

- Transición de sintaxis y conceptos
- Comprensión de JVM vs Node.js runtime
- Primer proyecto Spring Boot

Fase 2: APIs y Persistencia (Módulos 3-4)

Express/NestJS APIs → Spring Boot REST → Spring Data JPA

- Desarrollo de APIs REST enterprise
- Integración con PostgreSQL
- Comparación con TypeORM

Fase 3: Seguridad y Arquitectura (Módulos 5-6)

JWT en Node.js → Spring Security → Microservicios Java

- Autenticación y autorización robusta
- Arquitectura de microservicios
- Comunicación entre servicios

Fase 4: Testing y DevOps (Módulos 7-8)

Jest/Testing → JUnit/TestContainers → CI/CD Java

- Testing strategy completa

- Containerización y deployment
- Pipeline de producción

⚡ Fase 5: Avanzado y Observabilidad (Módulos 9-10)

Herramientas → WebFlux Reactivo → Monitoring Enterprise

- Programación reactiva
- Documentación automática
- Observabilidad y performance

💧 Metodología de Aprendizaje

🤖 Desarrollo Asistido por IA

Cada módulo incluye **prompts optimizados** para herramientas como:

- **GitHub Copilot** - Generación de código Java/Spring Boot
- **Claude/ChatGPT** - Explicaciones detalladas y comparaciones
- **v0.dev** - Generación de documentación y diagramas

📖 Aprendizaje Comparativo

- **Conceptos paralelos:** JavaScript ↔ Java
- **Frameworks equivalentes:** Express/NestJS ↔ Spring Boot
- **Herramientas similares:** npm ↔ Maven, Jest ↔ JUnit

🎯 Enfoque Práctico

- **Proyectos incrementales** - Cada módulo construye sobre el anterior
- **Casos de uso reales** - Problemas que enfrentas en Node.js
- **Best practices** - Patrones enterprise probados en Java

📊 Comparación de Ecosistemas

Aspecto	Node.js/TypeScript	Java/Spring Boot
Runtime	V8 Engine, Event Loop	JVM, Multithreading
Tipado	TypeScript (opcional)	Java (nativo)
Framework	Express/NestJS	Spring Boot
ORM	TypeORM/Prisma	Spring Data JPA/Hibernate
Testing	Jest/Vitest	JUnit 5/TestContainers
Build	npm/yarn/pnpm	Maven/Gradle

Aspecto	Node.js/TypeScript	Java/Spring Boot
Deployment	Docker/Vercel/Railway	Docker/AWS/K8s
Monitoring	Winston/Pino	Logback/Micrometer

Roadmap de Módulos

Módulos Principales (10 total)

1. **Fundamentos Java** - Sintaxis, OOP, JVM vs Node.js
2. **Spring Framework** - IoC, DI, primer proyecto
3. **APIs REST** - Controladores, validación, errores
4. **Base de Datos** - JPA, PostgreSQL, migraciones
5. **Seguridad** - JWT, Spring Security, OAuth2
6. **Microservicios** - Arquitectura, comunicación
7. **Testing** - JUnit, TestContainers, estrategias
8. **DevOps** - Docker, CI/CD, deployment
9. **Herramientas** - Maven, Swagger, WebFlux
10. **Observabilidad** - Logging, metrics, monitoring

Prompts IA (14 total)

Cada módulo incluye **1-2 prompts especializados** para desarrollo asistido por IA, siguiendo el modelo exitoso del curso Frontend.

Curso desarrollado con enfoque práctico para desarrolladores Node.js/TypeScript que buscan expandir sus habilidades al ecosistema Java enterprise.

Módulo 1: Fundamentos de Java para Desarrolladores Experimentados

Objetivo: Transición efectiva de JavaScript/TypeScript a Java, enfocándose en diferencias clave y ventajas del ecosistema Java para desarrollo backend enterprise.

1.1 Introducción a Java y la JVM

Concepto

Comprende Java como lenguaje **compilado y multiplataforma**, y el rol central de la **JVM (Java Virtual Machine)** como entorno de ejecución robusto para aplicaciones enterprise.

¿Por qué es importante?

Para desarrolladores Node.js, la **JVM es un cambio fundamental**:

- **Node.js:** Interpretado en runtime por V8 Engine

- **Java:** Compilado a bytecode, ejecutado en JVM optimizada
- **Ventajas JVM:** Gestión avanzada de memoria, multithreading real, optimizaciones en runtime

 Comparación Clave: Node.js vs JVM

Aspecto	Node.js	Java/JVM
Ejecución	Interpretado (V8)	Compilado (Bytecode)
Concurrencia	Event Loop (single-thread)	Multithreading nativo
Memoria	Garbage Collection V8	GC avanzado (G1, ZGC)
Performance	Excelente I/O	Excelente procesamiento
Escalabilidad	Horizontal	Vertical + Horizontal

 Prompt para IA

```
Genera un resumen comparativo detallado entre el modelo de ejecución de Java (JVM) y Node.js, enfocándote en:
```

```
1. Diferencias en compilación vs interpretación
2. Gestión de memoria y garbage collection
3. Modelos de concurrencia (Event Loop vs Threads)
4. Implicaciones para escalabilidad y rendimiento en aplicaciones backend
5. Cuándo elegir cada uno según el caso de uso
```

```
Incluye ejemplos de código "Hello World" en ambos que demuestren estas diferencias.
```

1.2 Sintaxis y Programación Orientada a Objetos ⚡

 Concepto

Revisión de sintaxis Java y profundización en **POO avanzada**: clases, herencia, polimorfismo, encapsulamiento, interfaces y clases abstractas.

 ¿Por qué es importante?

Aunque conoces TypeScript (que soporta OOP), **Java tiene convenciones específicas**:

- **Interfaces:** Contratos más estrictos que TypeScript
- **Herencia:** Single inheritance + múltiples interfaces
- **Polimorfismo:** Más explícito que en JavaScript
- **Encapsulamiento:** Modificadores de acceso más rigurosos

 Comparación: TypeScript vs Java OOP

```
// TypeScript - Flexibilidad
interface User {
  name: string;
  email?: string; // Opcional
}

class AdminUser implements User {
  name: string;
  email: string;

  constructor(name: string, email?: string) {
    this.name = name;
    this.email = email || "";
  }
}
```

```
// Java - Estructura más rígida
public interface User {
  String getName();
  String getEmail();
}

public class AdminUser implements User {
  private final String name;
  private final String email;

  public AdminUser(String name, String email) {
    this.name = name;
    this.email = email != null ? email : "";
  }

  @Override
  public String getName() { return name; }

  @Override
  public String getEmail() { return email; }
}
```

Prompt para IA

Crea un ejemplo práctico en Java que demuestre los conceptos de:

1. Herencia: Clase base Employee y clases derivadas Developer, Manager
2. Polimorfismo: Método calculateSalary() implementado diferente en cada clase
3. Interfaces: Interface Payable con método processPayment()
4. Encapsulamiento: Campos privados con getters/setters apropiados
5. Clase abstracta: AbstractEmployee con comportamiento común

Escenario: Sistema de gestión de empleados de una empresa tech.
Explica cómo estos conceptos crean una arquitectura flexible y mantenible.
Compara con cómo implementarías lo mismo en TypeScript/JavaScript.

1.3 Colecciones y APIs Modernas de Java

Concepto

Dominio de **Collections Framework** (List, Set, Map), **Streams API** para procesamiento funcional, y manejo de I/O básico.

¿Por qué es importante?

Las colecciones son el equivalente a **arrays y objects en JavaScript**, pero con:

- **Tipado fuerte:** Generics para type safety
- **Performance optimizada:** Diferentes implementaciones según uso
- **Streams API:** Programación funcional similar a map/filter/reduce

Comparación: JavaScript vs Java Collections

```
// JavaScript - Flexible pero sin tipado
const products = [
  { name: "Laptop", price: 1200, category: "tech" },
  { name: "Phone", price: 800, category: "tech" },
  { name: "Book", price: 25, category: "education" },
];

const expensiveProducts = products
  .filter((p) => p.price > 100)
  .map((p) => p.name)
  .sort();
```

```
// Java - Tipado fuerte y expresivo
List<Product> products = Arrays.asList(
    new Product("Laptop", 1200, "tech"),
    new Product("Phone", 800, "tech"),
    new Product("Book", 25, "education")
);

List<String> expensiveProducts = products.stream()
    .filter(p -> p.getPrice() > 100)
    .map(Product::getName)
    .sorted()
    .collect(Collectors.toList());
```

 Prompt para IA

Desarrolla una función Java completa que:

1. Reciba una `List<Product>` (con propiedades: `name`, `price`, `category`, `inStock`)

2. Filtre productos en stock con precio > 50

3. Agrupe por categoría usando `Collectors.groupingBy()`

4. Ordene cada grupo alfabéticamente por nombre

5. Calcule el precio promedio por categoría

6. Devuelva un `Map<String, ProductSummary>` con resumen por categoría

Requisitos:

- Usa Streams API exclusivamente

- Incluye manejo de Optional para casos edge

- Compara la sintaxis con el equivalente en JavaScript

- Explica las ventajas del tipado fuerte en este escenario



Módulo 2: Introducción a Spring Framework y Spring Boot

Objetivo: Comprender el ecosistema Spring y dominar Spring Boot para desarrollo ágil de aplicaciones enterprise, comparando con Express/NestJS.

2.1 Spring Framework vs Spring Boot

 Concepto

Spring Framework es el ecosistema completo para Java enterprise. **Spring Boot** es una capa que simplifica la creación de aplicaciones "production-ready" con configuración mínima.

 ¿Por qué es importante?

Para desarrolladores Node.js/TypeScript, Spring Boot es comparable a:

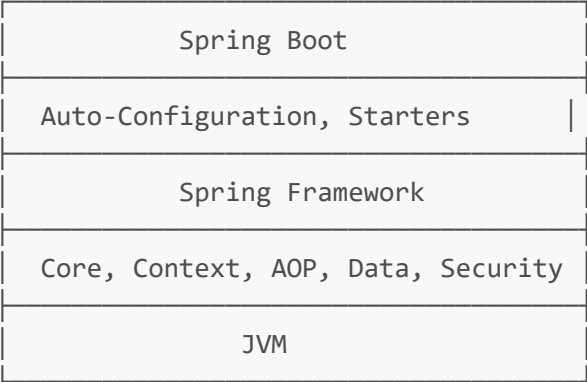
- **Express:** Minimalista, pero Spring Boot incluye más "out of the box"
- **NestJS:** Estructura opinionada similar, pero Spring Boot es más maduro
- **Fastify:** Performance focus, pero Spring Boot es más enterprise-oriented

 Comparación de Ecosistemas

Característica	Express	NestJS	Spring Boot
Filosofía	Minimalista	Opinionado	Convención sobre configuración
Configuración	Manual	Decoradores	Auto-configuración
DI Container	No nativo	Sí (similar Angular)	Sí (maduro)

Característica	Express	NestJS	Spring Boot
Testing	Jest (externo)	Jest integrado	JUnit integrado
ORM	TypeORM/Prisma	TypeORM integrado	JPA/Hibernate nativo
Security	Passport/custom	Guards/Guards	Spring Security
Production	PM2/Docker	Docker	JAR ejecutable

Arquitectura Spring Boot



Prompt para IA

Explica la relación entre Spring Framework y Spring Boot de manera clara:

1. ¿Qué problemas resuelve Spring Framework en el desarrollo Java?

2. ¿Cómo Spring Boot simplifica Spring Framework?

3. Comparación detallada con Express y NestJS:

- Filosofía de desarrollo
- Configuración vs convención
- Ecosistema y madurez
- Curva de aprendizaje

4. Crea una tabla comparativa de características clave

5. ¿Cuándo elegir Spring Boot vs NestJS para un proyecto nuevo?

Incluye ejemplos de código que muestren la diferencia en configuración.

2.2 Inversión de Control (IoC) e Inyección de Dependencias (DI)

Concepto

IoC: El framework controla la creación y gestión de objetos **DI:** Las dependencias se "inyectan" automáticamente donde se necesitan **Beans:** Objetos gestionados por el contenedor de Spring

🎯 ¿Por qué es importante?

La DI es el **corazón de Spring** y es comparable a:

- **NestJS:** `@Injectable()` y dependency injection
- **Angular:** Service injection similar
- **Node.js puro:** Requires manuales vs injection automática

📦 Comparación: Node.js vs Spring DI

```
// NestJS - Dependency Injection
@Inject()
export class UserService {
  constructor(private userRepository: UserRepository, private emailService:
EmailService) {}
}

@Controller("users")
export class UserController {
  constructor(private userService: UserService) {}
}
```

```
// Spring Boot - Dependency Injection
@Service
public class UserService {
  private final UserRepository userRepository;
  private final EmailService emailService;

  public UserService(UserRepository userRepository,
                    EmailService emailService) {
    this.userRepository = userRepository;
    this.emailService = emailService;
  }
}

@RestController
@RequestMapping("/users")
public class UserController {
  private final UserService userService;

  public UserController(UserService userService) {
    this.userService = userService;
  }
}
```

🔍 Anotaciones Clave de Spring

Anotación	Propósito	Equivalente NestJS
-----------	-----------	--------------------

Anotación	Propósito	Equivalente NestJS
<code>@Service</code>	Lógica de negocio	<code>@Injectable()</code>
<code>@Repository</code>	Acceso a datos	Repository pattern
<code>@Controller</code>	Web MVC	<code>@Controller()</code>
<code>@RestController</code>	REST API	<code>@Controller()</code>
<code>@Component</code>	Bean genérico	<code>@Injectable()</code>
<code>@Autowired</code>	Inyección (opcional)	Constructor injection

Prompt para IA

Genera un ejemplo completo de Dependency Injection en Spring Boot que incluya:

1. Servicio de usuario (UserService) con lógica de negocio
2. Repositorio de usuario (UserRepository) para acceso a datos
3. Servicio de email (EmailService) para notificaciones
4. Controlador REST (UserController) que use el servicio

Requisitos:

- Usa constructor injection (no `@Autowired` en campos)
- Incluye interfaces para better testability
- Demuestra el concepto de `@Primary` cuando hay múltiples implementaciones
- Explica cómo Spring resuelve las dependencias automáticamente
- Compara con la implementación manual que harías en Express/Node.js
- Muestra las ventajas de DI para testing (mocking)

2.3 Creación del Primer Proyecto Spring Boot

Concepto

Uso de **Spring Initializr** para bootstrap de proyectos y comprensión de la estructura estándar de un proyecto Spring Boot.

¿Por qué es importante?

Spring Initializr es el equivalente a:

- `npm create` o `npx create-react-app` en Node.js
- `nest new` en NestJS
- Pero con **configuración enterprise** incluida

Estructura de Proyecto Spring Boot

```
my-spring-app/
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   ├── com/example/myapp/
│   │   │   │   ├── MyAppApplication.java      # Entry point
│   │   │   │   ├── controller/               # REST controllers
│   │   │   │   ├── service/                  # Business logic
│   │   │   │   ├── repository/              # Data access
│   │   │   │   └── model/                    # Entities/DTOs
│   │   └── resources/
│   │       ├── application.properties        # Configuration
│   │       └── static/                      # Static files
│   └── test/                                # Tests
├── target/                                  # Build output
├── pom.xml                                  # Maven config
└── README.md
```

Comparación: Node.js vs Spring Boot Project

```
// package.json (Node.js)
{
  "name": "my-node-app",
  "scripts": {
    "start": "node dist/main.js",
    "dev": "nodemon src/main.ts",
    "build": "tsc"
  },
  "dependencies": {
    "express": "^4.18.0",
    "typeorm": "^0.3.0"
  }
}
```

```
<!-- pom.xml (Spring Boot) -->
<project>
  <groupId>com.example</groupId>
  <artifactId>my-spring-app</artifactId>
  <version>1.0.0</version>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
  </dependencies>
</project>
```

```
    </dependency>
  </dependencies>
</project>
```

Primer Código: Hello World Comparison

```
// NestJS Hello World
@Controller()
export class AppController {
  @Get()
  getHello(): string {
    return "Hello World!";
  }
}

@Module({
  controllers: [AppController],
})
export class AppModule {}
```

```
// Spring Boot Hello World
@RestController
public class HelloController {

  @GetMapping("/")
  public String hello() {
    return "Hello World!";
  }
}

@SpringBootApplication
public class MyAppApplication {
  public static void main(String[] args) {
    SpringApplication.run(MyAppApplication.class, args);
  }
}
```

Prompt para IA

Genera una guía paso a paso completa para crear el primer proyecto Spring Boot:

1. Configuración en Spring Initializr:
 - Group: com.example
 - Artifact: user-api
 - Dependencies: Spring Web, Spring Data JPA, PostgreSQL Driver
2. Estructura del proyecto resultante:

- Explica el propósito de cada directorio
 - Compara con estructura de proyecto NestJS
 - Describe los archivos de configuración clave
3. Implementación de primer endpoint:
- Controller simple con GET, POST endpoints
 - DTO para request/response
 - Service layer básico
4. Configuración de desarrollo:
- application.properties para PostgreSQL
 - Perfil de desarrollo vs producción
 - Hot reload equivalente a nodemon
5. Comandos Maven equivalentes a npm scripts:
- Compilar, ejecutar, testing, packaging

Incluye comparaciones constantes con el workflow de Node.js/NestJS.

Módulo 3: Desarrollo de APIs RESTful con Spring Boot

Objetivo: Construir APIs REST robustas y escalables, comparando patrones y best practices con Express/NestJS para crear servicios enterprise-grade.

3.1 Controladores REST y Buenas Prácticas

Concepto

Creación de **controladores REST** con `@RestController`, mapeo de solicitudes HTTP, manejo de parámetros, y aplicación de principios RESTful para APIs intuitivas y escalables.

¿Por qué es importante?

Para desarrolladores de Node.js/Express/NestJS, los controladores Spring Boot ofrecen:

- **Estructura similar a NestJS** con decoradores/anotaciones
- **Validación automática** integrada (vs middleware manual en Express)
- **Serialización JSON** automática (vs manual en Node.js)
- **Exception handling** centralizado y robusto

Comparación: Express vs NestJS vs Spring Boot

```
// Express - Manual y flexible
app.get("/api/users/:id", async (req, res) => {
  try {
    const user = await userService.findById(req.params.id);
```

```

        res.json(user);
    } catch (error) {
        res.status(404).json({ error: "User not found" });
    }
});

app.post("/api/users", async (req, res) => {
    try {
        const user = await userService.create(req.body);
        res.status(201).json(user);
    } catch (error) {
        res.status(400).json({ error: error.message });
    }
});

```

```

// NestJS - Estructurado con decoradores
@Controller("users")
export class UserController {
    constructor(private userService: UserService) {}

    @Get(":id")
    async findOne(@Param("id") id: string): Promise<User> {
        return this.userService.findById(id);
    }

    @Post()
    async create(@Body() createUserDto: CreateUserDto): Promise<User> {
        return this.userService.create(createUserDto);
    }
}

```

```

// Spring Boot - Enterprise y robusto
@RestController
@RequestMapping("/api/users")
public class UserController {
    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    @GetMapping("/{id}")
    public ResponseEntity<UserDto> findOne(@PathVariable String id) {
        UserDto user = userService.findById(id);
        return ResponseEntity.ok(user);
    }

    @PostMapping
    public ResponseEntity<UserDto> create(@Valid @RequestBody CreateUserDto dto) {

```

```
        UserDto user = userService.create(dto);
        return ResponseEntity.status(HttpStatus.CREATED).body(user);
    }
}
```

Principios RESTful en Spring Boot

HTTP Method	Endpoint	Propósito	Spring Annotation
GET	/api/users	Listar todos	@GetMapping
GET	/api/users/{id}	Obtener uno	@GetMapping("/{id}")
POST	/api/users	Crear nuevo	@PostMapping
PUT	/api/users/{id}	Actualizar completo	@PutMapping("/{id}")
PATCH	/api/users/{id}	Actualizar parcial	@PatchMapping("/{id}")
DELETE	/api/users/{id}	Eliminar	@DeleteMapping("/{id}")

Prompt para IA

Crea un controlador REST completo en Spring Boot para una entidad 'Task' (tarea) que incluya:

1. CRUD completo con endpoints RESTful apropiados

2. DTOs separados para request y response (CreateTaskDto, UpdateTaskDto, TaskDto)

3. Validaciones usando Bean Validation (@NotNull, @Size, @Email, etc.)

4. Manejo de parámetros de consulta para filtrado (?status=completed&assignee=john)

5. Paginación y ordenamiento (PageRequest, Sort)

6. ResponseEntity con códigos HTTP apropiados

7. Documentación con comentarios para Swagger

Entidad Task debe tener: id, title, description, status (PENDING/IN_PROGRESS/COMPLETED), assignee, dueDate, createdAt.

Compara la implementación con cómo harías lo mismo en Express y NestJS, destacando las ventajas de cada enfoque.

3.2 Manejo de Errores y Validaciones

Concepto

Implementación de **manejo global de errores** con `@ControllerAdvice` y **validación automática** de datos con JSR 303 (Bean Validation) para APIs robustas y consistentes.

¿Por qué es importante?

La robustez de una API enterprise depende de:

- **Error handling consistente** - Respuestas uniformes vs errores ad-hoc
- **Validación automática** - Menos código boilerplate vs validación manual
- **Logging estructurado** - Trazabilidad vs console.log dispersos
- **Security by default** - Protección automática vs implementación manual

Comparación: Manejo de Errores

```
// Express - Manual en cada endpoint
app.post("/api/users", async (req, res) => {
  try {
    // Validación manual
    if (!req.body.email) {
      return res.status(400).json({
        error: "Email is required",
        code: "VALIDATION_ERROR",
      });
    }

    const user = await userService.create(req.body);
    res.status(201).json(user);
  } catch (error) {
    console.error(error);
    res.status(500).json({
      error: "Internal server error",
      code: "INTERNAL_ERROR",
    });
  }
});
```

```
// NestJS - Con pipes y filters
@Controller("users")
export class UserController {
  @Post()
  @UsePipes(new ValidationPipe())
  async create(@Body() createUserDto: CreateUserDto): Promise<User> {
    return this.userService.create(createUserDto);
  }
}

// Global exception filter
@Catch()
export class AllExceptionsFilter implements ExceptionFilter {
  catch(exception: unknown, host: ArgumentsHost) {
    // Handle all exceptions globally
  }
}
```

```
// Spring Boot - Automático y centralizado
@RestController
@RequestMapping("/api/users")
public class UserController {

    @PostMapping
    public ResponseEntity<UserDto> create(@Valid @RequestBody CreateUserDto dto) {
        // Validación automática con @Valid
        // Exception handling automático con @ControllerAdvice
        UserDto user = userService.create(dto);
        return ResponseEntity.status(HttpStatus.CREATED).body(user);
    }
}

// Global exception handler
@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidationException(
        MethodArgumentNotValidException ex) {

        Map<String, String> errors = new HashMap<>();
        ex.getBindingResult().getFieldErrors().forEach(error ->
            errors.put(error.getField(), error.getDefaultMessage())
        );

        ErrorResponse errorResponse = ErrorResponse.builder()
            .message("Validation failed")
            .code("VALIDATION_ERROR")
            .details(errors)
            .timestamp(Instant.now())
            .build();

        return ResponseEntity.badRequest().body(errorResponse);
    }

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFoundException(
        ResourceNotFoundException ex) {

        ErrorResponse errorResponse = ErrorResponse.builder()
            .message(ex.getMessage())
            .code("RESOURCE_NOT_FOUND")
            .timestamp(Instant.now())
            .build();

        return ResponseEntity.notFound().build();
    }
}
```

 Validaciones con Bean Validation

```
// DTO con validaciones automáticas
public class CreateUserDto {

    @NotBlank(message = "Name is required")
    @Size(min = 2, max = 50, message = "Name must be between 2 and 50 characters")
    private String name;

    @NotBlank(message = "Email is required")
    @Email(message = "Email should be valid")
    private String email;

    @NotNull(message = "Age is required")
    @Min(value = 18, message = "Age must be at least 18")
    @Max(value = 120, message = "Age must be less than 120")
    private Integer age;

    @Pattern(regexp = "^\\+?[1-9]\\d{1,14}$", message = "Phone number is invalid")
    private String phone;

    // Getters and setters...
}
```

 Ventajas del Manejo Centralizado

Aspecto	Express Manual	NestJS	Spring Boot
Setup	Código en cada endpoint	Guards/Pipes/Filters	@ControllerAdvice automático
Consistencia	Depende del developer	Estructura enforced	Automática
Validación	Manual o middleware	Class-validator	Bean Validation
Logging	Manual	Interceptors	AOP automático
Testing	Mock cada handler	TestingModule	MockMvc

 Prompt para IA

Genera un sistema completo de manejo de errores y validaciones para Spring Boot que incluya:

1. GlobalExceptionHandler (@ControllerAdvice) que maneje:

- MethodArgumentNotValidException (errores de validación)

- ResourceNotFoundException (recursos no encontrados)

- DataIntegrityViolationException (errores de DB)

- RuntimeException (errores generales)

- AccessDeniedException (errores de seguridad)

2. Clase ErrorResponse estándar con:

- message, code, details, timestamp, path
 - Builder pattern para construcción fluida
3. DTOs con validaciones completas para User entity:
 - CreateUserDto, UpdateUserDto, ChangePasswordDto
 - Validaciones custom (@UniqueEmail, @StrongPassword)
 4. Custom exceptions específicas del dominio:
 - UserNotFoundException, EmailAlreadyExistsException
 5. Logging estructurado con SLF4J:
 - Log de errores con contexto
 - Correlation IDs para trazabilidad

Compara con implementation en Express/NestJS y explica las ventajas del enfoque Spring Boot.

3.3 Interacción con APIs de Terceros 🌐

📖 Concepto

Consumo de servicios externos usando **RestTemplate** (sincrónico) y **WebClient** (reactivo) para integración con APIs de terceros, microservicios y servicios cloud.

🔗 ¿Por qué es importante?

Las aplicaciones enterprise requieren integración con:

- **APIs de terceros** - Pagos, emails, notificaciones, analytics
- **Microservicios internos** - Comunicación entre servicios
- **Servicios cloud** - AWS, Azure, GCP APIs
- **Legacy systems** - Integración con sistemas existentes

🔗 Comparación: HTTP Clients

```
// Node.js con axios
const axios = require("axios");

class UserService {
  async getUserProfile(userId) {
    try {
      const response = await axios.get(`https://api.external.com/users/${userId}`,
    {
      headers: {
        Authorization: `Bearer ${this.apiKey}`,
        "Content-Type": "application/json",
      },
      timeout: 5000,
    });
      return response.data;
    }
  }
}
```

```

    } catch (error) {
        if (error.code === "ECONNABORTED") {
            throw new Error("Request timeout");
        }
        throw new Error(`API Error: ${error.response?.status}`);
    }
}
}

```

```

// Spring Boot con RestTemplate (Sincrónico)
@Service
public class UserService {
    private final RestTemplate restTemplate;
    private final String apiKey;

    public UserService(RestTemplate restTemplate,
                      @Value("${external.api.key}") String apiKey) {
        this.restTemplate = restTemplate;
        this.apiKey = apiKey;
    }

    public UserProfileDto getUserProfile(String userId) {
        HttpHeaders headers = new HttpHeaders();
        headers.setBearerAuth(apiKey);
        headers.setContentType(MediaType.APPLICATION_JSON);

        HttpEntity<String> entity = new HttpEntity<>(headers);

        try {
            ResponseEntity<UserProfileDto> response = restTemplate.exchange(
                "https://api.external.com/users/{userId}",
                HttpMethod.GET,
                entity,
                UserProfileDto.class,
                userId
            );

            return response.getBody();
        } catch (HttpClientErrorException e) {
            throw new ExternalApiException("User not found", e);
        } catch (ResourceAccessException e) {
            throw new ExternalApiException("API timeout", e);
        }
    }
}

```

```

// Spring Boot con WebClient (Reactivo)
@Service
public class UserService {

```

```

private final WebClient webClient;

public UserService(WebClient.Builder webClientBuilder,
                  @Value("${external.api.key}") String apiKey) {
    this.webClient = webClientBuilder
        .baseUrl("https://api.external.com")
        .defaultHeader("Authorization", "Bearer " + apiKey)
        .build();
}

public Mono<UserProfileDto> getUserProfile(String userId) {
    return webClient.get()
        .uri("/users/{userId}", userId)
        .retrieve()
        .onStatus(HttpStatus::is4xxClientError, response ->
            Mono.error(new UserNotFoundException("User not found")))
        .onStatus(HttpStatus::is5xxServerError, response ->
            Mono.error(new ExternalApiException("API server error")))
        .bodyToMono(UserProfileDto.class)
        .timeout(Duration.ofSeconds(5))
        .retry(3);
}
}

```

⚙️ Configuración de HTTP Clients

```

// RestTemplate Configuration
@Configuration
public class RestTemplateConfig {

    @Bean
    public RestTemplate restTemplate() {
        HttpComponentsClientHttpRequestFactory factory =
            new HttpComponentsClientHttpRequestFactory();
        factory.setConnectTimeout(5000);
        factory.setReadTimeout(10000);

        RestTemplate restTemplate = new RestTemplate(factory);

        // Error handler personalizado
        restTemplate.setErrorHandler(new CustomResponseErrorHandler());

        // Interceptors para logging
        restTemplate.getInterceptors().add(new LoggingInterceptor());

        return restTemplate;
    }
}

// WebClient Configuration
@Configuration

```

```
public class WebClientConfig {

    @Bean
    public WebClient.Builder webClientBuilder() {
        return WebClient.builder()
            .clientConnector(new ReactorClientHttpConnector(
                HttpClient.create()
                    .option(ChannelOption.CONNECT_TIMEOUT_MILLIS, 5000)
                    .responseTimeout(Duration.ofSeconds(10))
            ))
            .filter(ExchangeFilterFunction.ofRequestProcessor(
                clientRequest -> {
                    log.info("Request: {} {}",
                        clientRequest.method(),
                        clientRequest.url());
                    return Mono.just(clientRequest);
                }
            ));
    }
}
```

 Patrones de Integración

Patrón	Uso	Implementation
Circuit Breaker	Fallos en cascada	@CircuitBreaker (Resilience4j)
Retry	Errores transitorios	@Retryable o WebClient.retry()
Timeout	Requests lentos	Configuration + timeout handling
Bulkhead	Aislamiento de recursos	Thread pools separados
Cache	Respuestas frecuentes	@Cacheable (Spring Cache)

 Prompt para IA

Crea un servicio completo de integración con APIs externas que incluya:

1. PaymentService que integre con Stripe API:

- Crear payment intent

- Confirmar pago

- Obtener status de pago

- Webhook handling para eventos

2. EmailService que integre con SendGrid:

- Enviar emails transaccionales

- Templates dinámicos

- Tracking de delivery status

3. Configuración robusta:

- RestTemplate con connection pooling

```
- WebClient con backpressure handling
- Circuit breaker con Resilience4j
- Retry policy con exponential backoff
- Request/response logging

4. Error handling específico:
- Timeout exceptions
- Rate limiting (429)
- API key invalid (401)
- Service unavailable (503)

5. Testing:
- MockWebServer para integration tests
- @MockBean para unit tests
- Contract testing con WireMock

Compara las ventajas de RestTemplate vs WebClient vs fetch/axios en Node.js.
```

Módulo 4: Gestión de Bases de Datos con Spring Data JPA y PostgreSQL

Objetivo: Dominar la persistencia de datos en Java enterprise, comparando JPA/Hibernate con TypeORM y aprendiendo patrones avanzados de acceso a datos.

4.1 Introducción a JDBC, JPA y Hibernate

Concepto

JDBC es la base para conectividad con bases de datos en Java. **JPA** (Java Persistence API) es la especificación estándar para ORMs. **Hibernate** es la implementación más popular de JPA, equivalente a TypeORM pero más maduro.

¿Por qué es importante?

Para desarrolladores TypeScript/TypeORM, entender la jerarquía Java es crucial:

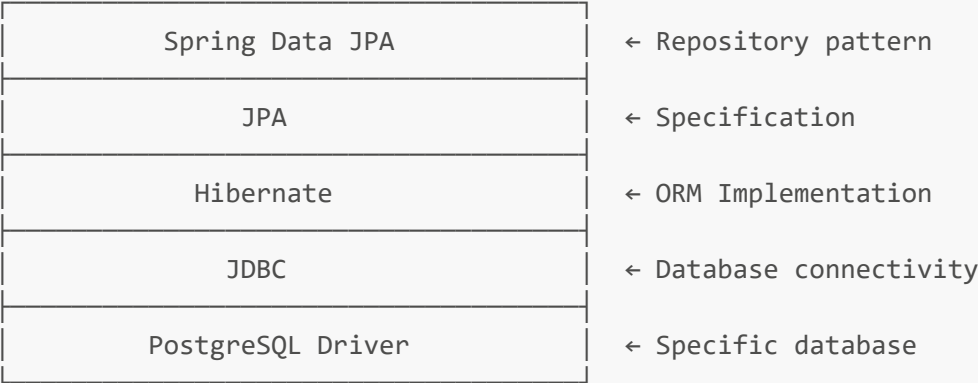
- **JDBC ↔ Database drivers** en Node.js (pg, mysql2)
- **JPA ↔ ORM specification** (como ActiveRecord pattern)
- **Hibernate ↔ TypeORM** pero con 20+ años de madurez enterprise

Comparación: Ecosistemas de Persistencia

Aspecto	Node.js/TypeORM	Java/JPA/Hibernate
Madurez	~5 años, comunidad activa	20+ años, ultra estable
Performance	Buena, optimizaciones limitadas	Excelente, lazy loading avanzado
Migrations	TypeORM migrations	Flyway/Liquibase (más robustos)

Aspecto	Node.js/TypeORM	Java/JPA/Hibernate
Caching	Redis manual	L1/L2 cache automático
Relations	Decoradores (@OneToMany)	Anotaciones (@OneToMany)
Queries	QueryBuilder + raw SQL	JPQL + Criteria API + native
Enterprise	Limitado	Connection pooling, JTA, etc.

📚 Arquitectura de Persistencia Java



📄 Código Comparativo: Entity Definition

```
// TypeORM Entity
@Entity()
export class User {
  @PrimaryGeneratedColumn("uuid")
  id: string;

  @Column({ unique: true })
  email: string;

  @Column()
  name: string;

  @CreateDateColumn()
  createdAt: Date;

  @OneToMany(() => Post, (post) => post.author)
  posts: Post[];
}

// JPA/Hibernate Entity
@Entity
@Table(name = "users")
```

```
public class User {  
    @Id  
    @GeneratedValue(strategy = GenerationType.UUID)  
    private UUID id;  
  
    @Column(unique = true, nullable = false)  
    private String email;  
  
    @Column(nullable = false)  
    private String name;  
  
    @CreationTimestamp  
    private LocalDateTime createdAt;  
  
    @OneToMany(mappedBy = "author", cascade = CascadeType.ALL, fetch =  
FetchType.LAZY)  
    private List<Post> posts = new ArrayList<>();  
  
    // Constructors, getters, setters...  
}
```

⚡ Ventajas de JPA/Hibernate sobre TypeORM

1. 🚀 **Performance:** L1/L2 caching automático, batch processing
2. 📦 **Lazy Loading:** Más sofisticado y confiable
3. 🔍 **Query Optimization:** Query plan caching, statistics
4. 📄 **Schema Management:** DDL generation más robusta
5. 🔒 **Transactions:** JTA para transacciones distribuidas
6. 📊 **Monitoring:** JMX metrics, query statistics
7. 🌐 **Standards:** JPA es estándar, multiple implementations

🤖 Prompt para IA

Crea una comparación exhaustiva entre TypeORM y JPA/Hibernate que incluya:

1. Análisis de arquitectura:
 - Capas de abstracción de cada ecosistema
 - Ventajas y desventajas arquitectónicas
 - Casos de uso ideales para cada uno
2. Comparación de features:
 - Entity mapping y relationships
 - Query capabilities (Query Builder vs JPQL vs Criteria API)
 - Caching strategies (Redis manual vs L1/L2 automático)
 - Migration handling (TypeORM vs Flyway)
 - Performance characteristics
3. Implementación práctica:
 - Define la misma entity (User con Posts) en ambos
 - Muestra consultas complejas en ambos

- Manejo de transacciones en ambos
4. Ecosystem maturity:
 - Community support y long-term viability
 - Enterprise features comparison
 - Learning curve y documentation quality
 5. Migration strategy:
 - Cómo migrar de TypeORM a JPA/Hibernate
 - Herramientas y best practices
 - Timeline y effort estimation

4.2 Spring Data JPA y Repositorios

Concepto

Spring Data JPA simplifica el acceso a datos implementando automáticamente interfaces de repositorio. Reduce drásticamente el código boilerplate comparado con DAOs tradicionales o repositorios TypeORM.

¿Por qué es importante?

Spring Data JPA ofrece ventajas significativas:

- **Query methods automáticos** - Generados por nombres de método
- **Custom queries** - JPQL y SQL nativo cuando se necesita
- **Paginación built-in** - PageRequest y Sort automáticos
- **Auditing automático** - CreatedDate, LastModified, etc.
- **Projections** - DTOs automáticos para performance

Comparación: Repository Patterns

```
// TypeORM Repository
@Inject()
export class UserRepository extends Repository<User> {
  constructor(
    @InjectRepository(User)
    private repository: Repository<User>
  ) {
    super();
  }

  async findByEmail(email: string): Promise<User | null> {
    return this.repository.findOne({ where: { email } });
  }

  async findActiveUsers(): Promise<User[]> {
    return this.repository
      .createQueryBuilder("user")
      .where("user.isActive = :isActive", { isActive: true })
```

```

        .orderBy("user.createdAt", "DESC")
        .getMany();
    }

    async findUsersWithPosts(page: number, limit: number): Promise<[User[], number]>
    {
        return this.repository
            .createQueryBuilder("user")
            .leftJoinAndSelect("user.posts", "posts")
            .skip(page * limit)
            .take(limit)
            .getManyAndCount();
    }
}

```

```

// Spring Data JPA Repository
@Repository
public interface UserRepository extends JpaRepository<User, UUID> {

    // Query method automático - no código necesario!
    Optional<User> findByEmail(String email);

    // Query method con múltiples criterios
    List<User> findByIsActiveTrueOrderByCreatedAtDesc();

    // Query method con paginación automática
    Page<User> findByIsActive(boolean isActive, Pageable pageable);

    // Custom query con JPQL
    @Query("SELECT u FROM User u LEFT JOIN FETCH u.posts WHERE u.isActive = true")
    List<User> findActiveUsersWithPosts();

    // Native SQL cuando sea necesario
    @Query(value = "SELECT * FROM users WHERE created_at > ?1", nativeQuery =
true)
    List<User> findUsersCreatedAfter(LocalDate date);

    // Projection para performance
    @Query("SELECT new com.example.dto.UserSummaryDto(u.id, u.name, u.email) FROM
User u")
    List<UserSummaryDto> findUserSummaries();

    // Modifying queries
    @Modifying
    @Query("UPDATE User u SET u.isActive = false WHERE u.lastLoginAt < ?1")
    int deactivateInactiveUsers(LocalDate cutoffDate);
}

```

Feature	TypeORM	Spring Data JPA
Query Methods	Manual QueryBuilder	Automático por nombre
Paginación	Manual skip/take	PageRequest built-in
Sorting	Manual ORDER BY	Sort parameter automático
Projections	Manual select	Interface/Class projections
Auditing	Manual @CreateDateColumn	@CreatedDate automático
Custom Queries	@Query decorator	@Query annotation
Bulk Operations	QueryBuilder update	@Modifying automático

Query Methods Automáticos

```
public interface UserRepository extends JpaRepository<User, UUID> {

    // Equivalentes automáticos a queries complejas TypeORM
    List<User> findByNameContainingIgnoreCase(String name);
    // SELECT * FROM users WHERE LOWER(name) LIKE LOWER(CONCAT('%', ?1, '%'))

    List<User> findByAgeGreaterThanAndIsActiveTrue(int age);
    // SELECT * FROM users WHERE age > ?1 AND is_active = true

    List<User> findTop5ByOrderByCreatedAtDesc();
    // SELECT * FROM users ORDER BY created_at DESC LIMIT 5

    boolean existsByEmail(String email);
    // SELECT COUNT(*) > 0 FROM users WHERE email = ?1

    long countByIsActiveTrue();
    // SELECT COUNT(*) FROM users WHERE is_active = true

    @Query("SELECT u FROM User u WHERE u.name LIKE %:name% AND u.age BETWEEN :minAge AND :maxAge")
    List<User> findByNameAndAgeBetween(@Param("name") String name,
                                       @Param("minAge") int minAge,
                                       @Param("maxAge") int maxAge);
}
```

Paginación y Sorting Avanzado

```
@Service
public class UserService {
    private final UserRepository userRepository;

    public Page<UserDto> getUsers(int page, int size, String sortBy, String
sortByDir) {
```

```

        Sort sort = Sort.by(Sort.Direction.fromString(sortDir), sortBy);
        Pageable pageable = PageRequest.of(page, size, sort);

        Page<User> users = userRepository.findByIsActiveTrue(pageable);

        return users.map(this::convertToDto); // Automatic DTO conversion
    }

    public Page<UserDto> searchUsers(UserSearchCriteria criteria, Pageable
pageable) {
        return userRepository.findAll(
            UserSpecification.withCriteria(criteria),
            pageable
        ).map(this::convertToDto);
    }
}

```

Prompt para IA

Diseña un sistema completo de repositorios Spring Data JPA para un e-commerce que incluya:

1. Entities principales:
 - User (con roles: CUSTOMER, ADMIN)
 - Product (con categorías y inventory)
 - Order (con OrderItems)
 - Review (ratings y comentarios)
2. Repository interfaces con query methods automáticos:
 - Búsqueda de productos por categoría, precio, disponibilidad
 - Órdenes por usuario, fecha, estado
 - Reviews por producto con ratings
 - Usuarios activos, inactivos, por rol
3. Custom queries avanzadas:
 - Top productos por ventas
 - Revenue por período
 - Usuarios con más órdenes
 - Productos con mejor rating
4. Specifications para búsqueda dinámica:
 - ProductSpecification con filtros múltiples
 - OrderSpecification por fecha, estado, monto
 - Combinación de criterios con AND/OR
5. Projections para performance:
 - ProductSummaryDto para listados
 - OrderSummaryDto para dashboard
 - UserStatsDto para analytics
6. Auditing automático:

- CreatedDate, LastModifiedDate
- CreatedBy, LastModifiedBy
- Soft delete implementation

Compara cada feature con la implementación equivalente en TypeORM y explica las ventajas de Spring Data JPA.

📁 Módulo 5: Seguridad y Autenticación

🔒 Conceptos Fundamentales de Seguridad

En el mundo Node.js/TypeScript, manejas la seguridad principalmente con librerías como `passport`, `jsonwebtoken`, `bcrypt`. Spring Security ofrece un framework integral y más robusto.

🔍 Comparación: Node.js vs Java

Concepto	Node.js/TypeScript	Java/Spring Boot
Middleware de Auth	<code>passport.js</code> , custom middleware	<code>@EnableWebSecurity</code> , Security Filter Chain
Hash de Passwords	<code>bcrypt.hash()</code>	<code>PasswordEncoder</code> (BCrypt, SCrypt, Argon2)
JWT Tokens	<code>jsonwebtoken</code> library	<code>spring-security-oauth2-jose</code>
Session Management	<code>express-session</code>	Spring Session, <code>@SessionScope</code>
CSRF Protection	<code>csrf</code> middleware	<code>csrf()</code> built-in
CORS	<code>cors</code> middleware	<code>@CrossOrigin</code> , <code>CorsConfiguration</code>

🏗️ Arquitectura de Spring Security

```
// Configuración básica de seguridad
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        return http
            .csrf(csrf -> csrf.disable())
```

```

        .cors(cors -> cors.configurationSource(corsConfigurationSource()))
        .sessionManagement(session ->
            session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/auth/**").permitAll()
            .requestMatchers("/api/public/**").permitAll()
            .requestMatchers(HttpMethod.GET, "/api/products/**").permitAll()
            .requestMatchers("/api/admin/**").hasRole("ADMIN")
            .anyRequest().authenticated())
        .oauth2ResourceServer(oauth2 -> oauth2
            .jwt(jwt -> jwt.jwtDecoder(jwtDecoder()))))
        .build();
    }
}

```

Comparación con Express.js:

```

// Node.js/Express equivalente
import passport from "passport";
import { Strategy as JwtStrategy } from "passport-jwt";

app.use(passport.initialize());

// Middleware de autenticación
const authMiddleware = (req: Request, res: Response, next: NextFunction) => {
    const token = req.headers.authorization?.split(" ")[1];

    if (!token) {
        return res.status(401).json({ error: "No token provided" });
    }

    try {
        const decoded = jwt.verify(token, process.env.JWT_SECRET!);
        req.user = decoded;
        next();
    } catch (error) {
        res.status(401).json({ error: "Invalid token" });
    }
};

// Rutas protegidas
app.get("/api/products", authMiddleware, getProducts);
app.get("/api/admin/*", requireAdmin, adminRoutes);

```

Implementación de JWT con Spring Security

Configuración JWT


```

@Component
public class JwtUtils {

    private static final String JWT_SECRET = "mySecretKey";
    private static final int JWT_EXPIRATION = 86400000; // 24 hours

    public String generateJwtToken(Authentication authentication) {
        UserPrincipal userPrincipal = (UserPrincipal)
authentication.getPrincipal();

        return Jwts.builder()
            .setSubject(userPrincipal.getUsername())
            .setIssuedAt(new Date())
            .setExpiration(new Date(System.currentTimeMillis() +
JWT_EXPIRATION))
            .signWith(SignatureAlgorithm.HS512, JWT_SECRET)
            .compact();
    }

    public String getUserNameFromJwtToken(String token) {
        return Jwts.parser()
            .setSigningKey(JWT_SECRET)
            .parseClaimsJws(token)
            .getBody()
            .getSubject();
    }

    public boolean validateJwtToken(String authToken) {
        try {
            Jwts.parser().setSigningKey(JWT_SECRET).parseClaimsJws(authToken);
            return true;
        } catch (JwtException | IllegalArgumentException e) {
            logger.error("JWT validation error: {}", e.getMessage());
        }
        return false;
    }
}

```

Filtro de Autenticación JWT

```

public class AuthTokenFilter extends OncePerRequestFilter {

    @Autowired
    private JwtUtils jwtUtils;

    @Autowired
    private UserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest request,

```

```

        HttpServletResponse response,
        FilterChain filterChain) throws
ServletException, IOException {
    try {
        String jwt = parseJwt(request);
        if (jwt != null && jwtUtils.validateJwtToken(jwt)) {
            String username = jwtUtils.getUserNameFromJwtToken(jwt);

            UserDetails userDetails =
userDetailsService.loadUserByUsername(username);
            UsernamePasswordAuthenticationToken authentication =
                new UsernamePasswordAuthenticationToken(userDetails, null,
userDetails.getAuthorities());
            authentication.setDetails(new
WebAuthenticationDetailsSource().buildDetails(request));

            SecurityContextHolder.getContext().setAuthentication(authentication);
        }
    } catch (Exception e) {
        logger.error("Cannot set user authentication: {}", e);
    }

    filterChain.doFilter(request, response);
}

private String parseJwt(HttpServletRequest request) {
    String headerAuth = request.getHeader("Authorization");

    if (StringUtils.hasText(headerAuth) && headerAuth.startsWith("Bearer ")) {
        return headerAuth.substring(7);
    }

    return null;
}
}

```

Manejo de Usuarios y Roles

Entidades de Seguridad

```

@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true)
    @NotBlank

```

```

    @Size(max = 50)
    private String username;

    @Column(unique = true)
    @Email
    @NotBlank
    @Size(max = 100)
    private String email;

    @NotBlank
    @Size(max = 120)
    private String password;

    @ManyToMany(fetch = FetchType.LAZY)
    @JoinTable(name = "user_roles",
        joinColumns = @JoinColumn(name = "user_id"),
        inverseJoinColumns = @JoinColumn(name = "role_id"))
    private Set<Role> roles = new HashSet<>();

    @CreationTimestamp
    private LocalDateTime createdAt;

    @UpdateTimestamp
    private LocalDateTime updatedAt;

    // Constructors, getters, setters
}

@Entity
@Table(name = "roles")
public class Role {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Enumerated(EnumType.STRING)
    @Column(length = 20)
    private ERole name;

    // Constructors, getters, setters
}

public enum ERole {
    ROLE_USER,
    ROLE_MODERATOR,
    ROLE_ADMIN
}

```

UserDetailsService Personalizado

```
@Service
@Transactional
public class UserDetailsServiceImpl implements UserDetailsService {

    @Autowired
    UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String username) throws
UsernameNotFoundException {
        User user = userRepository.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException("User Not Found:
" + username));

        return UserPrincipal.create(user);
    }
}

public class UserPrincipal implements UserDetails {
    private Long id;
    private String username;
    private String email;
    private String password;
    private Collection<? extends GrantedAuthority> authorities;

    public static UserPrincipal create(User user) {
        List<GrantedAuthority> authorities = user.getRoles().stream()
            .map(role -> new SimpleGrantedAuthority(role.getName().name()))
            .collect(Collectors.toList());

        return new UserPrincipal(
            user.getId(),
            user.getUsername(),
            user.getEmail(),
            user.getPassword(),
            authorities
        );
    }

    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        return authorities;
    }

    @Override
    public boolean isAccountNonExpired() { return true; }

    @Override
    public boolean isAccountNonLocked() { return true; }

    @Override
    public boolean isCredentialsNonExpired() { return true; }
```

```
@Override
public boolean isEnabled() { return true; }
}
```

Controladores de Autenticación

Auth Controller

```
@RestController
@RequestMapping("/api/auth")
@CrossOrigin(origins = "*", maxAge = 3600)
public class AuthController {

    @Autowired
    AuthenticationManager authenticationManager;

    @Autowired
    UserRepository userRepository;

    @Autowired
    RoleRepository roleRepository;

    @Autowired
    PasswordEncoder encoder;

    @Autowired
    JwtUtils jwtUtils;

    @PostMapping("/signin")
    public ResponseEntity<?> authenticateUser(@Valid @RequestBody LoginRequest
loginRequest) {

        Authentication authentication = authenticationManager.authenticate(
            new
UsernamePasswordAuthenticationToken(loginRequest.getUsername(),

loginRequest.getPassword()));

        SecurityContextHolder.getContext().setAuthentication(authentication);
        String jwt = jwtUtils.generateJwtToken(authentication);

        UserPrincipal userDetails = (UserPrincipal) authentication.getPrincipal();
        List<String> roles = userDetails.getAuthorities().stream()
            .map(item -> item.getAuthority())
            .collect(Collectors.toList());

        return ResponseEntity.ok(new JwtResponse(jwt,
                                                    userDetails.getId(),
                                                    userDetails.getUsername(),
                                                    userDetails.getEmail(),
                                                    roles));
    }
}
```

```
}

@PostMapping("/signup")
public ResponseEntity<?> registerUser(@Valid @RequestBody SignupRequest
signUpRequest) {
    if (userRepository.existsByUsername(signUpRequest.getUsername())) {
        return ResponseEntity.badRequest()
            .body(new MessageResponse("Error: Username is already
taken!"));
    }

    if (userRepository.existsByEmail(signUpRequest.getEmail())) {
        return ResponseEntity.badRequest()
            .body(new MessageResponse("Error: Email is already in use!"));
    }

    // Create new user's account
    User user = new User(signUpRequest.getUsername(),
        signUpRequest.getEmail(),
        encoder.encode(signUpRequest.getPassword()));

    Set<String> strRoles = signUpRequest.getRole();
    Set<Role> roles = new HashSet<>();

    if (strRoles == null) {
        Role userRole = roleRepository.findByName(ERole.ROLE_USER)
            .orElseThrow(() -> new RuntimeException("Error: Role is not
found."));
        roles.add(userRole);
    } else {
        strRoles.forEach(role -> {
            switch (role) {
                case "admin":
                    Role adminRole =
roleRepository.findByName(ERole.ROLE_ADMIN)
                        .orElseThrow(() -> new RuntimeException("Error:
Role is not found."));
                    roles.add(adminRole);
                    break;
                case "mod":
                    Role modRole =
roleRepository.findByName(ERole.ROLE_MODERATOR)
                        .orElseThrow(() -> new RuntimeException("Error:
Role is not found."));
                    roles.add(modRole);
                    break;
                default:
                    Role userRole = roleRepository.findByName(ERole.ROLE_USER)
                        .orElseThrow(() -> new RuntimeException("Error:
Role is not found."));
                    roles.add(userRole);
            }
        });
    }
}
```

```

        user.setRoles(roles);
        userRepository.save(user);

        return ResponseEntity.ok(new MessageResponse("User registered
successfully!"));
    }
}

```

🔒 Anotaciones de Seguridad

🔒 Method-Level Security

```

@RestController
@RequestMapping("/api/admin")
@PreAuthorize("hasRole('ADMIN')")
public class AdminController {

    @GetMapping("/users")
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<List<User>> getAllUsers() {
        List<User> users = userService.getAllUsers();
        return ResponseEntity.ok(users);
    }

    @PostMapping("/users/{userId}/ban")
    @PreAuthorize("hasRole('ADMIN') and #userId != authentication.principal.id")
    public ResponseEntity<?> banUser(@PathVariable Long userId) {
        userService.banUser(userId);
        return ResponseEntity.ok(new MessageResponse("User banned successfully"));
    }
}

@RestController
@RequestMapping("/api/user")
public class UserController {

    @GetMapping("/profile")
    @PreAuthorize("hasRole('USER') or hasRole('MODERATOR') or hasRole('ADMIN')")
    public ResponseEntity<?> getUserProfile(Authentication authentication) {
        UserPrincipal userPrincipal = (UserPrincipal)
authentication.getPrincipal();
        User user = userService.findById(userPrincipal.getId());
        return ResponseEntity.ok(user);
    }

    @PutMapping("/profile")
    @PreAuthorize("authentication.principal.id == #userId")
    public ResponseEntity<?> updateProfile(@PathVariable Long userId,
                                           @RequestBody UserUpdateRequest request) {
        User updatedUser = userService.updateUser(userId, request);
    }
}

```

```

        return ResponseEntity.ok(updatedUser);
    }
}

```

🌐 CORS y Headers de Seguridad

```

@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
            .allowedOriginPatterns("*")
            .allowedMethods("GET", "POST", "PUT", "DELETE", "PATCH",
"OPTIONS")
            .allowedHeaders("*")
            .allowCredentials(true)
            .maxAge(3600);
    }
}

@Component
public class SecurityHeadersFilter implements Filter {

    @Override
    public void doFilter(ServletRequest request, ServletResponse response,
FilterChain chain)
        throws IOException, ServletException {

        HttpServletResponse httpResponse = (HttpServletResponse) response;

        // Security headers
        httpResponse.setHeader("X-Content-Type-Options", "nosniff");
        httpResponse.setHeader("X-Frame-Options", "DENY");
        httpResponse.setHeader("X-XSS-Protection", "1; mode=block");
        httpResponse.setHeader("Strict-Transport-Security", "max-age=31536000;
includeSubDomains");
        httpResponse.setHeader("Content-Security-Policy", "default-src 'self'");

        chain.doFilter(request, response);
    }
}

```

🚀 Prompts de IA para Módulo 5: Seguridad

PROMPT GITHUB COPILOT - Configuración Spring Security:

Actúa como un experto en Spring Security. Necesito implementar un sistema de autenticación JWT completo para una aplicación e-commerce.

Genera:

1. SecurityConfig con configuración moderna:
 - JWT stateless authentication
 - CORS configurado para frontend React
 - Rutas públicas: /api/auth/**, /api/public/**
 - Rutas admin: /api/admin/** (solo ROLE_ADMIN)
 - Rate limiting para endpoints de auth
2. JwtAuthenticationFilter personalizado:
 - Validación de token JWT
 - Extracción de claims
 - Manejo de excepciones
 - Logging de seguridad
3. UserDetailsService con caché:
 - Carga de usuario desde base de datos
 - Cache con Redis para performance
 - Manejo de usuarios inactivos/bloqueados
4. Password policies avanzadas:
 - Validación de complejidad
 - Historial de passwords
 - Expiración de passwords
 - Bloqueo por intentos fallidos
5. Audit logging:
 - Login/logout events
 - Failed authentication attempts
 - Privilege escalation attempts
 - API access patterns

Incluye validaciones, manejo de errores y mejores prácticas de seguridad.

PROMPT CLAUDE - Auth Controller Avanzado:

Eres un desarrollador senior Java/Spring Boot. Necesito un AuthController empresarial completo con todas las funcionalidades modernas.

Implementa:

1. Endpoints de autenticación:
 - /signin con rate limiting
 - /signup con validación de email
 - /refresh-token para renovar JWT
 - /logout con blacklist de tokens
 - /forgot-password con token temporal
 - /reset-password con validación segura
2. Validaciones robustas:

- Email format y dominio
 - Password strength (mayús, mínús, números, símbolos)
 - Username únicos y alfanuméricos
 - CAPTCHA para signup/signin después de fallos
3. Respuestas seguras:
- No revelar información sensible
 - Mensajes genéricos para errores
 - Headers de seguridad
 - Logging de eventos importantes
4. Features empresariales:
- Two-Factor Authentication (2FA)
 - Device fingerprinting
 - Geolocation validation
 - Session management avanzado
5. Integration tests:
- TestContainers para base de datos
 - MockMvc para endpoints
 - Security testing escenarios
 - Performance testing

Compara cada implementación con las mejores prácticas de Node.js/Passport y explica las ventajas de Spring Security.

Módulo 6: Microservicios y Arquitectura Distribuida

Fundamentos de Microservicios

En el ecosistema Node.js, probablemente has trabajado con arquitecturas monolíticas o servicios separados usando Express.js. Spring Boot con Spring Cloud ofrece un ecosistema completo para microservicios.

Comparación: Node.js vs Spring Boot

Concepto	Node.js/TypeScript	Spring Boot/Cloud
Service Discovery	Consul, etcd, manual config	Eureka, Consul, Kubernetes
Load Balancing	nginx, HAProxy	Spring Cloud LoadBalancer, Ribbon
Circuit Breaker	opossum, hystrix-js	Hystrix, Resilience4j
API Gateway	Express Gateway, Kong	Spring Cloud Gateway, Zuul
Config Management	dotenv, config libs	Spring Cloud Config
Distributed Tracing	Jaeger, Zipkin manually	Spring Cloud Sleuth + Zipkin
Message Queues	RabbitMQ, Redis, SQS	Spring Cloud Stream, RabbitMQ

Arquitectura de Microservicios

```
// Service Registry (Eureka Server)
@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}

// application.yml para Eureka Server
server:
  port: 8761

eureka:
  instance:
    hostname: localhost
  client:
    registerWithEureka: false
    fetchRegistry: false
    serviceUrl:
      defaultZone: http://${eureka.instance.hostname}:${server.port}/eureka/

spring:
  application:
    name: eureka-server
```

Comparación con Node.js/Consul:

```
// Node.js con Consul para Service Discovery
import consul from "consul";

const consulClient = consul();

// Registrar servicio
async function registerService() {
    await consulClient.agent.service.register({
        id: "user-service-1",
        name: "user-service",
        address: "localhost",
        port: 3001,
        check: {
            http: "http://localhost:3001/health",
            interval: "10s",
        },
    });
}

// Descubrir servicios
async function discoverService(serviceName: string) {
    const services = await consulClient.health.service(serviceName);
}
```

```
return services[0]?.Service;
}
```

API Gateway con Spring Cloud Gateway

Configuración del Gateway

```
@SpringBootApplication
@EnableEurekaClient
public class ApiGatewayApplication {
    public static void main(String[] args) {
        SpringApplication.run(ApiGatewayApplication.class, args);
    }

    @Bean
    public RouteLocator customRouteLocator(RouteLocatorBuilder builder) {
        return builder.routes()
            .route("user-service", r -> r.path("/api/users/**")
                .filters(f -> f
                    .stripPrefix(1)
                    .addRequestHeader("X-Gateway", "Spring-Cloud-
Gateway")
                    .circuitBreaker(c -> c.name("userServiceCB")
                        .fallbackUri("forward:/fallback/users")))
                .uri("lb://user-service"))

            .route("product-service", r -> r.path("/api/products/**")
                .filters(f -> f
                    .stripPrefix(1)
                    .retry(retryConfig -> retryConfig.retries(3))
                    .requestRateLimiter(r1 -> r1
                        .setRateLimiter(redisRateLimiter())
                        .setKeyResolver(ipKeyResolver())))
                .uri("lb://product-service"))

            .route("order-service", r -> r.path("/api/orders/**")
                .and().header("Authorization")
                .filters(f -> f
                    .stripPrefix(1)
                    .modifyRequestBody(String.class, String.class,
                        (exchange, body) ->
Mono.just(enrichRequestBody(body))))
                .uri("lb://order-service"))
            .build();
    }

    @Bean
    public RedisRateLimiter redisRateLimiter() {
        return new RedisRateLimiter(10, 20, 1); // 10 requests per second, burst
20
    }
```

```

@Bean
public KeyResolver ipKeyResolver() {
    return exchange ->
        Mono.just(exchange.getRequest().getRemoteAddress().getAddress().getHostAddress());
}

```

Filtros Personalizados

```

@Component
public class AuthenticationFilter implements GlobalFilter, Ordered {

    @Autowired
    private JwtUtil jwtUtil;

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain)
    {
        ServerHttpRequest request = exchange.getRequest();

        if (isSecuredRoute(request)) {
            if (!hasAuthToken(request)) {
                return onError(exchange, "Missing authorization token",
                    HttpStatus.UNAUTHORIZED);
            }

            String token = getAuthToken(request);
            if (!jwtUtil.isValidToken(token)) {
                return onError(exchange, "Invalid authorization token",
                    HttpStatus.UNAUTHORIZED);
            }

            // Add user info to headers for downstream services
            String userId = jwtUtil.getUserIdFromToken(token);
            String userRole = jwtUtil.getUserRoleFromToken(token);

            ServerHttpRequest modifiedRequest = request.mutate()
                .header("X-User-Id", userId)
                .header("X-User-Role", userRole)
                .build();

            exchange = exchange.mutate().request(modifiedRequest).build();
        }

        return chain.filter(exchange);
    }

    private Mono<Void> onError(ServerWebExchange exchange, String err, HttpStatus
        httpStatus) {
        ServerHttpResponse response = exchange.getResponse();
    }

```

```

        response.setStatusCode(HttpStatus);
        return response.setComplete();
    }

    @Override
    public int getOrder() {
        return -1; // High priority
    }
}

@Component
public class LoggingFilter implements GlobalFilter, Ordered {

    private static final Logger logger =
        LoggerFactory.getLogger(LoggingFilter.class);

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain)
    {
        ServerHttpRequest request = exchange.getRequest();

        logger.info("Request: {} {} from {}",
            request.getMethod(),
            request.getURI(),
            request.getRemoteAddress());

        long startTime = System.currentTimeMillis();

        return chain.filter(exchange).then(Mono.fromRunnable(() -> {
            long duration = System.currentTimeMillis() - startTime;
            ServerHttpResponse response = exchange.getResponse();

            logger.info("Response: {} {} - Status: {} - Duration: {}ms",
                request.getMethod(),
                request.getURI(),
                response.getStatusCode(),
                duration);

        })));
    }

    @Override
    public int getOrder() {
        return 1;
    }
}

```

Configuración Distribuida con Spring Cloud Config

Config Server

```
@SpringBootApplication
@EnableConfigServer
@EnableEurekaClient
public class ConfigServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(ConfigServerApplication.class, args);
    }
}

// application.yml del Config Server
server:
    port: 8888

spring:
    application:
        name: config-server
    cloud:
        config:
            server:
                git:
                    uri: https://github.com/company/microservices-config
                    clone-on-start: true
                    default-label: main
                native:
                    search-locations: classpath:/config
    profiles:
        active: git

eureka:
    client:
        service-url:
            defaultZone: http://localhost:8761/eureka
```

Configuraciones por Servicio

```
# user-service.yml
server:
    port: 8081

spring:
    datasource:
        url: jdbc:postgresql://localhost:5432/userdb
        username: ${DB_USERNAME:postgres}
        password: ${DB_PASSWORD:postgres}
    jpa:
        hibernate:
            ddl-auto: validate
        show-sql: false
    redis:
        host: ${REDIS_HOST:localhost}
```

```

    port: ${REDIS_PORT:6379}

logging:
  level:
    com.company.userservice: DEBUG
    org.springframework.security: INFO

management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
  endpoint:
    health:
      show-details: always

# user-service-prod.yml
spring:
  datasource:
    url: jdbc:postgresql://${DB_HOST}:5432/userdb_prod
    hikari:
      maximum-pool-size: 20
      minimum-idle: 5
  jpa:
    show-sql: false
  redis:
    cluster:
      nodes: ${REDIS_CLUSTER_NODES}

logging:
  level:
    root: WARN
    com.company.userservice: INFO

```

Comunicación Entre Servicios

Usando OpenFeign

```

@FeignClient(name = "user-service", fallback = UserServiceFallback.class)
public interface UserServiceClient {

    @GetMapping("/users/{userId}")
    ResponseEntity<UserDto> getUserById(@PathVariable("userId") Long userId);

    @GetMapping("/users/{userId}/profile")
    ResponseEntity<UserProfileDto> getUserProfile(@PathVariable("userId") Long
userId);

    @PostMapping("/users")
    ResponseEntity<UserDto> createUser(@RequestBody CreateUserRequest request);

```



```
@PutMapping("/users/{userId}")
ResponseEntity<UserDto> updateUser(@PathVariable("userId") Long userId,
                                   @RequestBody UpdateUserRequest request);
}

@Component
public class UserServiceFallback implements UserServiceClient {

    @Override
    public ResponseEntity<UserDto> getUserById(Long userId) {
        UserDto fallbackUser = UserDto.builder()
            .id(userId)
            .username("unknown")
            .email("unknown@example.com")
            .status("UNAVAILABLE")
            .build();

        return ResponseEntity.ok(fallbackUser);
    }

    @Override
    public ResponseEntity<UserProfileDto> getUserProfile(Long userId) {
        return ResponseEntity.status(HttpStatus.SERVICE_UNAVAILABLE)
            .body(UserProfileDto.builder()
                .message("User service temporarily unavailable")
                .build());
    }
}

// Uso en otro servicio
@Service
public class OrderService {

    @Autowired
    private UserServiceClient userServiceClient;

    @Autowired
    private ProductServiceClient productServiceClient;

    public OrderDto createOrder(CreateOrderRequest request) {
        // Validar usuario
        UserDto user =
userServiceClient.getUserById(request.getUserId()).getBody();
        if ("UNAVAILABLE".equals(user.getStatus())) {
            throw new ServiceUnavailableException("User service is down");
        }

        // Validar productos
        List<ProductDto> products = request.getProductIds().stream()
            .map(productId ->
productServiceClient.getProductById(productId).getBody())
            .collect(Collectors.toList());

        // Crear orden
    }
}
```

```

        Order order = Order.builder()
            .userId(request.getUserId())
            .products(products)
            .totalAmount(calculateTotal(products))
            .status(OrderStatus.PENDING)
            .build();

        Order savedOrder = orderRepository.save(order);

        // Enviar evento de orden creada
        eventPublisher.publishEvent(new OrderCreatedEvent(savedOrder));

        return orderMapper.toDto(savedOrder);
    }
}

```

✉ Comunicación Asíncrona con Spring Cloud Stream

```

// Event Publisher
@Component
public class OrderEventPublisher {

    @Autowired
    private StreamBridge streamBridge;

    public void publishOrderCreated(OrderCreatedEvent event) {
        streamBridge.send("orderCreated-out-0", event);
    }

    public void publishOrderUpdated(OrderUpdatedEvent event) {
        streamBridge.send("orderUpdated-out-0", event);
    }
}

// Event Consumer
@Component
public class OrderEventConsumer {

    @Autowired
    private InventoryService inventoryService;

    @Autowired
    private NotificationService notificationService;

    @EventListener
    public void handleOrderCreated(OrderCreatedEvent event) {
        log.info("Processing order created event: {}", event.getOrderId());

        // Reducir inventario
        inventoryService.reserveProducts(event.getProductIds());
    }
}

```

```

        // Enviar notificación
        notificationService.sendOrderConfirmation(event.getUserId(),
event.getOrderId());
    }

    @RabbitListener(queues = "payment.processed")
    public void handlePaymentProcessed(PaymentProcessedEvent event) {
        log.info("Payment processed for order: {}", event.getOrderId());

        Order order = orderService.findById(event.getOrderId());
        order.setStatus(OrderStatus.CONFIRMED);
        orderService.save(order);

        // Enviar evento de orden confirmada
        orderEventPublisher.publishOrderConfirmed(new OrderConfirmedEvent(order));
    }
}

// Configuración de mensajería
@Configuration
public class MessagingConfig {

    @Bean
    public Queue orderCreatedQueue() {
        return QueueBuilder.durable("order.created").build();
    }

    @Bean
    public Queue paymentProcessedQueue() {
        return QueueBuilder.durable("payment.processed").build();
    }

    @Bean
    public TopicExchange orderExchange() {
        return new TopicExchange("order.exchange");
    }

    @Bean
    public Binding orderCreatedBinding() {
        return BindingBuilder.bind(orderCreatedQueue())
            .to(orderExchange())
            .with("order.created");
    }
}

```

🔑 Resilience Patterns

⚡ Circuit Breaker con Resilience4j

```

@Component
public class ProductServiceClient {

```

```
private final WebClient webClient;
private final CircuitBreaker circuitBreaker;
private final RetryTemplate retryTemplate;

public ProductServiceClient(WebClient.Builder webClientBuilder) {
    this.webClient = webClientBuilder.baseUrl("http://product-
service").build();

    this.circuitBreaker = CircuitBreaker.ofDefaults("productService");
    circuitBreaker.getEventPublisher()
        .onStateTransition(event ->
            log.info("Circuit breaker state transition: {}", event));

    this.retryTemplate = RetryTemplate.builder()
        .maxAttempts(3)
        .exponentialBackoff(1000, 2, 10000)
        .retryOn(Exception.class)
        .build();
}

public Mono<ProductDto> getProductById(Long productId) {
    return circuitBreaker.executeSupplier(() ->
        retryTemplate.execute(context ->
            webClient.get()
                .uri("/products/{id}", productId)
                .retrieve()
                .bodyToMono(ProductDto.class)
                .block()))
        .recover(throwable -> {
            log.error("Product service call failed, using fallback",
throwable);
            return getProductFallback(productId);
        });
}

private ProductDto getProductFallback(Long productId) {
    return ProductDto.builder()
        .id(productId)
        .name("Product Unavailable")
        .price(BigDecimal.ZERO)
        .available(false)
        .build();
}

}

// Configuración de Resilience4j
@Configuration
public class ResilienceConfig {

    @Bean
    public CircuitBreakerConfig circuitBreakerConfig() {
        return CircuitBreakerConfig.custom()
            .failureRateThreshold(50) // 50% failure rate

```

```

        .waitDurationInOpenState(Duration.ofSeconds(30))
        .slidingWindowSize(10)
        .minimumNumberOfCalls(5)
        .slowCallRateThreshold(50)
        .slowCallDurationThreshold(Duration.ofSeconds(2))
        .build();
    }

    @Bean
    public RetryConfig retryConfig() {
        return RetryConfig.custom()
            .maxAttempts(3)
            .waitDuration(Duration.ofSeconds(1))
            .retryExceptions(ConnectException.class, TimeoutException.class)
            .ignoreExceptions(IllegalArgumentException.class)
            .build();
    }

    @Bean
    public BulkheadConfig bulkheadConfig() {
        return BulkheadConfig.custom()
            .maxConcurrentCalls(10)
            .maxWaitDuration(Duration.ofSeconds(1))
            .build();
    }
}

```

Monitoring y Observabilidad

Métricas con Micrometer

```

@RestController
@RequestMapping("/api/orders")
public class OrderController {

    private final MeterRegistry meterRegistry;
    private final Counter orderCounter;
    private final Timer orderProcessingTimer;

    public OrderController(MeterRegistry meterRegistry) {
        this.meterRegistry = meterRegistry;
        this.orderCounter = Counter.builder("orders.created")
            .description("Number of orders created")
            .register(meterRegistry);
        this.orderProcessingTimer = Timer.builder("orders.processing.time")
            .description("Order processing time")
            .register(meterRegistry);
    }

    @PostMapping
    public ResponseEntity<OrderDto> createOrder(@RequestBody CreateOrderRequest

```

```
request) {
    return orderProcessingTimer.recordCallable(() -> {
        OrderDto order = orderService.createOrder(request);

        orderCounter.increment(
            Tags.of(
                "status", "success",
                "user_type", getUserType(request.getUserId())
            )
        );

        meterRegistry.gauge("orders.total.amount",
            order.getTotalAmount().doubleValue());

        return ResponseEntity.ok(order);
    });
}

@GetMapping("/metrics/custom")
public ResponseEntity<Map<String, Double>> getCustomMetrics() {
    Map<String, Double> metrics = new HashMap<>();

    metrics.put("orders.pending",
        orderService.countPendingOrders().doubleValue());
    metrics.put("orders.revenue.today",
        orderService.getTodayRevenue().doubleValue());
    metrics.put("orders.avg.processing.time", getAverageProcessingTime());

    return ResponseEntity.ok(metrics);
}

// Health Check personalizado
@Component
public class DatabaseHealthIndicator implements HealthIndicator {

    @Autowired
    private DataSource dataSource;

    @Override
    public Health health() {
        try (Connection connection = dataSource.getConnection()) {
            if (connection.isValid(1)) {
                return Health.up()
                    .withDetail("database", "Available")
                    .withDetail("validationQuery", "SELECT 1")
                    .build();
            }
        } catch (SQLException e) {
            return Health.down()
                .withDetail("database", "Unavailable")
                .withException(e)
                .build();
        }
    }
}
```

```
        return Health.down()
            .withDetail("database", "Unknown")
            .build();
    }
}
```

Prompts de IA para Módulo 6: Microservicios

PROMPT GITHUB COPILOT - Arquitectura Microservicios:

Actúa como un arquitecto de software senior especializado en microservicios Spring Boot. Necesito diseñar una arquitectura completa para un e-commerce.

Genera:

1. Service Registry con Eureka:
 - Configuración de Eureka Server
 - Client registration automático
 - Health checks y heartbeats
 - Load balancing strategies
2. API Gateway avanzado:
 - Spring Cloud Gateway con routing dinámico
 - Rate limiting por usuario/IP
 - Authentication/Authorization centralizada
 - Request/Response transformation
 - Circuit breaker integration
3. Configuration Management:
 - Spring Cloud Config Server
 - Encrypted properties
 - Refresh scope sin restart
 - Environment-specific configs
 - Git backend integration
4. Service Communication:
 - OpenFeign clients con fallbacks
 - Async messaging con RabbitMQ/Kafka
 - Event-driven architecture
 - Saga pattern para transactions distribuidas
5. Observability completa:
 - Distributed tracing con Sleuth/Zipkin
 - Metrics con Micrometer/Prometheus
 - Centralized logging con ELK
 - Custom health indicators

Compara cada patrón con implementaciones Node.js equivalentes y explica las ventajas del ecosistema Spring Cloud.

PROMPT CLAUDE - Resilience Patterns:

Eres un experto en sistemas distribuidos con Spring Boot. Necesito implementar todos los patrones de resiliencia para microservicios en producción.

Implementa:

1. Circuit Breaker con Resilience4j:
 - Configuración adaptive
 - Half-open state management
 - Fallback strategies
 - Metrics y alerting
2. Retry patterns inteligentes:
 - Exponential backoff
 - Jitter para evitar thundering herd
 - Conditional retry (por tipo de error)
 - Timeout configurations
3. Bulkhead pattern:
 - Thread pool isolation
 - Resource segregation
 - Priority queues
 - Graceful degradation
4. Rate Limiting:
 - Token bucket algorithm
 - Redis-based distributed limiting
 - Per-user/Per-API quotas
 - Sliding window counters
5. Chaos Engineering:
 - Chaos Monkey integration
 - Network latency injection
 - Random failures simulation
 - Load testing scenarios
6. Advanced monitoring:
 - SLA/SLO tracking
 - Error rate alerting
 - Performance baselines
 - Predictive scaling

Incluye testing strategies, deployment patterns y troubleshooting guides. Compara con Node.js/Express resilience libraries.

Módulo 7: Testing y Calidad de Código

Fundamentos de Testing en Spring Boot

En Node.js/TypeScript probablemente usas Jest, Mocha, o Vitest para testing. Spring Boot ofrece un ecosistema de testing más robusto y integrado con el framework.

 Comparación: Node.js vs Spring Boot

Concepto	Node.js/TypeScript	Spring Boot/Java
Unit Testing	Jest, Mocha, Vitest	JUnit 5, Mockito
Integration Testing	Supertest, TestContainers	@SpringBootTest, TestContainers
Mocking	jest.mock(), sinon	@MockBean, @Mock, Mockito
Test Database	sqlite, testcontainers	H2, TestContainers, @DataJpaTest
API Testing	Supertest, Postman	MockMvc, WebTestClient
Coverage	Istanbul/NYC	JaCoCo, Clover
E2E Testing	Playwright, Cypress	Selenium, TestContainers

 Arquitectura de Testing

```
// Configuración base de testing
@SpringBootTest
@AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.NONE)
@TestPropertySource(locations = "classpath:application-test.properties")
class BaseIntegrationTest {

    @Container
    static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>
("postgres:15")
        .withDatabaseName("testdb")
        .withUsername("test")
        .withPassword("test");

    @DynamicPropertySource
    static void configureProperties(DynamicPropertyRegistry registry) {
        registry.add("spring.datasource.url", postgres::getJdbcUrl);
        registry.add("spring.datasource.username", postgres::getUsername);
        registry.add("spring.datasource.password", postgres::getPassword);
    }
}

// Configuración de perfiles de test
# application-test.properties
spring.datasource.url=jdbc:h2:mem:testdb
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true
logging.level.org.springframework.web=DEBUG
logging.level.org.springframework.security=DEBUG
```

Comparación con Node.js/Jest:

```
// Node.js/Jest setup equivalente
import { Test, TestingModule } from "@nestjs/testing";
import { INestApplication } from "@nestjs/common";
import { getRepositoryToken } from "@nestjs/typeorm";
import { Repository } from "typeorm";

describe("UserController (e2e)", () => {
  let app: INestApplication;
  let userRepository: Repository<User>;

  beforeEach(async () => {
    const moduleFixture: TestingModule = await Test.createTestingModule({
      imports: [AppModule],
    })
      .overrideProvider(getRepositoryToken(User))
      .useValue(mockUserRepository)
      .compile();

    app = moduleFixture.createNestApplication();
    userRepository = moduleFixture.get<Repository<User>>(
      getRepositoryToken(User));
    await app.init();
  });
});
```

🔑 Unit Testing con JUnit 5 y Mockito**🔧 Testing de Servicios**

```
@ExtendWith(MockitoExtension.class)
class UserServiceTest {

  @Mock
  private UserRepository userRepository;

  @Mock
  private PasswordEncoder passwordEncoder;

  @Mock
  private EmailService emailService;

  @InjectMocks
  private UserService userService;

  @Captor
  private ArgumentCaptor<User> userCaptor;

  @BeforeEach
```

```
void setUp() {
    // Setup común para todos los tests
}

@Test
@DisplayName("Should create user successfully when valid data provided")
void shouldCreateUserSuccessfully() {
    // Given
    CreateUserRequest request = CreateUserRequest.builder()
        .username("testuser")
        .email("test@example.com")
        .password("password123")
        .build();

    User mockUser = User.builder()
        .id(1L)
        .username("testuser")
        .email("test@example.com")
        .password("encodedPassword")
        .build();

    when(userRepository.existsByUsername("testuser")).thenReturn(false);
    when(userRepository.existsByEmail("test@example.com")).thenReturn(false);
    when(passwordEncoder.encode("password123")).thenReturn("encodedPassword");
    when(userRepository.save(any(User.class))).thenReturn(mockUser);

    // When
    UserDto result = userService.createUser(request);

    // Then
    assertNotNull(result);
    assertEquals(result.getUsername(), "testuser");
    assertEquals(result.getEmail(), "test@example.com");

    verify(userRepository).save(userCaptor.capture());
    User savedUser = userCaptor.getValue();
    assertEquals(savedUser.getPassword(), "encodedPassword");

    verify(emailService).sendWelcomeEmail("test@example.com", "testuser");
}

@Test
@DisplayName("Should throw exception when username already exists")
void shouldThrowExceptionWhenUsernameExists() {
    // Given
    CreateUserRequest request = CreateUserRequest.builder()
        .username("existinguser")
        .email("test@example.com")
        .password("password123")
        .build();

    when(userRepository.existsByUsername("existinguser")).thenReturn(true);

    // When & Then
```

```

        assertThatThrownBy(() -> userService.createUser(request))
            .isInstanceOf(UsernameAlreadyExistsException.class)
            .hasMessage("Username 'existinguser' is already taken");

        verify(userRepository, never()).save(any(User.class));
        verify(emailService, never()).sendWelcomeEmail(anyString(), anyString());
    }

    @ParameterizedTest
    @ValueSource(strings = {"", " ", "ab", "a".repeat(51)})
    @DisplayName("Should validate username length and format")
    void shouldValidateUsername(String invalidUsername) {
        // Given
        CreateUserRequest request = CreateUserRequest.builder()
            .username(invalidUsername)
            .email("test@example.com")
            .password("password123")
            .build();

        // When & Then
        assertThatThrownBy(() -> userService.createUser(request))
            .isInstanceOf(ValidationException.class);
    }

    @Test
    @DisplayName("Should handle repository exception gracefully")
    void shouldHandleRepositoryException() {
        // Given
        CreateUserRequest request = CreateUserRequest.builder()
            .username("testuser")
            .email("test@example.com")
            .password("password123")
            .build();

        when(userRepository.existsByUsername("testuser")).thenReturn(false);
        when(userRepository.existsByEmail("test@example.com")).thenReturn(false);
        when(passwordEncoder.encode("password123")).thenReturn("encodedPassword");
        when(userRepository.save(any(User.class))).thenThrow(new
DataAccessException("DB Error") {}));

        // When & Then
        assertThatThrownBy(() -> userService.createUser(request))
            .isInstanceOf(ServiceException.class)
            .hasCauseInstanceOf(DataAccessException.class);
    }
}

```

Testing de Repositories

```

@DataJpaTest
@AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.NONE)

```

```
class UserRepositoryTest extends BaseIntegrationTest {

    @Autowired
    private EntityManager entityManager;

    @Autowired
    private UserRepository userRepository;

    @Test
    @DisplayName("Should find user by username")
    void shouldFindUserByUsername() {
        // Given
        User user = User.builder()
            .username("testuser")
            .email("test@example.com")
            .password("password")
            .build();

        entityManager.persistAndFlush(user);

        // When
        Optional<User> found = userRepository.findByUsername("testuser");

        // Then
        assertThat(found).isPresent();
        assertThat(found.get().getUsername()).isEqualTo("testuser");
        assertThat(found.get().getEmail()).isEqualTo("test@example.com");
    }

    @Test
    @DisplayName("Should find users created after date")
    void shouldFindUsersCreatedAfterDate() {
        // Given
        LocalDateTime cutoffDate = LocalDateTime.now().minusDays(1);

        User oldUser = User.builder()
            .username("olduser")
            .email("old@example.com")
            .password("password")
            .createdAt(cutoffDate.minusHours(1))
            .build();

        User newUser = User.builder()
            .username("newuser")
            .email("new@example.com")
            .password("password")
            .createdAt(cutoffDate.plusHours(1))
            .build();

        entityManager.persist(oldUser);
        entityManager.persist(newUser);
        entityManager.flush();

        // When
```

```

        List<User> recentUsers = userRepository.findByCreatedAtAfter(cutoffDate);

        // Then
        assertThat(recentUsers).hasSize(1);
        assertThat(recentUsers.get(0).getUsername()).isEqualTo("newuser");
    }

    @Test
    @DisplayName("Should count users by role")
    void shouldCountUsersByRole() {
        // Given
        Role userRole = new Role(ERole.ROLE_USER);
        Role adminRole = new Role(ERole.ROLE_ADMIN);

        entityManager.persist(userRole);
        entityManager.persist(adminRole);

        User user1 = createUserWithRole("user1", userRole);
        User user2 = createUserWithRole("user2", userRole);
        User admin = createUserWithRole("admin", adminRole);

        entityManager.persist(user1);
        entityManager.persist(user2);
        entityManager.persist(admin);
        entityManager.flush();

        // When
        Long userCount = userRepository.countByRolesName(ERole.ROLE_USER);
        Long adminCount = userRepository.countByRolesName(ERole.ROLE_ADMIN);

        // Then
        assertThat(userCount).isEqualTo(2);
        assertThat(adminCount).isEqualTo(1);
    }

    private User createUserWithRole(String username, Role role) {
        User user = User.builder()
            .username(username)
            .email(username + "@example.com")
            .password("password")
            .build();
        user.getRoles().add(role);
        return user;
    }
}

```

Integration Testing

Testing de Controllers

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.NONE)
class UserControllerIntegrationTest extends BaseIntegrationTest {

    @Autowired
    private TestRestTemplate restTemplate;

    @Autowired
    private UserRepository userRepository;

    @Autowired
    private JwtUtils jwtUtils;

    private String adminToken;
    private String userToken;

    @BeforeEach
    void setUp() {
        userRepository.deleteAll();

        // Crear usuarios de prueba
        User admin = createUser("admin", "admin@example.com", ERole.ROLE_ADMIN);
        User user = createUser("user", "user@example.com", ERole.ROLE_USER);

        userRepository.save(admin);
        userRepository.save(user);

        // Generar tokens
        adminToken = generateToken(admin);
        userToken = generateToken(user);
    }

    @Test
    @DisplayName("Should create user when valid data provided")
    void shouldCreateUser() {
        // Given
        CreateUserRequest request = CreateUserRequest.builder()
            .username("newuser")
            .email("newuser@example.com")
            .password("password123")
            .build();

        HttpHeaders headers = new HttpHeaders();
        headers.set("Authorization", "Bearer " + adminToken);
        HttpEntity<CreateUserRequest> entity = new HttpEntity<>(request, headers);

        // When
        ResponseEntity<UserDto> response = restTemplate.postForEntity(
            "/api/admin/users", entity, UserDto.class);

        // Then
        assertThat(response.getStatusCode()).isEqualTo(HttpStatus.CREATED);
        assertThat(response.getBody()).isNotNull();
    }
}
```

```
        assertThat(response.getBody().getUsername()).isEqualTo("newuser");

        Optional<User> savedUser = userRepository.findByUsername("newuser");
        assertThat(savedUser).isPresent();
    }

    @Test
    @DisplayName("Should return 403 when non-admin tries to create user")
    void shouldReturn403ForNonAdmin() {
        // Given
        CreateUserRequest request = CreateUserRequest.builder()
            .username("newuser")
            .email("newuser@example.com")
            .password("password123")
            .build();

        HttpHeaders headers = new HttpHeaders();
        headers.set("Authorization", "Bearer " + userToken);
        HttpEntity<CreateUserRequest> entity = new HttpEntity<>(request, headers);

        // When
        ResponseEntity<String> response = restTemplate.postForEntity(
            "/api/admin/users", entity, String.class);

        // Then
        assertThat(response.getStatusCode()).isEqualTo(HttpStatus.FORBIDDEN);
    }

    @Test
    @DisplayName("Should get user profile when authenticated")
    void shouldGetUserProfile() {
        // Given
        HttpHeaders headers = new HttpHeaders();
        headers.set("Authorization", "Bearer " + userToken);
        HttpEntity<String> entity = new HttpEntity<>(headers);

        // When
        ResponseEntity<UserDto> response = restTemplate.exchange(
            "/api/user/profile", HttpMethod.GET, entity, UserDto.class);

        // Then
        assertThat(response.getStatusCode()).isEqualTo(HttpStatus.OK);
        assertThat(response.getBody()).isNotNull();
        assertThat(response.getBody().getUsername()).isEqualTo("user");
    }

    private User createUser(String username, String email, ERole role) {
        User user = User.builder()
            .username(username)
            .email(email)
            .password("encodedPassword")
            .build();

        Role userRole = new Role(role);
```



```

        user.getRoles().add(userRole);

        return user;
    }

    private String generateToken(User user) {
        // Implementación simplificada
        return jwtUtils.generateTokenFromUsername(user.getUsername());
    }
}

```

Testing con MockMvc

```

@WebMvcTest(UserController.class)
@Import(SecurityConfig.class)
class UserControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private UserService userService;

    @MockBean
    private JwtUtils jwtUtils;

    @MockBean
    private UserDetailsServiceImpl userDetailsService;

    private ObjectMapper objectMapper = new ObjectMapper();

    @Test
    @WithMockUser(roles = "ADMIN")
    @DisplayName("Should create user with valid request")
    void shouldCreateUserWithValidRequest() throws Exception {
        // Given
        CreateUserRequest request = CreateUserRequest.builder()
            .username("testuser")
            .email("test@example.com")
            .password("password123")
            .build();

        UserDto expectedResponse = UserDto.builder()
            .id(1L)
            .username("testuser")
            .email("test@example.com")
            .build();

        when(userService.createUser(any(CreateUserRequest.class)))
            .thenReturn(expectedResponse);
    }
}

```

```

// When & Then
mockMvc.perform(post("/api/admin/users")
    .contentType(MediaType.APPLICATION_JSON)
    .content(objectMapper.writeValueAsString(request)))
    .andExpect(status().isCreated())
    .andExpect(jsonPath("$.username").value("testuser"))
    .andExpect(jsonPath("$.email").value("test@example.com"))
    .andExpect(jsonPath("$.id").value(1))
    .andDo(print());

verify(userService).createUser(argThat(req ->
    req.getUsername().equals("testuser") &&
    req.getEmail().equals("test@example.com")));
}

@Test
@WithMockUser(roles = "USER")
@DisplayName("Should return 403 when user tries to access admin endpoint")
void shouldReturn403ForUserRole() throws Exception {
    // Given
    CreateUserRequest request = CreateUserRequest.builder()
        .username("testuser")
        .email("test@example.com")
        .password("password123")
        .build();

    // When & Then
    mockMvc.perform(post("/api/admin/users")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(request)))
        .andExpect(status().isForbidden());

    verifyNoInteractions(userService);
}

@ParameterizedTest
@MethodSource("invalidUserRequests")
@WithMockUser(roles = "ADMIN")
@DisplayName("Should validate request fields")
void shouldValidateRequestFields(CreateUserRequest request, String
expectedError) throws Exception {
    mockMvc.perform(post("/api/admin/users")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(request)))
        .andExpect(status().isBadRequest())
        .andExpect(jsonPath("$.errors").exists())

        .andExpect(jsonPath("$.errors[*]").value(hasItem(containsString(expectedError))));
}

static Stream<Arguments> invalidUserRequests() {
    return Stream.of(
        Arguments.of(

```

```

CreateUserRequest.builder().username("").email("test@example.com").password("password123").build(),
    "Username cannot be blank"
),
Arguments.of(

CreateUserRequest.builder().username("testuser").email("invalid-email").password("password123").build(),
    "Email should be valid"
),
Arguments.of(

CreateUserRequest.builder().username("testuser").email("test@example.com").password("123").build(),
    "Password must be at least 8 characters"
)
);
}
}

```

Testing con TestContainers

Integration Tests con Base de Datos Real

```

@SpringBootTest
@Testcontainers
class ProductServiceIntegrationTest {

    @Container
    static PostgreSQLContainer<> postgres = new PostgreSQLContainer<>
("postgres:15")
        .withDatabaseName("testdb")
        .withUsername("test")
        .withPassword("test")
        .withReuse(true);

    @Container
    static RedisContainer redis = new
RedisContainer(DockerImageName.parse("redis:7-alpine"))
        .withReuse(true);

    @Container
    static RabbitMQContainer rabbitmq = new
RabbitMQContainer(DockerImageName.parse("rabbitmq:3-management"))
        .withReuse(true);

    @DynamicPropertySource
    static void configureProperties(DynamicPropertyRegistry registry) {
        // Database
        registry.add("spring.datasource.url", postgres::getJdbcUrl);
        registry.add("spring.datasource.username", postgres::getUsername);
    }
}

```

```
registry.add("spring.datasource.password", postgres::getPassword);

// Redis
registry.add("spring.redis.host", redis::getHost);
registry.add("spring.redis.port", redis::getFirstMappedPort);

// RabbitMQ
registry.add("spring.rabbitmq.host", rabbitmq::getHost);
registry.add("spring.rabbitmq.port", rabbitmq::getFirstMappedPort);
registry.add("spring.rabbitmq.username", rabbitmq::getAdminUsername);
registry.add("spring.rabbitmq.password", rabbitmq::getAdminPassword);
}

@Autowired
private ProductService productService;

@Autowired
private ProductRepository productRepository;

@Autowired
private RedisTemplate<String, Object> redisTemplate;

@Test
@Transactional
@Rollback
@DisplayName("Should create product and cache it")
void shouldCreateProductAndCacheIt() {
    // Given
    CreateProductRequest request = CreateProductRequest.builder()
        .name("Test Product")
        .description("Test Description")
        .price(BigDecimal.valueOf(99.99))
        .categoryId(1L)
        .build();

    // When
    ProductDto created = productService.createProduct(request);

    // Then
    assertThat(created).isNotNull();
    assertThat(created.getName()).isEqualTo("Test Product");

    // Verify database
    Optional<Product> inDb = productRepository.findById(created.getId());
    assertThat(inDb).isPresent();

    // Verify cache
    String cacheKey = "product:" + created.getId();
    Object cached = redisTemplate.opsForValue().get(cacheKey);
    assertThat(cached).isNotNull();
}

@Test
@DisplayName("Should handle database connection failure gracefully")
```

```

void shouldHandleDatabaseFailure() {
    // Given - stop the database container
    postgres.stop();

    CreateProductRequest request = CreateProductRequest.builder()
        .name("Test Product")
        .description("Test Description")
        .price(BigDecimal.valueOf(99.99))
        .categoryId(1L)
        .build();

    // When & Then
    assertThatThrownBy(() -> productService.createProduct(request))
        .isInstanceOf(DataAccessException.class);

    // Restart for other tests
    postgres.start();
}
}

```

Testing de Performance

Load Testing con Spring Boot

```

@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.DEFINED_PORT)
class PerformanceTest {

    @Autowired
    private TestRestTemplate restTemplate;

    @Test
    @DisplayName("Should handle concurrent user creation")
    void shouldHandleConcurrentUserCreation() throws InterruptedException {
        int numberOfThreads = 10;
        int requestsPerThread = 50;
        CountDownLatch latch = new CountDownLatch(numberOfThreads);
        AtomicInteger successCount = new AtomicInteger(0);
        AtomicInteger errorCount = new AtomicInteger(0);

        ExecutorService executor = Executors.newFixedThreadPool(numberOfThreads);

        for (int i = 0; i < numberOfThreads; i++) {
            final int threadId = i;
            executor.submit(() -> {
                try {
                    for (int j = 0; j < requestsPerThread; j++) {
                        CreateUserRequest request = CreateUserRequest.builder()
                            .username("user_" + threadId + "_" + j)
                            .email("user_" + threadId + "_" + j +
                                "@example.com")
                            .password("password123")

```

```

        .build();

        try {
            ResponseEntity<UserDto> response =
restTemplate.postForEntity(
                "/api/users", request, UserDto.class);

            if (response.getStatusCode().is2xxSuccessful()) {
                successCount.incrementAndGet();
            } else {
                errorCount.incrementAndGet();
            }
        } catch (Exception e) {
            errorCount.incrementAndGet();
        }
    }
} finally {
    latch.countDown();
}
});
}

boolean finished = latch.await(60, TimeUnit.SECONDS);
executor.shutdown();

assertThat(finished).isTrue();
assertThat(successCount.get()).isGreaterThan(numberOfThreads *
requestsPerThread * 0.8); // 80% success rate

System.out.println("Success: " + successCount.get() + ", Errors: " +
errorCount.get());
}

@Test
@DisplayName("Should respond within acceptable time limits")
void shouldRespondWithinTimeLimits() {
    long startTime = System.currentTimeMillis();

    ResponseEntity<List> response = restTemplate.getForEntity("/api/products",
List.class);

    long duration = System.currentTimeMillis() - startTime;

    assertThat(response.getStatusCode()).isEqualTo(HttpStatus.OK);
    assertThat(duration).isLessThan(2000); // Less than 2 seconds
}
}

```

Prompts de IA para Módulo 7: Testing

PROMPT GITHUB COPILOT - Testing Strategy Completa:

Actúa como un QA Engineer senior especializado en Spring Boot. Necesito implementar una estrategia de testing completa para una aplicación e-commerce.

Genera:

1. Test Architecture:
 - Unit tests con JUnit 5 y Mockito
 - Integration tests con @SpringBootTest
 - Repository tests con @DataJpaTest
 - Web layer tests con @WebMvcTest
 - TestContainers para database real
2. Advanced Testing Patterns:
 - Test slices para diferentes capas
 - Custom test configurations
 - Test data builders y fixtures
 - Parameterized tests con @ParameterizedTest
 - Test profiles y property sources
3. Mocking Strategies:
 - @MockBean vs @Mock diferencias
 - ArgumentCaptor para validaciones
 - Custom matchers con Mockito
 - Spy objects para partial mocking
 - Verification patterns avanzados
4. Performance Testing:
 - Load testing con concurrent requests
 - Database performance tests
 - Memory leak detection
 - Response time assertions
 - Throughput measurements
5. Test Coverage y Quality:
 - JaCoCo configuration completa
 - Mutation testing con PiTest
 - Test code quality metrics
 - CI/CD integration

Compara cada patrón con Jest/Mocha en Node.js y explica las ventajas del ecosistema Spring Boot.

PROMPT CLAUDE - TestContainers y E2E Testing:

Eres un especialista en testing de microservicios con Spring Boot. Necesito implementar testing de integración completo usando TestContainers.

Implementa:

1. TestContainers Setup:
 - PostgreSQL, Redis, RabbitMQ containers
 - Shared containers para performance
 - Network configuration
 - Custom container configurations
 - Health checks y wait strategies
2. Integration Test Scenarios:
 - Database transactions testing
 - Message queue integration
 - External API mocking con WireMock
 - File system operations
 - Cache behavior validation
3. End-to-End Testing:
 - Complete user journeys
 - Multi-service interactions
 - Authentication flows
 - Error scenario testing
 - Data consistency validation
4. Test Data Management:
 - Database seeding strategies
 - Test data isolation
 - Cleanup procedures
 - Test data factories
 - Snapshot testing
5. CI/CD Integration:
 - Docker-in-Docker configuration
 - Parallel test execution
 - Test reporting y artifacts
 - Flaky test detection
 - Test environment management
6. Advanced Scenarios:
 - Chaos engineering tests
 - Security penetration testing
 - Performance regression tests
 - Contract testing con Pact

Incluye best practices, troubleshooting guides y performance optimizations.

Módulo 8: DevOps y Deployment

Fundamentos de DevOps para Spring Boot

En el mundo Node.js/TypeScript, probablemente has usado Docker, GitHub Actions, y plataformas como Vercel o Heroku. Con Spring Boot y Java, el ecosistema DevOps es más robusto y empresarial.

Comparación: Node.js vs Spring Boot

Concepto	Node.js/TypeScript	Spring Boot/Java
Containerization	Docker con alpine/node	Docker con OpenJDK, native images
Build Tools	npm, yarn, pnpm	Maven, Gradle
CI/CD	GitHub Actions, Vercel	GitHub Actions, Jenkins, GitLab CI
Deployment	Vercel, Netlify, PM2	Kubernetes, Docker Swarm, OpenShift
Monitoring	New Relic, DataDog	Micrometer + Prometheus + Grafana
Configuration	.env files, config libs	Spring Cloud Config, Kubernetes ConfigMaps
Secrets Management	dotenv, AWS Secrets	Spring Cloud Vault, Kubernetes Secrets

 Containerización con Docker

```
# Multi-stage Dockerfile para Spring Boot
FROM eclipse-temurin:21-jdk-alpine AS builder

WORKDIR /app
COPY mvnw .
COPY .mvn .mvn
COPY pom.xml .
COPY src src

# Cache dependencies
RUN ./mvnw dependency:go-offline -B

# Build application
RUN ./mvnw clean package -DskipTests

# Runtime stage
FROM eclipse-temurin:21-jre-alpine

LABEL maintainer="your-team@company.com"
LABEL version="1.0.0"
LABEL description="E-commerce API Spring Boot Application"

# Create non-root user
RUN addgroup -g 1001 -S appgroup && \
    adduser -u 1001 -S appuser -G appgroup

# Install security updates
RUN apk update && apk upgrade && apk add --no-cache curl

WORKDIR /app

# Copy JAR from builder stage
COPY --from=builder /app/target/*.jar app.jar

# Create directories and set permissions
```

```

RUN mkdir -p /app/logs /app/tmp && \
    chown -R appuser:appgroup /app

# Switch to non-root user
USER appuser

# Health check
HEALTHCHECK --interval=30s --timeout=10s --start-period=60s --retries=3 \
    CMD curl -f http://localhost:8080/actuator/health || exit 1

# JVM optimizations
ENV JAVA_OPTS="-XX:+UseContainerSupport \
    -XX:MaxRAMPercentage=75.0 \
    -XX:+UseG1GC \
    -XX:+UseStringDeduplication \
    -Djava.security.egd=file:/dev/./urandom"

EXPOSE 8080

ENTRYPOINT ["sh", "-c", "java $JAVA_OPTS -jar app.jar"]

```

Comparación con Node.js Dockerfile:

```

# Node.js equivalent
FROM node:18-alpine AS builder

WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production

FROM node:18-alpine AS runtime
RUN addgroup -g 1001 -S nodejs && \
    adduser -S nextjs -u 1001

WORKDIR /app
COPY --from=builder /app/node_modules ./node_modules
COPY . .

USER nextjs

EXPOSE 3000

CMD ["npm", "start"]

```

Docker Compose para Desarrollo

```

# docker-compose.yml
version: "3.8"

```

```
services:
  app:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: ecommerce-api
    ports:
      - "8080:8080"
    environment:
      - SPRING_PROFILES_ACTIVE=docker
      - SPRING_DATASOURCE_URL=jdbc:postgresql://postgres:5432/ecommerce
      - SPRING_DATASOURCE_USERNAME=postgres
      - SPRING_DATASOURCE_PASSWORD=postgres
      - SPRING_REDIS_HOST=redis
      - SPRING_RABBITMQ_HOST=rabbitmq
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
      rabbitmq:
        condition: service_healthy
    networks:
      - app-network
    volumes:
      - ./logs:/app/logs
    restart: unless-stopped
    deploy:
      resources:
        limits:
          memory: 1G
          cpus: "0.5"

  postgres:
    image: postgres:15-alpine
    container_name: ecommerce-db
    environment:
      POSTGRES_DB: ecommerce
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./docker/init.sql:/docker-entrypoint-initdb.d/init.sql
    ports:
      - "5432:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - app-network

  redis:
```

```
image: redis:7-alpine
container_name: ecommerce-cache
command: redis-server --appendonly yes --requirepass redis123
volumes:
  - redis_data:/data
  - ./docker/redis.conf:/usr/local/etc/redis/redis.conf
ports:
  - "6379:6379"
healthcheck:
  test: ["CMD", "redis-cli", "ping"]
  interval: 10s
  timeout: 5s
  retries: 5
networks:
  - app-network

rabbitmq:
image: rabbitmq:3-management-alpine
container_name: ecommerce-mq
environment:
  RABBITMQ_DEFAULT_USER: admin
  RABBITMQ_DEFAULT_PASS: admin123
volumes:
  - rabbitmq_data:/var/lib/rabbitmq
  - ./docker/rabbitmq.conf:/etc/rabbitmq/rabbitmq.conf
ports:
  - "5672:5672"
  - "15672:15672"
healthcheck:
  test: ["CMD", "rabbitmq-diagnostics", "ping"]
  interval: 30s
  timeout: 10s
  retries: 3
networks:
  - app-network

prometheus:
image: prom/prometheus:latest
container_name: ecommerce-prometheus
command:
  - "--config.file=/etc/prometheus/prometheus.yml"
  - "--storage.tsdb.path=/prometheus"
  - "--web.console.libraries=/etc/prometheus/console_libraries"
  - "--web.console.templates=/etc/prometheus/consoles"
volumes:
  - ./docker/prometheus.yml:/etc/prometheus/prometheus.yml
  - prometheus_data:/prometheus
ports:
  - "9090:9090"
networks:
  - app-network

grafana:
image: grafana/grafana:latest
```

```

    container_name: ecommerce-grafana
    environment:
      - GF_SECURITY_ADMIN_PASSWORD=admin123
    volumes:
      - grafana_data:/var/lib/grafana
      - ./docker/grafana/dashboards:/etc/grafana/provisioning/dashboards
      - ./docker/grafana/datasources:/etc/grafana/provisioning/datasources
    ports:
      - "3000:3000"
    depends_on:
      - prometheus
    networks:
      - app-network

volumes:
  postgres_data:
  redis_data:
  rabbitmq_data:
  prometheus_data:
  grafana_data:

networks:
  app-network:
    driver: bridge

```

CI/CD con GitHub Actions

Pipeline Completo

```

# .github/workflows/ci-cd.yml
name: CI/CD Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]
  release:
    types: [published]

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${github.repository}

jobs:
  test:
    runs-on: ubuntu-latest

    services:
      postgres:
        image: postgres:15

```

```
env:
  POSTGRES_PASSWORD: postgres
  POSTGRES_DB: testdb
options: >-
  --health-cmd pg_isready
  --health-interval 10s
  --health-timeout 5s
  --health-retries 5
ports:
  - 5432:5432

redis:
  image: redis:7
  options: >-
    --health-cmd "redis-cli ping"
    --health-interval 10s
    --health-timeout 5s
    --health-retries 5
  ports:
    - 6379:6379

steps:
  - name: Checkout code
    uses: actions/checkout@v4

  - name: Set up JDK 21
    uses: actions/setup-java@v4
    with:
      java-version: "21"
      distribution: "temurin"
      cache: maven

  - name: Cache Maven dependencies
    uses: actions/cache@v3
    with:
      path: ~/.m2
      key: ${ runner.os }-m2-${ hashFiles('**/pom.xml') }
      restore-keys: ${ runner.os }-m2

  - name: Run tests
    run: ./mvnw test -Dspring.profiles.active=test
    env:
      SPRING_DATASOURCE_URL: jdbc:postgresql://localhost:5432/testdb
      SPRING_DATASOURCE_USERNAME: postgres
      SPRING_DATASOURCE_PASSWORD: postgres
      SPRING_REDIS_HOST: localhost
      SPRING_REDIS_PORT: 6379

  - name: Generate test report
    uses: dorny/test-reporter@v1
    if: success() || failure()
    with:
      name: Maven Tests
      path: target/surefire-reports/*.xml
```

```
    reporter: java-junit

  - name: Code coverage
    run: ./mvnw jacoco:report

  - name: Upload coverage to Codecov
    uses: codecov/codecov-action@v3
    with:
      token: ${ secrets.CODECOV_TOKEN }
      file: target/site/jacoco/jacoco.xml

  - name: SonarCloud Scan
    uses: SonarSource/sonarcloud-github-action@master
    env:
      GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
      SONAR_TOKEN: ${ secrets.SONAR_TOKEN }

security-scan:
  runs-on: ubuntu-latest
  needs: test

  steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Set up JDK 21
      uses: actions/setup-java@v4
      with:
        java-version: "21"
        distribution: "temurin"

    - name: OWASP Dependency Check
      run: ./mvnw org.owasp:dependency-check-maven:check

    - name: Upload OWASP report
      uses: actions/upload-artifact@v3
      if: always()
      with:
        name: owasp-report
        path: target/dependency-check-report.html

    - name: Trivy vulnerability scanner
      uses: aquasecurity/trivy-action@master
      with:
        scan-type: "fs"
        scan-ref: "."
        format: "sarif"
        output: "trivy-results.sarif"

    - name: Upload Trivy scan results
      uses: github/codeql-action/upload-sarif@v2
      with:
        sarif_file: "trivy-results.sarif"
```

```
build-and-push:
  runs-on: ubuntu-latest
  needs: [test, security-scan]
  if: github.event_name != 'pull_request'

  permissions:
    contents: read
    packages: write

  steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Set up JDK 21
      uses: actions/setup-java@v4
      with:
        java-version: "21"
        distribution: "temurin"
        cache: maven

    - name: Build application
      run: ./mvnw clean package -DskipTests

    - name: Set up Docker Buildx
      uses: docker/setup-buildx-action@v3

    - name: Log in to Container Registry
      uses: docker/login-action@v3
      with:
        registry: ${ env.REGISTRY }
        username: ${ github.actor }
        password: ${ secrets.GITHUB_TOKEN }

    - name: Extract metadata
      id: meta
      uses: docker/metadata-action@v5
      with:
        images: ${ env.REGISTRY }/${ env.IMAGE_NAME }
        tags: |
          type=ref,event=branch
          type=ref,event=pr
          type=semver,pattern={{version}}
          type=semver,pattern={{major}}.{{minor}}
          type=sha,prefix={{branch}}-

    - name: Build and push Docker image
      uses: docker/build-push-action@v5
      with:
        context: .
        push: true
        tags: ${ steps.meta.outputs.tags }
        labels: ${ steps.meta.outputs.labels }
        cache-from: type=gha
        cache-to: type=gha,mode=max
```



```

    platforms: linux/amd64,linux/arm64

  deploy-staging:
    runs-on: ubuntu-latest
    needs: build-and-push
    if: github.ref == 'refs/heads/develop'
    environment: staging

    steps:
      - name: Deploy to staging
        run: |
          echo "Deploying to staging environment"
          # kubectl commands or Terraform apply

  deploy-production:
    runs-on: ubuntu-latest
    needs: build-and-push
    if: github.event_name == 'release'
    environment: production

    steps:
      - name: Deploy to production
        run: |
          echo "Deploying to production environment"
          # kubectl commands or Terraform apply

```

Kubernetes Deployment

Kubernetes Manifests

```

# k8s/namespace.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: ecommerce
  labels:
    name: ecommerce

---
# k8s/configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
  namespace: ecommerce
data:
  application.yml: |
    spring:
      profiles:
        active: kubernetes
      jpa:

```

```
    hibernate:
      ddl-auto: validate
      show-sql: false
    redis:
      timeout: 2000ms
      jedis:
        pool:
          max-active: 8
          max-idle: 8
          min-idle: 0

server:
  port: 8080
  servlet:
    context-path: /api

management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
  endpoint:
    health:
      show-details: always
  metrics:
    export:
      prometheus:
        enabled: true

---
# k8s/secret.yaml
apiVersion: v1
kind: Secret
metadata:
  name: app-secrets
  namespace: ecommerce
type: Opaque
data:
  # Base64 encoded values
  db-username: cG9zdGdyZXM=
  db-password: cG9zdGdyZXM=
  jwt-secret: bXltZWNyZXRLZXk=
  redis-password: cmVkaXMxMjM=

---
# k8s/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ecommerce-api
  namespace: ecommerce
  labels:
    app: ecommerce-api
    version: v1
```

```
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 1
  selector:
    matchLabels:
      app: ecommerce-api
  template:
    metadata:
      labels:
        app: ecommerce-api
        version: v1
    annotations:
      prometheus.io/scrape: "true"
      prometheus.io/port: "8080"
      prometheus.io/path: "/actuator/prometheus"
  spec:
    serviceAccountName: ecommerce-api
    containers:
      - name: ecommerce-api
        image: ghcr.io/company/ecommerce-api:latest
        imagePullPolicy: Always
        ports:
          - containerPort: 8080
            name: http
            protocol: TCP
        env:
          - name: SPRING_PROFILES_ACTIVE
            value: "kubernetes"
          - name: SPRING_DATASOURCE_URL
            value: "jdbc:postgresql://postgres-service:5432/ecommerce"
          - name: SPRING_DATASOURCE_USERNAME
            valueFrom:
              secretKeyRef:
                name: app-secrets
                key: db-username
          - name: SPRING_DATASOURCE_PASSWORD
            valueFrom:
              secretKeyRef:
                name: app-secrets
                key: db-password
          - name: SPRING_REDIS_HOST
            value: "redis-service"
          - name: SPRING_REDIS_PASSWORD
            valueFrom:
              secretKeyRef:
                name: app-secrets
                key: redis-password
          - name: JWT_SECRET
            valueFrom:
              secretKeyRef:
```

```
        name: app-secrets
        key: jwt-secret
    - name: JAVA_OPTS
      value: "-XX:+UseContainerSupport -XX:MaxRAMPercentage=75.0"

resources:
  requests:
    memory: "512Mi"
    cpu: "250m"
  limits:
    memory: "1Gi"
    cpu: "500m"

livenessProbe:
  httpGet:
    path: /actuator/health/liveness
    port: 8080
  initialDelaySeconds: 120
  periodSeconds: 30
  timeoutSeconds: 10
  failureThreshold: 3

readinessProbe:
  httpGet:
    path: /actuator/health/readiness
    port: 8080
  initialDelaySeconds: 30
  periodSeconds: 10
  timeoutSeconds: 5
  failureThreshold: 3

startupProbe:
  httpGet:
    path: /actuator/health/startup
    port: 8080
  initialDelaySeconds: 30
  periodSeconds: 10
  timeoutSeconds: 5
  failureThreshold: 30

volumeMounts:
  - name: config-volume
    mountPath: /app/config
  - name: logs-volume
    mountPath: /app/logs

volumes:
  - name: config-volume
    configMap:
      name: app-config
  - name: logs-volume
    emptyDir: {}

imagePullSecrets:
```

```
- name: ghcr-secret

---
# k8s/service.yaml
apiVersion: v1
kind: Service
metadata:
  name: ecommerce-api-service
  namespace: ecommerce
  labels:
    app: ecommerce-api
spec:
  type: ClusterIP
  ports:
    - port: 80
      targetPort: 8080
      protocol: TCP
      name: http
  selector:
    app: ecommerce-api

---
# k8s/ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: ecommerce-api-ingress
  namespace: ecommerce
  annotations:
    kubernetes.io/ingress.class: "nginx"
    cert-manager.io/cluster-issuer: "letsencrypt-prod"
    nginx.ingress.kubernetes.io/rate-limit: "100"
    nginx.ingress.kubernetes.io/rate-limit-rps: "10"
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
spec:
  tls:
    - hosts:
        - api.ecommerce.com
      secretName: ecommerce-api-tls
  rules:
    - host: api.ecommerce.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: ecommerce-api-service
                port:
                  number: 80

---
# k8s/hpa.yaml
apiVersion: autoscaling/v2
```

```
kind: HorizontalPodAutoscaler
metadata:
  name: ecommerce-api-hpa
  namespace: ecommerce
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: ecommerce-api
  minReplicas: 3
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 80
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 300
      policies:
        - type: Percent
          value: 100
          periodSeconds: 15
    scaleDown:
      stabilizationWindowSeconds: 300
      policies:
        - type: Percent
          value: 50
          periodSeconds: 60
```

Helm Charts

Helm Chart Structure

```
# helm/Chart.yaml
apiVersion: v2
name: ecommerce-api
description: E-commerce API Helm Chart
version: 0.1.0
appVersion: "1.0.0"
type: application

dependencies:
```

```
- name: postgresql
  version: "12.1.9"
  repository: "https://charts.bitnami.com/bitnami"
  condition: postgresql.enabled
- name: redis
  version: "17.4.3"
  repository: "https://charts.bitnami.com/bitnami"
  condition: redis.enabled

# helm/values.yaml
replicaCount: 3

image:
  repository: ghcr.io/company/ecommerce-api
  pullPolicy: IfNotPresent
  tag: "latest"

nameOverride: ""
fullnameOverride: ""

serviceAccount:
  create: true
  annotations: {}
  name: ""

podAnnotations:
  prometheus.io/scrape: "true"
  prometheus.io/port: "8080"
  prometheus.io/path: "/actuator/prometheus"

podSecurityContext:
  fsGroup: 1001
  runAsUser: 1001
  runAsGroup: 1001
  runAsNonRoot: true

securityContext:
  allowPrivilegeEscalation: false
  readOnlyRootFilesystem: true
  capabilities:
    drop:
      - ALL

service:
  type: ClusterIP
  port: 80
  targetPort: 8080

ingress:
  enabled: true
  className: "nginx"
  annotations:
    cert-manager.io/cluster-issuer: "letsencrypt-prod"
    nginx.ingress.kubernetes.io/rate-limit: "100"
```

```
hosts:
  - host: api.ecommerce.com
  paths:
    - path: /
    pathType: Prefix
tls:
  - secretName: ecommerce-api-tls
  hosts:
    - api.ecommerce.com

resources:
  limits:
    cpu: 500m
    memory: 1Gi
  requests:
    cpu: 250m
    memory: 512Mi

autoscaling:
  enabled: true
  minReplicas: 3
  maxReplicas: 10
  targetCPUUtilizationPercentage: 70
  targetMemoryUtilizationPercentage: 80

nodeSelector: {}
tolerations: []
affinity: {}

# Application specific configuration
app:
  profiles:
    active: "kubernetes"

  database:
    host: "postgresql"
    port: 5432
    name: "ecommerce"

  redis:
    host: "redis-master"
    port: 6379

  secrets:
    dbUsername: "postgres"
    dbPassword: "changeme"
    jwtSecret: "changeme"
    redisPassword: "changeme"

# Dependencies
postgresql:
  enabled: true
  auth:
    postgresPassword: "changeme"
```



```
    database: "ecommerce"
  primary:
    persistence:
      size: 10Gi

  redis:
    enabled: true
    auth:
      password: "changeme"
    master:
      persistence:
        size: 5Gi
```

Monitoring y Observabilidad

Configuración de Prometheus

```
# docker/prometheus.yml
global:
  scrape_interval: 15s
  evaluation_interval: 15s

rule_files:
  - "alerts.yml"

scrape_configs:
  - job_name: "ecommerce-api"
    static_configs:
      - targets: ["app:8080"]
    metrics_path: "/actuator/prometheus"
    scrape_interval: 15s

  - job_name: "postgres-exporter"
    static_configs:
      - targets: ["postgres-exporter:9187"]

  - job_name: "redis-exporter"
    static_configs:
      - targets: ["redis-exporter:9121"]

alerting:
  alertmanagers:
    - static_configs:
        - targets:
            - alertmanager:9093

# alerts.yml
groups:
  - name: ecommerce-api
    rules:
      - alert: HighErrorRate
```

```
    expr: rate(http_requests_total{status=~"5.."}[5m]) > 0.1
    for: 5m
    labels:
      severity: critical
    annotations:
      summary: "High error rate detected"
      description: "Error rate is {{ $value }} errors per second"

- alert: HighResponseTime
  expr: histogram_quantile(0.95,
rate(http_request_duration_seconds_bucket[5m])) > 2
  for: 5m
  labels:
    severity: warning
  annotations:
    summary: "High response time detected"
    description: "95th percentile response time is {{ $value }} seconds"

- alert: DatabaseConnectionPool
  expr: hikaricp_connections_active / hikaricp_connections_max > 0.8
  for: 2m
  labels:
    severity: warning
  annotations:
    summary: "Database connection pool usage high"
    description: "Connection pool usage is {{ $value }}%"
```

Prompts de IA para Módulo 8: DevOps

PROMPT GITHUB COPILOT - DevOps Pipeline Completo:

Actúa como un DevOps Engineer senior especializado en Spring Boot. Necesito implementar un pipeline CI/CD completo para microservicios.

Genera:

1. Docker Strategy:
 - Multi-stage Dockerfile optimizado
 - Security best practices (non-root user, minimal base image)
 - JVM optimization para containers
 - Health checks y graceful shutdown
 - Build cache optimization
2. CI/CD Pipeline avanzado:
 - GitHub Actions con matrix builds
 - Parallel testing (unit, integration, security)
 - Code quality gates (SonarQube, CodeClimate)
 - Container vulnerability scanning
 - Artifact signing y provenance
3. Kubernetes Deployment:

- Complete manifests (Deployment, Service, Ingress, HPA)
- ConfigMaps y Secrets management
- Health probes configuration
- Resource limits y requests
- Network policies y RBAC

4. Monitoring Stack:

- Prometheus metrics configuration
- Grafana dashboards
- Alert rules y notifications
- Log aggregation con ELK/Loki
- Distributed tracing setup

5. Security Hardening:

- Image vulnerability scanning
- Runtime security policies
- Secret rotation strategies
- Network segmentation
- Compliance scanning

Compara cada tool con equivalentes Node.js/JavaScript y explica las ventajas empresariales.

PROMPT CLAUDE - Kubernetes Production Setup:

Eres un especialista en Kubernetes y Spring Boot deployments. Necesito configurar un cluster production-ready para microservicios.

Implementa:

1. Helm Charts avanzados:

- Template functions y helpers
- Multi-environment configurations
- Dependencies management
- Hooks para migrations y setup
- Chart testing strategies

2. Observability completa:

- Distributed tracing con Jaeger/Zipkin
- Custom metrics con Micrometer
- Log correlation y structured logging
- Performance profiling
- Error tracking y alerting

3. Scaling Strategies:

- HPA con custom metrics
- VPA para resource optimization
- Cluster autoscaling
- Pod disruption budgets
- Traffic splitting para canary

4. Security Implementation:

- Pod Security Standards

- Network Policies

- Service Mesh integration (Istio)

- Secrets management con Vault

- Certificate management

5. Disaster Recovery:

- Backup strategies

- Cross-region replication

- Failover automation

- Recovery testing

- RTO/RPO optimization

6. Performance Optimization:

- JVM tuning para containers

- Connection pooling optimization

- Cache strategies

- Database scaling

- CDN integration

Incluye production checklist, troubleshooting guides y performance benchmarks.

Módulo 9: Herramientas y Ecosystem

Herramientas de Desarrollo

En el mundo Node.js/TypeScript, usas herramientas como ESLint, Prettier, Nodemon, y npm scripts. El ecosistema Java tiene herramientas equivalentes y más potentes para el desarrollo empresarial.

Comparación: Node.js vs Java

Concepto	Node.js/TypeScript	Java/Spring Boot
Build Tools	npm, yarn, webpack	Maven, Gradle
Code Quality	ESLint, Prettier	Checkstyle, SpotBugs, PMD
Hot Reload	nodemon, ts-node	Spring Boot DevTools
Dependency Management	package.json, lock files	pom.xml, gradle.build
IDE Support	VS Code, WebStorm	IntelliJ IDEA, Eclipse, VS Code
Profiling	Node.js profiler, clinic.js	JProfiler, VisualVM, async-profiler

Maven vs npm/yarn

```
<!-- pom.xml - Maven configuration -->
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">

<modelVersion>4.0.0</modelVersion>
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.0</version>
    <relativePath/>
</parent>

<groupId>com.company</groupId>
<artifactId>ecommerce-api</artifactId>
<version>1.0.0</version>
<packaging>jar</packaging>
<name>E-commerce API</name>
<description>Spring Boot E-commerce REST API</description>

<properties>
    <java.version>21</java.version>
    <spring-cloud.version>2023.0.0</spring-cloud.version>
    <testcontainers.version>1.19.3</testcontainers.version>
    <maven.compiler.source>21</maven.compiler.source>
    <maven.compiler.target>21</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>

<dependencies>
    <!-- Spring Boot Starters -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-cache</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-actuator</artifactId>
    </dependency>
```

```

<!-- Database -->
<dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>org.flywaydb</groupId>
    <artifactId>flyway-core</artifactId>
</dependency>

<!-- Cache -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>

<!-- Testing -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.testcontainers</groupId>
    <artifactId>junit-jupiter</artifactId>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.testcontainers</groupId>
    <artifactId>postgresql</artifactId>
    <scope>test</scope>
</dependency>

<!-- Development Tools -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-devtools</artifactId>
    <scope>runtime</scope>
    <optional>true</optional>
</dependency>
</dependencies>

<build>
    <plugins>
        <!-- Spring Boot Plugin -->
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
            <configuration>
                <excludes>
                    <exclude>
                        <groupId>org.springframework.boot</groupId>
                        <artifactId>spring-boot-devtools</artifactId>

```

```
        </exclude>
      </excludes>
    </configuration>
  </plugin>

  <!-- Compiler Plugin -->
  <plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-compiler-plugin</artifactId>
    <version>3.11.0</version>
    <configuration>
      <source>21</source>
      <target>21</target>
      <compilerArgs>
        <arg>-parameters</arg>
      </compilerArgs>
    </configuration>
  </plugin>

  <!-- Surefire Plugin for Tests -->
  <plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-surefire-plugin</artifactId>
    <version>3.2.2</version>
    <configuration>
      <includes>
        <include>**/*Test.java</include>
        <include>**/*Tests.java</include>
      </includes>
    </configuration>
  </plugin>

  <!-- JaCoCo for Code Coverage -->
  <plugin>
    <groupId>org.jacoco</groupId>
    <artifactId>jacoco-maven-plugin</artifactId>
    <version>0.8.11</version>
    <executions>
      <execution>
        <goals>
          <goal>prepare-agent</goal>
        </goals>
      </execution>
      <execution>
        <id>report</id>
        <phase>test</phase>
        <goals>
          <goal>report</goal>
        </goals>
      </execution>
    </executions>
  </plugin>
</plugins>
```

```

    </build>
</project>

```

Comparación con package.json:

```

{
  "name": "ecommerce-api",
  "version": "1.0.0",
  "description": "Node.js E-commerce REST API",
  "main": "dist/server.js",
  "scripts": {
    "start": "node dist/server.js",
    "dev": "nodemon src/server.ts",
    "build": "tsc",
    "test": "jest",
    "test:watch": "jest --watch",
    "test:coverage": "jest --coverage",
    "lint": "eslint src/**/*.ts",
    "lint:fix": "eslint src/**/*.ts --fix",
    "format": "prettier --write src/**/*.ts"
  },
  "dependencies": {
    "express": "^4.18.2",
    "typeorm": "^0.3.17",
    "jsonwebtoken": "^9.0.2",
    "bcryptjs": "^2.4.3",
    "redis": "^4.6.8",
    "pg": "^8.11.3"
  },
  "devDependencies": {
    "@types/node": "^20.5.9",
    "typescript": "^5.2.2",
    "nodemon": "^3.0.1",
    "jest": "^29.6.4",
    "eslint": "^8.48.0",
    "prettier": "^3.0.3"
  }
}

```

🔍 Calidad de Código con Maven

```

<!-- Plugins adicionales en pom.xml para calidad de código -->
<plugin>
  <groupId>com.github.spotbugs</groupId>
  <artifactId>spotbugs-maven-plugin</artifactId>
  <version>4.8.2.0</version>
  <configuration>
    <effort>Max</effort>
    <threshold>Low</threshold>
  </configuration>
</plugin>

```



```

        <xmlOutput>true</xmlOutput>
    </configuration>
    <executions>
        <execution>
            <goals>
                <goal>check</goal>
            </goals>
        </execution>
    </executions>
</plugin>

<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-checkstyle-plugin</artifactId>
    <version>3.3.1</version>
    <configuration>
        <configLocation>checkstyle.xml</configLocation>
        <includeTestSourceDirectory>true</includeTestSourceDirectory>
        <failOnViolation>true</failOnViolation>
        <logViolationsToConsole>true</logViolationsToConsole>
    </configuration>
    <executions>
        <execution>
            <id>validate</id>
            <phase>validate</phase>
            <goals>
                <goal>check</goal>
            </goals>
        </execution>
    </executions>
</plugin>

<plugin>
    <groupId>com.github.ekryd.sortpom</groupId>
    <artifactId>sortpom-maven-plugin</artifactId>
    <version>3.3.0</version>
    <configuration>
        <sortProperties>true</sortProperties>
        <keepBlankLines>true</keepBlankLines>
        <sortDependencies>scope</sortDependencies>
    </configuration>
    <executions>
        <execution>
            <goals>
                <goal>verify</goal>
            </goals>
        </execution>
    </executions>
</plugin>

```

⚡ Spring Boot DevTools - Hot Reload

Spring Boot DevTools es el equivalente a **nodemon** en Node.js, pero con funcionalidades más avanzadas.

Comparación: nodemon vs DevTools

```
<!-- Ya incluido en dependencies -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-devtools</artifactId>
  <scope>runtime</scope>
  <optional>true</optional>
</dependency>
```

Configuración en application.yml:

```
# application-dev.yml
spring:
  devtools:
    restart:
      enabled: true
      exclude: static/**,public/**
      additional-paths: src/main/java
      poll-interval: 1s
      quiet-period: 400ms
    livereload:
      enabled: true
      port: 35729
    remote:
      secret: myremotesecret
      context-path: /myapp

  jpa:
    show-sql: true
    hibernate:
      ddl-auto: update

  h2:
    console:
      enabled: true
      path: /h2-console
```

Comparación con nodemon.json:

```
{
  "watch": ["src"],
  "ext": "ts,js",
  "ignore": ["src/**/*.spec.ts"],
  "exec": "ts-node src/server.ts",
  "env": {
```

```
"NODE_ENV": "development"
},
"delay": "1000"
}
```

⚙ Configuración de IDEs

IntelliJ IDEA - Settings recomendados:

```
# .idea/workspace.xml customizations
# Enable automatic compilation
compiler.automake.allow.when.app.running=true

# Enable annotation processing
annotation.processing.enabled=true

# Spring Boot DevTools settings
spring.devtools.restart.enabled=true
spring.devtools.restart.poll-interval=1s

# Code style settings
editor.codeStyle.defaultRightMargin=120
editor.inspections.enabled=true
```

VS Code - settings.json para Java:

```
{
  "java.configuration.updateBuildConfiguration": "automatic",
  "java.compile.nullAnalysis.mode": "automatic",
  "java.saveActions.organizeImports": true,
  "java.format.enabled": true,
  "java.format.onType.enabled": true,
  "spring-boot.ls.checkstyle.on": true,
  "spring-boot.ls.problem.application-properties.on": true,
  "files.exclude": {
    "**/target": true,
    "**/.classpath": true,
    "**/.project": true,
    "**/.settings": true,
    "**/.factorypath": true
  },
  "java.test.config": {
    "workingDirectory": "${workspaceFolder}"
  },
  "spring.initializr.defaultLanguage": "Java",
  "spring.initializr.defaultJavaVersion": "21",
  "spring.initializr.defaultPackaging": "Jar"
}
```

Profiles y Configuración por Ambiente

```
# application.yml (común)
spring:
  application:
    name: ecommerce-api
  profiles:
    active: ${SPRING_PROFILES_ACTIVE:dev}

server:
  port: ${SERVER_PORT:8080}
  servlet:
    context-path: /api

management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,loggers,env
  endpoint:
    health:
      show-details: when-authorized

---
# application-dev.yml
spring:
  config:
    activate:
      on-profile: dev

  datasource:
    url: jdbc:h2:mem:devdb
    driver-class-name: org.h2.Driver
    username: sa
    password:

  jpa:
    hibernate:
      ddl-auto: create-drop
    show-sql: true

  h2:
    console:
      enabled: true

logging:
  level:
    com.company.ecommerce: DEBUG
    org.springframework.web: DEBUG

---
# application-test.yml
```

```
spring:
  config:
    activate:
      on-profile: test

  datasource:
    url: jdbc:h2:mem:testdb
    driver-class-name: org.h2.Driver

  jpa:
    hibernate:
      ddl-auto: create-drop
      show-sql: false

logging:
  level:
    root: WARN
    com.company.ecommerce: INFO

---
# application-prod.yml
spring:
  config:
    activate:
      on-profile: prod

  datasource:
    url: ${DATABASE_URL}
    username: ${DATABASE_USERNAME}
    password: ${DATABASE_PASSWORD}
    hikari:
      maximum-pool-size: 20
      minimum-idle: 5

  jpa:
    hibernate:
      ddl-auto: validate
      show-sql: false

  redis:
    host: ${REDIS_HOST}
    port: ${REDIS_PORT:6379}
    password: ${REDIS_PASSWORD}

logging:
  level:
    root: WARN
    com.company.ecommerce: INFO

management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
```

Comparación con Node.js environments:

```
// Node.js config/index.ts
interface Config {
  port: number;
  database: {
    url: string;
    ssl: boolean;
  };
  redis: {
    url: string;
  };
  jwt: {
    secret: string;
    expiresIn: string;
  };
}

const configs: Record<string, Config> = {
  development: {
    port: 3000,
    database: {
      url: "postgresql://localhost:5432/ecommerce_dev",
      ssl: false,
    },
    redis: {
      url: "redis://localhost:6379",
    },
    jwt: {
      secret: "dev-secret",
      expiresIn: "24h",
    },
  },
  production: {
    port: parseInt(process.env.PORT || "8080"),
    database: {
      url: process.env.DATABASE_URL!,
      ssl: true,
    },
    redis: {
      url: process.env.REDIS_URL!,
    },
    jwt: {
      secret: process.env.JWT_SECRET!,
      expiresIn: "1h",
    },
  },
};

export default configs[process.env.NODE_ENV || "development"];
```

⚙ Comandos Maven Útiles

```
# Comandos básicos de desarrollo
./mvnw spring-boot:run                                # Ejecutar aplicación
./mvnw spring-boot:run -Dspring-boot.run.profiles=dev  # Con profile específico
./mvnw spring-boot:run -Dspring-boot.run.jvmArguments="-Xdebug -Xrunjdwp:transport=dt_socket,server=y,suspend=n,address=5005"

# Testing
./mvnw test                                             # Ejecutar tests
./mvnw test -Dtest=UserServiceTest                    # Test específico
./mvnw test -Dspring.profiles.active=test              # Con profile de test
./mvnw jacoco:report                                   # Generar reporte de coverage

# Build y packaging
./mvnw clean compile                                  # Compilar
./mvnw clean package                                  # Generar JAR
./mvnw clean package -DskipTests                      # Sin tests
./mvnw clean install                                  # Instalar en repositorio local

# Code quality
./mvnw checkstyle:check                               # Verificar estilo
./mvnw spotbugs:check                                 # Análisis de bugs
./mvnw sonar:sonar                                    # SonarQube analysis

# Dependencies
./mvnw dependency:tree                                # Ver árbol de dependencias
./mvnw dependency:analyze                             # Analizar dependencias
./mvnw versions:display-dependency-updates            # Ver actualizaciones disponibles

# Development helpers
./mvnw spring-boot:build-info                          # Generar build info
./mvnw spring-boot:repackage                          # Reempaquetar JAR
./mvnw clean spring-boot:run -Ddebug                  # Debug mode
```

Comparación con npm scripts:

```
{
  "scripts": {
    "dev": "nodemon src/server.ts",
    "build": "tsc",
    "start": "node dist/server.js",
    "test": "jest",
    "test:watch": "jest --watch",
    "test:coverage": "jest --coverage",
    "lint": "eslint src/**/*.ts",
    "lint:fix": "eslint src/**/*.ts --fix",
    "format": "prettier --write src/**/*.ts",
    "typecheck": "tsc --noEmit"
```

```
}  
}
```

🔍 Profiling y Debugging

🐛 Debugging con IntelliJ IDEA

```
// Configuración de debugging en application-debug.yml  
spring:  
  profiles:  
    active: debug  
  
logging:  
  level:  
    com.company.ecommerce: DEBUG  
    org.springframework.security: DEBUG  
    org.springframework.web: DEBUG  
    org.hibernate.SQL: DEBUG  
    org.hibernate.type.descriptor.sql.BasicBinder: TRACE  
  
management:  
  endpoints:  
    web:  
      exposure:  
        include: "*"br/>  endpoint:  
    health:  
      show-details: always  
  loggers:  
    enabled: true  
  
# JVM debug options  
# -agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=*:5005
```

Comparación con Node.js debugging:

```
// launch.json para VS Code  
{  
  "version": "0.2.0",  
  "configurations": [  
    {  
      "name": "Debug Node.js",  
      "type": "node",  
      "request": "launch",  
      "program": "${workspaceFolder}/src/server.ts",  
      "preLaunchTask": "tsc: build",  
      "outFiles": ["${workspaceFolder}/dist/**/*.js"],  
      "envFile": "${workspaceFolder}/.env.debug"  
    }  
  ]  
}
```



```
]
}
```

Profiling con JProfiler/VisualVM

```
// Configuración de JVM para profiling
@Component
@Profile("profiling")
public class ProfilingConfig {

    @EventListener
    public void handleContextRefresh(ContextRefreshedEvent event) {
        // Enable JFR (Java Flight Recorder)
        System.setProperty("java.util.logging.manager",
            "org.apache.logging.log4j.jul.LogManager");
    }

    @Bean
    @ConditionalOnProperty("profiling.jmx.enabled")
    public MBeanExporter mBeanExporter() {
        MBeanExporter exporter = new MBeanExporter();
        exporter.setDefaultDomain("ecommerce-api");
        return exporter;
    }
}

// Custom metrics para profiling
@Component
public class PerformanceMetrics {

    private final MeterRegistry meterRegistry;
    private final Timer.Sample sample;

    public PerformanceMetrics(MeterRegistry meterRegistry) {
        this.meterRegistry = meterRegistry;
        this.sample = Timer.start(meterRegistry);
    }

    @EventListener
    public void onApplicationReady(ApplicationReadyEvent event) {
        sample.stop(Timer.builder("application.startup.time")
            .description("Time taken to start the application")
            .register(meterRegistry));
    }

    public void recordDatabaseQuery(String query, long duration) {
        Timer.builder("database.query.time")
            .tag("query", query)
            .register(meterRegistry)
            .record(duration, TimeUnit.MILLISECONDS);
    }
}
```

```
}
}
```

Database Migration con Flyway

Configuración de Flyway

```
<!-- pom.xml - Flyway plugin -->
<plugin>
  <groupId>org.flywaydb</groupId>
  <artifactId>flyway-maven-plugin</artifactId>
  <version>9.22.3</version>
  <configuration>
    <url>jdbc:postgresql://localhost:5432/ecommerce</url>
    <user>postgres</user>
    <password>postgres</password>
    <schemas>
      <schema>public</schema>
    </schemas>
    <locations>
      <location>classpath:db/migration</location>
    </locations>
  </configuration>
</plugin>
```

```
# application.yml - Flyway configuration
spring:
  flyway:
    enabled: true
    locations: classpath:db/migration
    baseline-on-migrate: true
    validate-on-migrate: true
    out-of-order: false
    clean-disabled: true
    schemas: public
```

Migration Files Structure

```
-- src/main/resources/db/migration/V001__Create_users_table.sql
CREATE TABLE users (
  id BIGSERIAL PRIMARY KEY,
  username VARCHAR(50) UNIQUE NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  password VARCHAR(255) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```

);

CREATE INDEX idx_users_username ON users(username);
CREATE INDEX idx_users_email ON users(email);

-- src/main/resources/db/migration/V002__Create_roles_table.sql
CREATE TABLE roles (
    id BIGSERIAL PRIMARY KEY,
    name VARCHAR(50) UNIQUE NOT NULL
);

CREATE TABLE user_roles (
    user_id BIGINT REFERENCES users(id) ON DELETE CASCADE,
    role_id BIGINT REFERENCES roles(id) ON DELETE CASCADE,
    PRIMARY KEY (user_id, role_id)
);

INSERT INTO roles (name) VALUES ('ROLE_USER'), ('ROLE_ADMIN'), ('ROLE_MODERATOR');

-- src/main/resources/db/migration/V003__Create_products_table.sql
CREATE TABLE categories (
    id BIGSERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    description TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE products (
    id BIGSERIAL PRIMARY KEY,
    name VARCHAR(200) NOT NULL,
    description TEXT,
    price DECIMAL(10,2) NOT NULL,
    stock_quantity INTEGER NOT NULL DEFAULT 0,
    category_id BIGINT REFERENCES categories(id),
    is_active BOOLEAN DEFAULT true,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_products_category ON products(category_id);
CREATE INDEX idx_products_price ON products(price);
CREATE INDEX idx_products_active ON products(is_active);

```

Comparación con TypeORM migrations:

```

// Node.js/TypeORM migration
import { MigrationInterface, QueryRunner, Table, Index } from "typeorm";

export class CreateUsersTable1699123456789 implements MigrationInterface {
    public async up(queryRunner: QueryRunner): Promise<void> {
        await queryRunner.createTable(
            new Table({

```

```

        name: "users",
        columns: [
            {
                name: "id",
                type: "bigint",
                isPrimary: true,
                isGenerated: true,
                generationStrategy: "increment",
            },
            {
                name: "username",
                type: "varchar",
                length: "50",
                isUnique: true,
            },
            {
                name: "email",
                type: "varchar",
                length: "100",
                isUnique: true,
            },
            {
                name: "password",
                type: "varchar",
                length: "255",
            },
            {
                name: "created_at",
                type: "timestamp",
                default: "CURRENT_TIMESTAMP",
            },
        ],
    })
);

    await queryRunner.createIndex("users", new Index("idx_users_username",
["username"]));
}

    public async down(queryRunner: QueryRunner): Promise<void> {
        await queryRunner.dropTable("users");
    }
}

```

API Documentation con OpenAPI

Configuración de OpenAPI 3

```

<!-- pom.xml dependency -->
<dependency>
    <groupId>org.springdoc</groupId>

```

```

<artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
<version>2.2.0</version>
</dependency>

```

```

// OpenAPI Configuration
@Configuration
public class OpenApiConfig {

    @Bean
    public OpenAPI customOpenAPI() {
        return new OpenAPI()
            .info(new Info()
                .title("E-commerce API")
                .version("1.0.0")
                .description("Spring Boot E-commerce REST API with
comprehensive endpoints")
                .contact(new Contact()
                    .name("Development Team")
                    .email("dev@company.com")
                    .url("https://company.com"))
                .license(new License()
                    .name("MIT License")
                    .url("https://opensource.org/licenses/MIT")))
            .servers(Arrays.asList(
                new
Server().url("http://localhost:8080").description("Development server"),
                new
Server().url("https://api.company.com").description("Production server")))
            .components(new Components()
                .addSecuritySchemes("bearerAuth", new SecurityScheme()
                    .type(SecurityScheme.Type.HTTP)
                    .scheme("bearer")
                    .bearerFormat("JWT")
                    .description("JWT Authentication")))
            .addSecurityItem(new SecurityRequirement().addList("bearerAuth")));
    }

    @Bean
    public GroupedOpenApi publicApi() {
        return GroupedOpenApi.builder()
            .group("public")
            .pathsToMatch("/api/public/**")
            .build();
    }

    @Bean
    public GroupedOpenApi adminApi() {
        return GroupedOpenApi.builder()
            .group("admin")
            .pathsToMatch("/api/admin/**")
            .build();
    }
}

```

```

    }

    @Bean
    public GroupedOpenApi userApi() {
        return GroupedOpenApi.builder()
            .group("user")
            .pathsToMatch("/api/user/**", "/api/products/**")
            .build();
    }
}

```

Documentación de Controllers

```

@RestController
@RequestMapping("/api/products")
@Tag(name = "Products", description = "Product management endpoints")
@SecurityRequirement(name = "bearerAuth")
public class ProductController {

    @GetMapping
    @Operation(
        summary = "Get all products",
        description = "Retrieve a paginated list of all products with optional filtering",
        responses = {
            @ApiResponse(responseCode = "200", description = "Products retrieved successfully",
                content = @Content(schema = @Schema(implementation = PageProductDto.class))),
            @ApiResponse(responseCode = "400", description = "Invalid request parameters",
                content = @Content(schema = @Schema(implementation = ErrorResponse.class)))
        }
    )
    public ResponseEntity<Page<ProductDto>> getAllProducts(
        @Parameter(description = "Page number (0-based)", example = "0")
        @RequestParam(defaultValue = "0") int page,

        @Parameter(description = "Page size", example = "20")
        @RequestParam(defaultValue = "20") int size,

        @Parameter(description = "Sort field", example = "name")
        @RequestParam(defaultValue = "id") String sortBy,

        @Parameter(description = "Sort direction", example = "asc")
        @RequestParam(defaultValue = "asc") String sortDir,

        @Parameter(description = "Filter by category ID", example = "1")
        @RequestParam(required = false) Long categoryId,
    )
}

```

```

        @Parameter(description = "Filter by minimum price", example = "10.00")
        @RequestParam(required = false) BigDecimal minPrice,

        @Parameter(description = "Filter by maximum price", example =
"100.00")
        @RequestParam(required = false) BigDecimal maxPrice) {

    Page<ProductDto> products = productService.getAllProducts(
        page, size, sortBy, sortDir, categoryId, minPrice, maxPrice);
    return ResponseEntity.ok(products);
}

@PostMapping
@Operation(
    summary = "Create a new product",
    description = "Create a new product with the provided details",
    responses = {
        @ApiResponse(responseCode = "201", description = "Product created
successfully",
            content = @Content(schema = @Schema(implementation =
ProductDto.class))),
        @ApiResponse(responseCode = "400", description = "Invalid product
data",
            content = @Content(schema = @Schema(implementation =
ErrorResponse.class))),
        @ApiResponse(responseCode = "403", description = "Access denied -
Admin role required",
            content = @Content(schema = @Schema(implementation =
ErrorResponse.class)))
    }
)
@PreAuthorize("hasRole('ADMIN')")
public ResponseEntity<ProductDto> createProduct(
    @Valid @RequestBody
    @io.swagger.v3.oas.annotations.parameters.RequestBody(
        description = "Product details to create",
        required = true,
        content = @Content(schema = @Schema(implementation =
CreateProductRequest.class))
    ) CreateProductRequest request) {

    ProductDto createdProduct = productService.createProduct(request);
    return ResponseEntity.status(HttpStatus.CREATED).body(createdProduct);
}

// DTO Documentation
@Schema(description = "Product data transfer object")
public class ProductDto {

    @Schema(description = "Product unique identifier", example = "1")
    private Long id;

    @Schema(description = "Product name", example = "Laptop Gaming", required =

```

```

true)
    private String name;

    @Schema(description = "Product description", example = "High-performance
gaming laptop")
    private String description;

    @Schema(description = "Product price", example = "999.99", required = true)
    private BigDecimal price;

    @Schema(description = "Available stock quantity", example = "50")
    private Integer stockQuantity;

    @Schema(description = "Product category")
    private CategoryDto category;

    @Schema(description = "Product creation timestamp", example = "2023-12-
01T10:30:00Z")
    private LocalDateTime createdAt;

    // getters and setters
}

```

Comparación con Swagger en Node.js:

```

// Node.js with swagger-jsdoc
/**
 * @swagger
 * /api/products:
 *   get:
 *     summary: Get all products
 *     description: Retrieve a paginated list of products
 *     parameters:
 *       - in: query
 *         name: page
 *         schema:
 *           type: integer
 *           default: 0
 *         description: Page number
 *     responses:
 *       200:
 *         description: Products retrieved successfully
 *         content:
 *           application/json:
 *             schema:
 *               type: object
 *               properties:
 *                 content:
 *                   type: array
 *                   items:
 *                     $ref: '#/components/schemas/Product'
 */

```



```
app.get("/api/products", (req, res) => {  
    // implementation  
});
```

Monitoring Tools y Actuator

Spring Boot Actuator Avanzado

```
// Configuración completa de Actuator  
@Configuration  
public class ActuatorConfig {  
  
    @Bean  
    public HealthIndicator customHealthIndicator() {  
        return new CompositeHealthIndicator(new OrderedHealthAggregator())  
            .withDetail("database", databaseHealthIndicator())  
            .withDetail("redis", redisHealthIndicator())  
            .withDetail("external-api", externalApiHealthIndicator());  
    }  
  
    @Component  
    public class DatabaseHealthIndicator implements HealthIndicator {  
  
        @Autowired  
        private DataSource dataSource;  
  
        @Override  
        public Health health() {  
            try (Connection connection = dataSource.getConnection()) {  
                if (connection.isValid(1)) {  
                    return Health.up()  
                        .withDetail("database", "Available")  
                        .withDetail("connection-pool",  
getConnectionPoolInfo())  
                        .withDetail("query-time", measureQueryTime())  
                        .build();  
                }  
            } catch (SQLException e) {  
                return Health.down()  
                    .withDetail("database", "Unavailable")  
                    .withException(e)  
                    .build();  
            }  
            return Health.down().withDetail("database", "Unknown").build();  
        }  
  
        private Map<String, Object> getConnectionPoolInfo() {  
            // HikariCP pool info  
            Map<String, Object> poolInfo = new HashMap<>();  
            poolInfo.put("active", 5);  
            poolInfo.put("idle", 3);  
        }  
    }  
}
```

```

        poolInfo.put("max", 10);
        return poolInfo;
    }

    private long measureQueryTime() {
        long start = System.currentTimeMillis();
        // Execute simple query
        return System.currentTimeMillis() - start;
    }
}

@Component
public class ExternalApiHealthIndicator implements HealthIndicator {

    @Autowired
    private WebClient webClient;

    @Override
    public Health health() {
        try {
            long start = System.currentTimeMillis();
            String response = webClient.get()
                .uri("https://api.external-service.com/health")
                .retrieve()
                .bodyToMono(String.class)
                .timeout(Duration.ofSeconds(5))
                .block();

            long responseTime = System.currentTimeMillis() - start;

            return Health.up()
                .withDetail("external-api", "Available")
                .withDetail("response-time", responseTime + "ms")
                .withDetail("response", response)
                .build();

        } catch (Exception e) {
            return Health.down()
                .withDetail("external-api", "Unavailable")
                .withDetail("error", e.getMessage())
                .build();
        }
    }
}

// Custom Endpoints
@Component
@Endpoint(id = "custom-info")
public class CustomInfoEndpoint {

    @ReadOperation
    public Map<String, Object> info() {
        Map<String, Object> info = new HashMap<>();
    }
}

```

```

        info.put("application", getApplicationInfo());
        info.put("system", getSystemInfo());
        info.put("business", getBusinessMetrics());
        return info;
    }

    @ReadOperation
    public Map<String, Object> businessMetrics(@Selector String metric) {
        Map<String, Object> metrics = new HashMap<>();

        switch (metric) {
            case "users":
                metrics.put("total-users", userService.getTotalUsers());
                metrics.put("active-users", userService.getActiveUsers());
                metrics.put("new-users-today", userService.getNewUsersToday());
                break;
            case "orders":
                metrics.put("total-orders", orderService.getTotalOrders());
                metrics.put("pending-orders", orderService.getPendingOrders());
                metrics.put("revenue-today", orderService.getTodayRevenue());
                break;
        }

        return metrics;
    }

    private Map<String, Object> getApplicationInfo() {
        Map<String, Object> app = new HashMap<>();
        app.put("name", "E-commerce API");
        app.put("version", "1.0.0");
        app.put("build-time", "2023-12-01T10:30:00Z");
        app.put("spring-boot-version", SpringBootVersion.getVersion());
        return app;
    }

    private Map<String, Object> getSystemInfo() {
        Map<String, Object> system = new HashMap<>();
        Runtime runtime = Runtime.getRuntime();

        system.put("cpu-cores", runtime.availableProcessors());
        system.put("max-memory", runtime.maxMemory() / 1024 / 1024 + " MB");
        system.put("free-memory", runtime.freeMemory() / 1024 / 1024 + " MB");
        system.put("used-memory", (runtime.totalMemory() - runtime.freeMemory()) /
1024 / 1024 + " MB");
        system.put("java-version", System.getProperty("java.version"));
        system.put("os", System.getProperty("os.name"));

        return system;
    }
}

```

```
@Component
public class BusinessMetrics {

    private final MeterRegistry meterRegistry;
    private final Counter userRegistrations;
    private final Counter orderCreations;
    private final Gauge activeUsers;
    private final Timer orderProcessingTime;

    public BusinessMetrics(MeterRegistry meterRegistry) {
        this.meterRegistry = meterRegistry;

        this.userRegistrations = Counter.builder("user.registrations")
            .description("Number of user registrations")
            .tag("type", "business")
            .register(meterRegistry);

        this.orderCreations = Counter.builder("order creations")
            .description("Number of orders created")
            .tag("type", "business")
            .register(meterRegistry);

        this.activeUsers = Gauge.builder("users.active")
            .description("Number of currently active users")
            .register(meterRegistry, this,
BusinessMetrics::getActiveUserCount);

        this.orderProcessingTime = Timer.builder("order.processing.time")
            .description("Time taken to process orders")
            .register(meterRegistry);
    }

    public void recordUserRegistration(String userType) {
        userRegistrations.increment(Tags.of("user_type", userType));
    }

    public void recordOrderCreation(String orderType, BigDecimal amount) {
        orderCreations.increment(
            Tags.of(
                "order_type", orderType,
                "amount_range", getAmountRange(amount)
            )
        );
    }

    public void recordOrderProcessingTime(Duration duration) {
        orderProcessingTime.record(duration);
    }

    private double getActiveUserCount() {
        // Logic to get active user count
        return userService.getActiveUserCount();
    }
}
```

```

    }

    private String getAmountRange(BigDecimal amount) {
        if (amount.compareTo(BigDecimal.valueOf(100)) < 0) return "0-100";
        if (amount.compareTo(BigDecimal.valueOf(500)) < 0) return "100-500";
        if (amount.compareTo(BigDecimal.valueOf(1000)) < 0) return "500-1000";
        return "1000+";
    }

    // Custom histogram for response sizes
    @EventListener
    public void handleHttpRequest(HttpRequestEvent event) {
        DistributionSummary.builder("http.response.size")
            .description("HTTP response size in bytes")
            .tag("method", event.getMethod())
            .tag("status", String.valueOf(event.getStatus()))
            .register(meterRegistry)
            .record(event.getResponseSize());
    }
}

```

Alerting y Monitoring Configuration

```

# application-monitoring.yml
management:
  endpoints:
    web:
      exposure:
        include: "*"
      base-path: /actuator
    jmx:
      exposure:
        include: "*"

  endpoint:
    health:
      show-details: always
      show-components: always
      group:
        live:
          include: ping,diskSpace
        ready:
          include: db,redis
  metrics:
    enabled: true
  prometheus:
    enabled: true
  loggers:
    enabled: true
  env:
    enabled: true

```

```
    show-values: never

health:
  circuitbreakers:
    enabled: true
  ratelimiters:
    enabled: true
  diskspace:
    enabled: true
    threshold: 10GB

metrics:
  enable:
    all: true
  export:
    prometheus:
      enabled: true
      step: 1m
    influx:
      enabled: false
  distribution:
    percentiles-histogram:
      http.server.requests: true
      order.processing.time: true
    percentiles:
      http.server.requests: 0.5, 0.9, 0.95, 0.99
      order.processing.time: 0.5, 0.9, 0.95, 0.99
  slo:
    http.server.requests: 100ms,200ms,500ms,1s,2s

info:
  build:
    enabled: true
  env:
    enabled: true
  java:
    enabled: true
  os:
    enabled: true

logging:
  level:
    org.springframework.boot.actuate: INFO
    io.micrometer: INFO
```

⚡ Performance Optimization

🔑 JVM Tuning para Production

```
# JVM options para diferentes escenarios
# Production (High Memory)
```

```

JAVA_OPTS="-Xms2g -Xmx4g \
    -XX:+UseG1GC \
    -XX:MaxGCPauseMillis=100 \
    -XX:+UseStringDeduplication \
    -XX:+UseContainerSupport \
    -XX:MaxRAMPercentage=75.0 \
    -XX:+HeapDumpOnOutOfMemoryError \
    -XX:HeapDumpPath=/tmp/heapdump.hprof \
    -XX:+ExitOnOutOfMemoryError \
    -Djava.security.egd=file:/dev/./urandom"

```

Container (Limited Memory)

```

JAVA_OPTS="-XX:+UseContainerSupport \
    -XX:MaxRAMPercentage=75.0 \
    -XX:+UseG1GC \
    -XX:+UseStringDeduplication \
    -XX:+PrintGCDetails \
    -XX:+PrintGCTimeStamps \
    -Xlog:gc:gc.log"

```

Development (Fast Startup)

```

JAVA_OPTS="-Xms512m -Xmx1g \
    -XX:+UseG1GC \
    -XX:+UseStringDeduplication \
    -XX:TieredStopAtLevel=1 \
    -noverify"

```

Database Performance Optimization

```

// HikariCP Configuration
@Configuration
public class DatabaseConfig {

    @Bean
    @Primary
    @ConfigurationProperties("spring.datasource.hikari")
    public HikariConfig hikariConfig() {
        HikariConfig config = new HikariConfig();

        // Connection pool settings
        config.setMaximumPoolSize(20);
        config.setMinimumIdle(5);
        config.setConnectionTimeout(30000);
        config.setIdleTimeout(600000);
        config.setMaxLifetime(1800000);
        config.setLeakDetectionThreshold(60000);

        // Performance settings
        config.setAutoCommit(false);
        config.setConnectionTestQuery("SELECT 1");
        config.setValidationTimeout(5000);
    }
}

```

```
// Monitoring
config.setRegisterMbeans(true);
config.setPoolName("HikariPool-EcommerceAPI");

return config;
}

@Bean
public DataSource dataSource() {
    return new HikariDataSource(hikariConfig());
}
}

// Query Optimization
@Repository
public class ProductRepositoryCustomImpl implements ProductRepositoryCustom {

    @PersistenceContext
    private EntityManager entityManager;

    @Override
    public Page<Product> findProductsOptimized(ProductSearchCriteria criteria,
        Pageable pageable) {
        // Use Criteria API for dynamic queries
        CriteriaBuilder cb = entityManager.getCriteriaBuilder();
        CriteriaQuery<Product> query = cb.createQuery(Product.class);
        Root<Product> product = query.from(Product.class);

        // Join fetch to avoid N+1 problem
        product.fetch("category", JoinType.LEFT);
        product.fetch("images", JoinType.LEFT);

        List<Predicate> predicates = new ArrayList<>();

        if (criteria.getCategoryId() != null) {
            predicates.add(cb.equal(product.get("category").get("id"),
criteria.getCategoryId()));
        }

        if (criteria.getMinPrice() != null) {
            predicates.add(cb.greaterThanOrEqualTo(product.get("price"),
criteria.getMinPrice()));
        }

        if (criteria.getMaxPrice() != null) {
            predicates.add(cb.lessThanOrEqualTo(product.get("price"),
criteria.getMaxPrice()));
        }

        if (StringUtils.hasText(criteria.getSearchTerm())) {
            predicates.add(cb.like(cb.lower(product.get("name")),
                "%" + criteria.getSearchTerm().toLowerCase() +
                "%"));
        }
    }
}
```



```
}

query.where(predicates.toArray(new Predicate[0]));

// Apply sorting
if (pageable.getSort().isSorted()) {
    List<Order> orders = pageable.getSort().stream()
        .map(order -> order.isAscending() ?
            cb.asc(product.get(order.getProperty())) :
            cb.desc(product.get(order.getProperty())))
        .collect(Collectors.toList());
    query.orderBy(orders);
}

TypedQuery<Product> typedQuery = entityManager.createQuery(query);
typedQuery.setFirstResult((int) pageable.getOffset());
typedQuery.setMaxResults(pageable.getPageSize());

List<Product> products = typedQuery.getResultList();

// Count query for pagination
CriteriaQuery<Long> countQuery = cb.createQuery(Long.class);
Root<Product> countRoot = countQuery.from(Product.class);
countQuery.select(cb.count(countRoot));
countQuery.where(predicates.toArray(new Predicate[0]));

Long total = entityManager.createQuery(countQuery).getSingleResult();

return new PageImpl<>(products, pageable, total);
}
```

Prompts de IA para Módulo 9: Herramientas

PROMPT GITHUB COPILOT - Herramientas Development Stack:

Actúa como un lead developer especializado en Spring Boot tooling. Necesito configurar un environment de desarrollo completo y productivo.

Genera:

1. Maven configuration avanzada:
 - Multi-module project setup
 - Profile-specific configurations
 - Plugin management optimizado
 - Dependency version management
 - Build optimization strategies
2. IDE Integration completa:
 - IntelliJ IDEA configurations
 - VS Code extensions y settings

- Debug configurations
 - Code templates y live templates
 - Inspection profiles
3. Development Tools:
 - Spring Boot DevTools configuración
 - Hot reload optimization
 - Database migration con Flyway
 - Test data management
 - Local development setup
 4. Code Quality Pipeline:
 - Checkstyle, SpotBugs, PMD integration
 - SonarQube configuration
 - JaCoCo coverage setup
 - Git hooks pre-commit
 - CI/CD quality gates
 5. Monitoring y Performance:
 - Actuator endpoints customization
 - Micrometer metrics completos
 - Performance profiling setup
 - JVM tuning recommendations
 - Database optimization

Compara cada herramienta con equivalentes Node.js y explica las ventajas del ecosistema Java.

PROMPT CLAUDE - OpenAPI y Documentation:

Eres un API architect especializado en documentación y OpenAPI. Necesito implementar documentación API empresarial completa para microservicios Spring Boot.

Implementa:

1. OpenAPI 3.0 Configuration:
 - Multi-server setup (dev, staging, prod)
 - Security schemes (JWT, OAuth2, API Key)
 - Global response examples
 - Error response schemas
 - API versioning strategy
2. Advanced Documentation:
 - Controller documentation completa
 - DTO schemas con validaciones
 - Response examples realistas
 - Error scenarios documentation
 - API testing examples
3. Documentation Automation:

- Maven plugin integration
 - CI/CD documentation generation
 - Multi-format export (HTML, PDF, Postman)
 - Changelog automation
 - Breaking changes detection
4. Developer Experience:
- Interactive API explorer
 - Code generation para clientes
 - SDK generation automático
 - Testing playground
 - Mock server generation
5. Enterprise Features:
- API governance policies
 - Documentation review process
 - Version compatibility matrix
 - Migration guides
 - Performance benchmarks

Incluye best practices, automation scripts y integration con API gateways.

🎯 Módulo 10: Observabilidad y Monitoring

📊 Fundamentos de Observabilidad

En el ecosistema Node.js/TypeScript, probablemente has usado herramientas como Winston para logs, New Relic para APM, y métricas básicas. Spring Boot con Micrometer ofrece un ecosistema completo de observabilidad empresarial.

🔍 Comparación: Node.js vs Spring Boot

Concepto	Node.js/TypeScript	Spring Boot/Java
Logging	Winston, Bunyan, Pino	Logback, Log4j2, SLF4J
Metrics	prom-client, StatsD	Micrometer + Prometheus
Tracing	OpenTelemetry, Jaeger	Spring Cloud Sleuth + Zipkin
APM	New Relic, DataDog	Micrometer + Grafana + Prometheus
Health Checks	Custom endpoints	Spring Boot Actuator
Error Tracking	Sentry, Bugsnag	Sentry + custom handlers
Dashboard	Grafana, Custom	Grafana + Spring Boot Admin

🏛️ Los Tres Pilares de Observabilidad

1. Logs (Structured Logging) 2. Metrics (Time Series Data)
3. Traces (Distributed Request Tracking)

Logging Avanzado con Logback

Configuración de Logback

```
<!-- src/main/resources/logback-spring.xml -->
<?xml version="1.0" encoding="UTF-8"?>
<configuration>

    <!-- Properties -->
    <property name="LOG_PATH" value="${LOG_PATH:-./logs}"/>
    <property name="APP_NAME" value="ecommerce-api"/>

    <!-- Console Appender for Development -->
    <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
        <encoder
class="net.logstash.logback.encoder.LoggingEventCompositeJsonEncoder">
            <providers>
                <timestamp/>
                <version/>
                <logLevel/>
                <loggerName/>
                <mdc/>
                <arguments/>
                <message/>
                <stackTrace/>
            </providers>
        </encoder>
    </appender>

    <!-- File Appender for Production -->
    <appender name="FILE" class="ch.qos.logback.core.rolling.RollingFileAppender">
        <file>${LOG_PATH}/${APP_NAME}.log</file>
        <encoder
class="net.logstash.logback.encoder.LoggingEventCompositeJsonEncoder">
            <providers>
                <timestamp/>
                <version/>
                <logLevel/>
                <loggerName/>
                <mdc/>
                <arguments/>
                <message/>
                <stackTrace/>
            </providers>
        </encoder>
        <rollingPolicy class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">
            <fileNamePattern>${LOG_PATH}/${APP_NAME}.%d{yyyy-MM-
dd}.%i.log.gz</fileNamePattern>
            <maxFileSize>100MB</maxFileSize>
            <maxHistory>30</maxHistory>
            <totalSizeCap>3GB</totalSizeCap>
        </rollingPolicy>
```

```

</appender>

<!-- Async Appender for Performance -->
<appender name="ASYNC_FILE" class="ch.qos.logback.classic.AsyncAppender">
  <appender-ref ref="FILE"/>
  <queueSize>512</queueSize>
  <discardingThreshold>0</discardingThreshold>
  <includeCallerData>false</includeCallerData>
</appender>

<!-- Error File Appender -->
<appender name="ERROR_FILE"
class="ch.qos.logback.core.rolling.RollingFileAppender">
  <file>${LOG_PATH}/${APP_NAME}-error.log</file>
  <filter class="ch.qos.logback.classic.filter.LevelFilter">
    <level>ERROR</level>
    <onMatch>ACCEPT</onMatch>
    <onMismatch>DENY</onMismatch>
  </filter>
  <encoder
class="net.logstash.logback.encoder.LoggingEventCompositeJsonEncoder">
    <providers>
      <timestamp/>
      <version/>
      <logLevel/>
      <loggerName/>
      <mdc/>
      <arguments/>
      <message/>
      <stackTrace/>
    </providers>
  </encoder>
  <rollingPolicy class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">
    <fileNamePattern>${LOG_PATH}/${APP_NAME}-error.%d{yyyy-MM-
dd}.log.gz</fileNamePattern>
    <maxHistory>60</maxHistory>
  </rollingPolicy>
</appender>

<!-- Profiles -->
<springProfile name="development">
  <root level="DEBUG">
    <appender-ref ref="CONSOLE"/>
  </root>
  <logger name="com.company.ecommerce" level="DEBUG"/>
  <logger name="org.springframework.web" level="DEBUG"/>
</springProfile>

<springProfile name="production">
  <root level="INFO">
    <appender-ref ref="ASYNC_FILE"/>
    <appender-ref ref="ERROR_FILE"/>
  </root>
  <logger name="com.company.ecommerce" level="INFO"/>

```

```

        <logger name="org.springframework.security" level="WARN"/>
        <logger name="org.hibernate" level="WARN"/>
    </springProfile>

</configuration>

```

Structured Logging Implementation

```

// Logging Service con MDC
@Component
public class LoggingService {

    private static final Logger logger =
        LoggerFactory.getLogger(LoggingService.class);

    public void logUserAction(String userId, String action, String resource,
        Object details) {
        try {
            MDC.put("userId", userId);
            MDC.put("action", action);
            MDC.put("resource", resource);
            MDC.put("timestamp", Instant.now().toString());

            logger.info("User action performed: {}",
                Map.of(
                    "userId", userId,
                    "action", action,
                    "resource", resource,
                    "details", details,
                    "ip", getCurrentUserIP(),
                    "userAgent", getCurrentUserAgent()
                ));
        } finally {
            MDC.clear();
        }
    }

    public void logBusinessEvent(String eventType, Object eventData) {
        try {
            MDC.put("eventType", eventType);
            MDC.put("eventId", UUID.randomUUID().toString());

            logger.info("Business event: {}",
                Map.of(
                    "eventType", eventType,
                    "eventData", eventData,
                    "timestamp", Instant.now(),
                    "source", "ecommerce-api"
                ));
        } finally {
            MDC.clear();
        }
    }
}

```

```
    }
}

    public void logPerformanceMetric(String operation, long durationMs, boolean
success) {
    try {
        MDC.put("operation", operation);
        MDC.put("duration", String.valueOf(durationMs));
        MDC.put("success", String.valueOf(success));

        if (durationMs > 1000) {
            logger.warn("Slow operation detected: {} took {}ms", operation,
durationMs);
        } else {
            logger.debug("Operation completed: {} in {}ms", operation,
durationMs);
        }
    } finally {
        MDC.clear();
    }
}

    public void logError(String operation, Exception error, Object context) {
    try {
        MDC.put("operation", operation);
        MDC.put("errorType", error.getClass().getSimpleName());
        MDC.put("errorMessage", error.getMessage());

        logger.error("Operation failed: {} - Error: {}", operation,
error.getMessage(), error);

        // Send to external error tracking
        sendToErrorTracking(operation, error, context);
    } finally {
        MDC.clear();
    }
}

    private void sendToErrorTracking(String operation, Exception error, Object
context) {
        // Integration with Sentry, Bugsnag, etc.
    }
}

// Request Logging Filter
@Component
public class RequestLoggingFilter implements Filter {

    private static final Logger logger =
LoggerFactory.getLogger(RequestLoggingFilter.class);

    @Override
    public void doFilter(ServletRequest request, ServletResponse response,
FilterChain chain)
```

```

        throws IOException, ServletException {

    HttpServletRequest httpRequest = (HttpServletRequest) request;
    HttpServletResponse httpResponse = (HttpServletResponse) response;

    String requestId = UUID.randomUUID().toString();
    long startTime = System.currentTimeMillis();

    try {
        MDC.put("requestId", requestId);
        MDC.put("method", httpRequest.getMethod());
        MDC.put("uri", httpRequest.getRequestURI());
        MDC.put("userAgent", httpRequest.getHeader("User-Agent"));
        MDC.put("ip", getClientIP(httpRequest));

        logger.info("Request started: {} {}", httpRequest.getMethod(),
httpRequest.getRequestURI());

        chain.doFilter(request, response);

    } finally {
        long duration = System.currentTimeMillis() - startTime;

        MDC.put("status", String.valueOf(httpResponse.getStatus()));
        MDC.put("duration", String.valueOf(duration));

        logger.info("Request completed: {} {} - Status: {} - Duration: {}ms",
            httpRequest.getMethod(),
            httpRequest.getRequestURI(),
            httpResponse.getStatus(),
            duration);

        MDC.clear();
    }
}

private String getClientIP(HttpServletRequest request) {
    String xForwardedFor = request.getHeader("X-Forwarded-For");
    if (xForwardedFor != null && !xForwardedFor.isEmpty()) {
        return xForwardedFor.split(",")[0].trim();
    }
    return request.getRemoteAddr();
}
}

```

Comparación con Winston en Node.js:

```

// Node.js/Winston equivalent
import winston from "winston";

const logger = winston.createLogger({
    level: "info",

```



```

format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.errors({ stack: true }),
    winston.format.json()
),
defaultMeta: { service: "ecommerce-api" },
transports: [
    new winston.transports.File({ filename: "error.log", level: "error" }),
    new winston.transports.File({ filename: "combined.log" }),
    new winston.transports.Console({
        format: winston.format.simple(),
    }),
],
});

// Usage
logger.info("User action", {
    userId: "123",
    action: "login",
    ip: req.ip,
    userAgent: req.get("User-Agent"),
});

```

Metrics con Micrometer y Prometheus

Custom Business Metrics

```

@Component
public class EcommerceMetrics {

    private final MeterRegistry meterRegistry;

    // Counters
    private final Counter userRegistrations;
    private final Counter orderCreations;
    private final Counter paymentProcessed;
    private final Counter loginAttempts;

    // Timers
    private final Timer orderProcessingTime;
    private final Timer paymentProcessingTime;
    private final Timer databaseQueryTime;

    // Gauges
    private final AtomicInteger activeUsers = new AtomicInteger(0);
    private final AtomicInteger pendingOrders = new AtomicInteger(0);

    // Distribution Summary
    private final DistributionSummary orderValue;
    private final DistributionSummary cartSize;

```

```
public EcommerceMetrics(MeterRegistry meterRegistry, UserService userService,
OrderService orderService) {
    this.meterRegistry = meterRegistry;

    // Initialize counters
    this.userRegistrations = Counter.builder("ecommerce.users.registrations")
        .description("Number of user registrations")
        .tag("type", "business")
        .register(meterRegistry);

    this.orderCreations = Counter.builder("ecommerce.orders.created")
        .description("Number of orders created")
        .register(meterRegistry);

    this.paymentProcessed = Counter.builder("ecommerce.payments.processed")
        .description("Number of payments processed")
        .register(meterRegistry);

    this.loginAttempts = Counter.builder("ecommerce.auth.login.attempts")
        .description("Number of login attempts")
        .register(meterRegistry);

    // Initialize timers
    this.orderProcessingTime =
    Timer.builder("ecommerce.orders.processing.time")
        .description("Time taken to process orders")
        .register(meterRegistry);

    this.paymentProcessingTime =
    Timer.builder("ecommerce.payments.processing.time")
        .description("Time taken to process payments")
        .register(meterRegistry);

    this.databaseQueryTime = Timer.builder("ecommerce.database.query.time")
        .description("Database query execution time")
        .register(meterRegistry);

    // Initialize gauges
    Gauge.builder("ecommerce.users.active")
        .description("Number of active users")
        .register(meterRegistry, activeUsers, AtomicInteger::get);

    Gauge.builder("ecommerce.orders.pending")
        .description("Number of pending orders")
        .register(meterRegistry, pendingOrders, AtomicInteger::get);

    Gauge.builder("ecommerce.inventory.low.stock")
        .description("Number of products with low stock")
        .register(meterRegistry, this,
EcommerceMetrics::getLowStockCount);

    // Initialize distribution summaries
    this.orderValue = DistributionSummary.builder("ecommerce.orders.value")
        .description("Distribution of order values")
```

```
        .baseUnit("currency")
        .register(meterRegistry);

    this.cartSize = DistributionSummary.builder("ecommerce.cart.size")
        .description("Distribution of cart sizes")
        .baseUnit("items")
        .register(meterRegistry);

    // Scheduled metrics updates
    scheduleMetricsUpdates(userService, orderService);
}

// Business event tracking
public void recordUserRegistration(String userType, String source) {
    userRegistrations.increment(
        Tags.of(
            "user_type", userType,
            "source", source,
            "country", getCurrentUserCountry()
        )
    );
}

public void recordOrderCreation(BigDecimal orderValue, String orderType, int
itemCount) {
    orderCreations.increment(
        Tags.of(
            "order_type", orderType,
            "value_range", getValueRange(orderValue),
            "item_count_range", getItemCountRange(itemCount)
        )
    );

    this.orderValue.record(orderValue.doubleValue());
    this.cartSize.record(itemCount);
}

public void recordPaymentProcessing(String paymentMethod, String status,
Duration processingTime) {
    paymentProcessed.increment(
        Tags.of(
            "payment_method", paymentMethod,
            "status", status
        )
    );

    paymentProcessingTime.record(processingTime);
}

public void recordLoginAttempt(String status, String userType) {
    loginAttempts.increment(
        Tags.of(
            "status", status,
            "user_type", userType
        )
    );
}
```

```

        )
    );
}

public Timer.Sample startOrderProcessing() {
    return Timer.start(meterRegistry);
}

public void recordOrderProcessingComplete(Timer.Sample sample, String status)
{
    sample.stop(Timer.builder("ecommerce.orders.processing.time")
        .tag("status", status)
        .register(meterRegistry));
}

// Helper methods
private String getValueRange(BigDecimal value) {
    if (value.compareTo(BigDecimal.valueOf(50)) <= 0) return "0-50";
    if (value.compareTo(BigDecimal.valueOf(100)) <= 0) return "51-100";
    if (value.compareTo(BigDecimal.valueOf(500)) <= 0) return "101-500";
    if (value.compareTo(BigDecimal.valueOf(1000)) <= 0) return "501-1000";
    return "1000+";
}

private String getItemCountRange(int count) {
    if (count <= 1) return "1";
    if (count <= 3) return "2-3";
    if (count <= 5) return "4-5";
    if (count <= 10) return "6-10";
    return "10+";
}

private double getLowStockCount() {
    // Implementation to get low stock count
    return productService.getLowStockCount();
}

private void scheduleMetricsUpdates(UserService userService, OrderService
orderService) {
    ScheduledExecutorService scheduler = Executors.newScheduledThreadPool(1);

    scheduler.scheduleAtFixedRate(() -> {
        activeUsers.set(userService.getActiveUserCount());
        pendingOrders.set(orderService.getPendingOrderCount());
    }, 0, 30, TimeUnit.SECONDS);
}
}

```

Prometheus Configuration

```
# prometheus.yml
global:
  scrape_interval: 15s
  evaluation_interval: 15s

rule_files:
  - "ecommerce_alerts.yml"

scrape_configs:
  - job_name: "ecommerce-api"
    static_configs:
      - targets: ["ecommerce-api:8080"]
    metrics_path: "/actuator/prometheus"
    scrape_interval: 15s
    scrape_timeout: 10s

  - job_name: "ecommerce-database"
    static_configs:
      - targets: ["postgres-exporter:9187"]

  - job_name: "ecommerce-redis"
    static_configs:
      - targets: ["redis-exporter:9121"]

alerting:
  alertmanagers:
    - static_configs:
        - targets:
            - alertmanager:9093
```

```
# ecommerce_alerts.yml
groups:
  - name: ecommerce-api-alerts
    rules:
      - alert: HighErrorRate
        expr: rate(http_server_requests_total{status=~"5.."}[5m]) > 0.1
        for: 2m
        labels:
          severity: critical
        annotations:
          summary: "High error rate detected"
          description: "Error rate is {{ $value }} errors per second"

      - alert: SlowResponseTime
        expr: histogram_quantile(0.95,
rate(http_server_requests_duration_seconds_bucket[5m])) > 2
        for: 5m
        labels:
          severity: warning
        annotations:
          summary: "Slow response time detected"
```

```

        description: "95th percentile response time is {{ $value }} seconds"

- alert: HighDatabaseConnectionUsage
  expr: (hikaricp_connections_active / hikaricp_connections_max) > 0.8
  for: 3m
  labels:
    severity: warning
  annotations:
    summary: "High database connection usage"
    description: "Database connection usage is {{ $value }}%"

- alert: LowInventoryStock
  expr: ecommerce_inventory_low_stock > 10
  for: 1m
  labels:
    severity: info
  annotations:
    summary: "Low inventory stock detected"
    description: "{{ $value }} products have low stock"

- alert: HighOrderProcessingTime
  expr: histogram_quantile(0.90,
rate(ecommerce_orders_processing_time_seconds_bucket[10m])) > 30
  for: 5m
  labels:
    severity: warning
  annotations:
    summary: "Order processing time is high"
    description: "90th percentile order processing time is {{ $value }}
seconds"

```

🔍 Distributed Tracing con Spring Cloud Sleuth

📄 Configuración de Tracing

```

<!-- pom.xml dependencies -->
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-sleuth</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-sleuth-zipkin</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-zipkin</artifactId>
</dependency>

```

```
# application.yml - Tracing configuration
spring:
  sleuth:
    sampler:
      probability: 1.0 # 100% sampling in dev, 0.1 in prod
    zipkin:
      base-url: http://zipkin:9411
      sender:
        type: web
    web:
      client:
        enabled: true
        additional-skip-pattern: "/actuator.*|/health.*"
    opentracing:
      jaeger:
        enabled: false
    baggage:
      remote-fields: user-id, session-id, correlation-id
      correlation-enabled: true
      correlation-fields: user-id, session-id

  application:
    name: ecommerce-api

logging:
  pattern:
    level: "[%X{traceId:-},%X{spanId:-}] %-5level"
    console: "%d{yyyy-MM-dd HH:mm:ss} [%X{traceId:-},%X{spanId:-}] %-5level
%logger{36} - %msg%n"
    file: "%d{yyyy-MM-dd HH:mm:ss} [%X{traceId:-},%X{spanId:-}] %-5level
%logger{36} - %msg%n"
```

🔗 Custom Spans y Trace Enrichment

```
// Tracing Service
@Component
public class TracingService {

    private final Tracer tracer;
    private final BaggageManager baggageManager;

    public TracingService(Tracer tracer, BaggageManager baggageManager) {
        this.tracer = tracer;
        this.baggageManager = baggageManager;
    }

    public void setUserContext(String userId, String sessionId) {
        // Set baggage for cross-service context
        baggageManager.createBaggage("user-id", userId).makeCurrent();
        baggageManager.createBaggage("session-id", sessionId).makeCurrent();
    }
}
```

```
// Add to current span
Span currentSpan = tracer.nextSpan().name("user-context-setup");
currentSpan.tag("user.id", userId);
currentSpan.tag("session.id", sessionId);
currentSpan.start();
currentSpan.end();
}

public <T> T traceOperation(String operationName, Supplier<T> operation) {
    Span span = tracer.nextSpan().name(operationName);

    try (Tracer.SpanInScope spanInScope = tracer.withSpanInScope(span)) {
        span.start();
        span.tag("component", "business-logic");

        T result = operation.get();

        span.tag("success", "true");
        return result;
    } catch (Exception e) {
        span.tag("success", "false");
        span.tag("error", e.getMessage());
        throw e;
    } finally {
        span.end();
    }
}

public void traceAsyncOperation(String operationName, Runnable operation) {
    Span span = tracer.nextSpan().name(operationName);

    CompletableFuture.runAsync(() -> {
        try (Tracer.SpanInScope spanInScope = tracer.withSpanInScope(span)) {
            span.start();
            span.tag("async", "true");

            operation.run();

            span.tag("success", "true");
        } catch (Exception e) {
            span.tag("success", "false");
            span.tag("error", e.getMessage());
        } finally {
            span.end();
        }
    });
}

// Service Layer Tracing
@Service
@NewSpan("order-service")
```



```
public class OrderService {

    @Autowired
    private TracingService tracingService;

    @Autowired
    private UserServiceClient userServiceClient;

    @Autowired
    private PaymentServiceClient paymentServiceClient;

    @NewSpan("create-order")
    public OrderDto createOrder(CreateOrderRequest request) {
        return tracingService.traceOperation("order-creation", () -> {

            // Step 1: Validate user
            Span userValidationSpan = tracer.nextSpan().name("validate-user");
            UserDto user;
            try (Tracer.SpanInScope spanInScope =
tracer.withSpanInScope(userValidationSpan)) {
                userValidationSpan.start();
                userValidationSpan.tag("user.id", request.getUserId().toString());

                user = userServiceClient.getUserById(request.getUserId());

                userValidationSpan.tag("user.valid", "true");
            } finally {
                userValidationSpan.end();
            }

            // Step 2: Calculate total
            Span calculationSpan = tracer.nextSpan().name("calculate-order-
total");
            BigDecimal total;
            try (Tracer.SpanInScope spanInScope =
tracer.withSpanInScope(calculationSpan)) {
                calculationSpan.start();
                calculationSpan.tag("items.count",
String.valueOf(request.getItems().size()));

                total = calculateOrderTotal(request.getItems());

                calculationSpan.tag("order.total", total.toString());
            } finally {
                calculationSpan.end();
            }

            // Step 3: Process payment
            Span paymentSpan = tracer.nextSpan().name("process-payment");
            PaymentDto payment;
            try (Tracer.SpanInScope spanInScope =
tracer.withSpanInScope(paymentSpan)) {
                paymentSpan.start();
                paymentSpan.tag("payment.amount", total.toString());
```

```

        paymentSpan.tag("payment.method", request.getPaymentMethod());

        payment = paymentServiceClient.processPayment(
            PaymentRequest.builder()
                .amount(total)
                .method(request.getPaymentMethod())
                .userId(request.getUserId())
                .build()
        );

        paymentSpan.tag("payment.status", payment.getStatus());
        paymentSpan.tag("payment.id", payment.getId().toString());
    } finally {
        paymentSpan.end();
    }

    // Step 4: Save order
    Order order = Order.builder()
        .userId(request.getUserId())
        .items(request.getItems())
        .totalAmount(total)
        .paymentId(payment.getId())
        .status(OrderStatus.CONFIRMED)
        .build();

    Order savedOrder = orderRepository.save(order);

    // Step 5: Send confirmation (async)
    tracingService.traceAsyncOperation("send-order-confirmation", () -> {
        emailService.sendOrderConfirmation(user.getEmail(), savedOrder);
    });

    return orderMapper.toDto(savedOrder);
});
}

@ContinueSpan
public OrderDto updateOrderStatus(@SpanTag("order.id") Long orderId,
                                   @SpanTag("new.status") OrderStatus newStatus)
{
    Order order = orderRepository.findById(orderId)
        .orElseThrow(() -> new OrderNotFoundException("Order not found: "
+ orderId));

    order.setStatus(newStatus);
    Order updatedOrder = orderRepository.save(order);

    // Add custom span event
    Span currentSpan = tracer.currentSpan();
    if (currentSpan != null) {
        currentSpan.event("order-status-updated");
        currentSpan.tag("previous.status", order.getStatus().toString());
    }
}

```

```

        return orderMapper.toDto(updatedOrder);
    }
}

```

🌐 Cross-Service Tracing con Feign

```

// Feign Client with Tracing
@FeignClient(name = "user-service",
    configuration = FeignTracingConfiguration.class,
    fallback = UserServiceFallback.class)
public interface UserServiceClient {

    @GetMapping("/users/{userId}")
    UserDto getUserById(@PathVariable("userId") Long userId);

    @GetMapping("/users/{userId}/preferences")
    UserPreferencesDto getUserPreferences(@PathVariable("userId") Long userId);
}

@Configuration
public class FeignTracingConfiguration {

    @Bean
    public RequestInterceptor tracingRequestInterceptor() {
        return requestTemplate -> {
            Span currentSpan = tracer.currentSpan();
            if (currentSpan != null) {
                TraceContext traceContext = currentSpan.context();
                requestTemplate.header("X-Trace-Id", traceContext.traceId());
                requestTemplate.header("X-Span-Id", traceContext.spanId());

                // Add baggage headers
                BaggageInScope baggage = baggageManager.getBaggage("user-id");
                if (baggage != null) {
                    requestTemplate.header("X-User-Id", baggage.get());
                }
            }
        };
    }

    @Bean
    public ErrorDecoder tracingErrorDecoder() {
        return (methodKey, response) -> {
            Span currentSpan = tracer.currentSpan();
            if (currentSpan != null) {
                currentSpan.tag("http.status_code",
String.valueOf(response.status()));
                currentSpan.tag("error", "true");
            }
            return new FeignException.Default(methodKey, response);
        };
    }
}

```

```

    };
  }
}

```

Comparación con OpenTelemetry en Node.js:

```

// Node.js/OpenTelemetry equivalent
import { trace, context, SpanKind } from "@opentelemetry/api";
import { NodeSDK } from "@opentelemetry/auto-instrumentations-node";

const tracer = trace.getTracer("ecommerce-api");

export async function createOrder(request: CreateOrderRequest): Promise<OrderDto>
{
  return tracer.startActiveSpan("create-order", async (span) => {
    span.setAttributes({
      "user.id": request.userId,
      "items.count": request.items.length,
    });

    try {
      // Step 1: Validate user
      const user = await tracer.startActiveSpan("validate-user", async (userSpan)
=> {
        userSpan.setAttributes({ "user.id": request.userId });
        const result = await userServiceClient.getUserById(request.userId);
        userSpan.setAttributes({ "user.valid": true });
        return result;
      });

      // Step 2: Process payment
      const payment = await tracer.startActiveSpan("process-payment", async
(paymentSpan) => {
        paymentSpan.setAttributes({
          "payment.amount": total,
          "payment.method": request.paymentMethod,
        });
        return await paymentServiceClient.processPayment(paymentRequest);
      });

      span.setAttributes({ "order.status": "success" });
      return orderDto;
    } catch (error) {
      span.recordException(error);
      span.setAttributes({ "order.status": "failed" });
      throw error;
    } finally {
      span.end();
    }
  });
}

```

Grafana Dashboards

E-commerce Business Dashboard

```
{
  "dashboard": {
    "id": null,
    "title": "E-commerce Business Metrics",
    "tags": ["ecommerce", "business"],
    "timezone": "browser",
    "panels": [
      {
        "title": "Order Volume",
        "type": "stat",
        "targets": [
          {
            "expr": "increase(ecommerce_orders_created_total[1h])",
            "legendFormat": "Orders/Hour"
          }
        ],
        "fieldConfig": {
          "defaults": {
            "color": {
              "mode": "thresholds"
            },
            "thresholds": {
              "steps": [
                { "color": "red", "value": 0 },
                { "color": "yellow", "value": 10 },
                { "color": "green", "value": 50 }
              ]
            }
          }
        }
      },
      {
        "title": "Revenue Trend",
        "type": "timeseries",
        "targets": [
          {
            "expr": "sum(increase(ecommerce_orders_value_sum[5m]))",
            "legendFormat": "Revenue"
          }
        ]
      },
      {
        "title": "Order Processing Time",
        "type": "timeseries",
        "targets": [
          {
            "expr": "histogram_quantile(0.50,
```

```

    rate(ecommerce_orders_processing_time_seconds_bucket[5m]))",
    "legendFormat": "P50"
  },
  {
    "expr": "histogram_quantile(0.95,
rate(ecommerce_orders_processing_time_seconds_bucket[5m]))",
    "legendFormat": "P95"
  },
  {
    "expr": "histogram_quantile(0.99,
rate(ecommerce_orders_processing_time_seconds_bucket[5m]))",
    "legendFormat": "P99"
  }
]
},
{
  "title": "Active Users",
  "type": "gauge",
  "targets": [
    {
      "expr": "ecommerce_users_active",
      "legendFormat": "Active Users"
    }
  ],
  "fieldConfig": {
    "defaults": {
      "min": 0,
      "max": 1000,
      "unit": "short"
    }
  }
},
{
  "title": "Error Rate by Endpoint",
  "type": "table",
  "targets": [
    {
      "expr": "sum(rate(http_server_requests_total{status=~\"5..\"}[5m])) by (uri) / sum(rate(http_server_requests_total[5m])) by (uri) * 100",
      "format": "table",
      "legendFormat": "{{uri}}"
    }
  ]
},
{
  "title": "Top Selling Products",
  "type": "bargauge",
  "targets": [
    {
      "expr": "topk(10, sum(increase(ecommerce_orders_created_total[1h])) by (product_id))",
      "legendFormat": "Product {{product_id}}"
    }
  ]
}
]

```

```

    }
  ],
  "time": {
    "from": "now-1h",
    "to": "now"
  },
  "refresh": "30s"
}
}

```

🔑 Technical Performance Dashboard

```

{
  "dashboard": {
    "title": "E-commerce API Performance",
    "panels": [
      {
        "title": "Request Rate",
        "type": "timeseries",
        "targets": [
          {
            "expr": "sum(rate(http_server_requests_total[5m])) by (method, uri)",
            "legendFormat": "{{method}} {{uri}}"
          }
        ]
      },
      {
        "title": "Response Time Distribution",
        "type": "heatmap",
        "targets": [
          {
            "expr":
"sum(increase(http_server_requests_duration_seconds_bucket[2m])) by (le)",
            "format": "heatmap",
            "legendFormat": "{{le}}"
          }
        ]
      },
      {
        "title": "Database Connection Pool",
        "type": "timeseries",
        "targets": [
          {
            "expr": "hikaricp_connections_active",
            "legendFormat": "Active Connections"
          },
          {
            "expr": "hikaricp_connections_idle",
            "legendFormat": "Idle Connections"
          }
        ]
      }
    ]
  }
}

```

```

        "expr": "hikaricp_connections_max",
        "legendFormat": "Max Connections"
    }
]
},
{
    "title": "JVM Memory Usage",
    "type": "timeseries",
    "targets": [
        {
            "expr": "jvm_memory_used_bytes{area=\"heap\"}",
            "legendFormat": "Heap Used"
        },
        {
            "expr": "jvm_memory_max_bytes{area=\"heap\"}",
            "legendFormat": "Heap Max"
        }
    ]
},
{
    "title": "GC Activity",
    "type": "timeseries",
    "targets": [
        {
            "expr": "rate(jvm_gc_collection_seconds_sum[5m])",
            "legendFormat": "GC Time/sec"
        },
        {
            "expr": "rate(jvm_gc_collection_seconds_count[5m])",
            "legendFormat": "GC Collections/sec"
        }
    ]
}
]
}
}

```

Error Tracking y Alerting Avanzado

Error Tracking Integration

```

// Sentry Integration
@Configuration
public class SentryConfig {

    @Bean
    public SentryOptions sentryOptions() {
        SentryOptions options = new SentryOptions();
        options.setDsn("https://your-sentry-dsn@sentry.io/project-id");
        options.setEnvironment(getEnvironment());
        options.setRelease(getApplicationVersion());
    }
}

```



```

        options.setTracesSampleRate(0.1);
        options.setDebug(false);

        // Custom tags
        options.setTag("application", "ecommerce-api");
        options.setTag("server", getServerName());

        return options;
    }

    @EventListener
    public void handleApplicationError(ApplicationErrorEvent event) {
        Sentry.withScope(scope -> {
            scope.setTag("error.type",
                event.getError().getClass().getSimpleName());
            scope.setTag("user.id", getCurrentUserId());
            scope.setTag("operation", event.getOperation());
            scope.setLevel(SentryLevel.ERROR);

            scope.setContext("request", Map.of(
                "url", getCurrentRequestUrl(),
                "method", getCurrentRequestMethod(),
                "user-agent", getCurrentUserAgent(),
                "ip", getCurrentUserIP()
            ));

            scope.setContext("business", Map.of(
                "order-id", event.getOrderId(),
                "user-type", getCurrentUserType(),
                "feature-flags", getActiveFeatureFlags()
            ));

            Sentry.captureException(event.getError());
        });
    }
}

// Global Exception Handler con Error Tracking
@ControllerAdvice
@Slf4j
public class GlobalExceptionHandler {

    @Autowired
    private ErrorTrackingService errorTrackingService;

    @Autowired
    private MetricsService metricsService;

    @ExceptionHandler(BusinessException.class)
    public ResponseEntity<ErrorResponse> handleBusinessException(BusinessException
ex, HttpServletRequest request) {
        String errorId = UUID.randomUUID().toString();

        ErrorContext context = ErrorContext.builder()

```

```

        .errorId(errorId)
        .errorType("BUSINESS_ERROR")
        .operation(ex.getOperation())
        .userId(getCurrentUserId(request))
        .requestUrl(request.getRequestURL().toString())
        .requestMethod(request.getMethod())
        .userAgent(request.getHeader("User-Agent"))
        .build();

errorTrackingService.trackError(ex, context);
metricsService.incrementErrorCounter("business", ex.getErrorCode());

log.error("Business error [{}]: {} in operation: {}",
        errorId, ex.getMessage(), ex.getOperation(), ex);

return ResponseEntity.badRequest()
        .body(ErrorResponse.builder()
                .errorId(errorId)
                .message(ex.getMessage())
                .errorCode(ex.getErrorCode())
                .timestamp(Instant.now())
                .build());
}

@ExceptionHandler(DataAccessException.class)
public ResponseEntity<ErrorResponse>
handleDataAccessException(DataAccessException ex, HttpServletRequest request) {
    String errorId = UUID.randomUUID().toString();

    ErrorContext context = ErrorContext.builder()
            .errorId(errorId)
            .errorType("DATABASE_ERROR")
            .severity("HIGH")
            .build();

    errorTrackingService.trackError(ex, context);
    metricsService.incrementErrorCounter("database", "connection");

    // Send immediate alert for database errors
    alertService.sendImmediateAlert("Database Error",
            "Critical database error detected: " +
ex.getMessage(),
            AlertSeverity.HIGH);

    log.error("Database error [{}]: {}", errorId, ex.getMessage(), ex);

    return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(ErrorResponse.builder()
                    .errorId(errorId)
                    .message("Internal server error")
                    .timestamp(Instant.now())
                    .build());
}

```

```

    @ExceptionHandler(RateLimitExceededException.class)
    public ResponseEntity<ErrorResponse>
    handleRateLimitException(RateLimitExceededException ex, HttpServletRequest
    request) {
        String userIP = getClientIP(request);

        metricsService.incrementErrorCounter("rate_limit", "exceeded");

        // Track suspicious activity
        if (ex.getAttemptCount() > 100) {
            securityService.flagSuspiciousActivity(userIP, "RATE_LIMIT_ABUSE");
        }

        log.warn("Rate limit exceeded for IP: {} - {} attempts", userIP,
        ex.getAttemptCount());

        return ResponseEntity.status(HttpStatus.TOO_MANY_REQUESTS)
            .header("Retry-After", "60")
            .body(ErrorResponse.builder()
                .message("Rate limit exceeded")
                .retryAfter(60)
                .timestamp(Instant.now())
                .build());
    }
}

```

SLI/SLO Implementation

```

// SLI/SLO Metrics Service
@Component
public class SLOMetricsService {

    private final MeterRegistry meterRegistry;
    private final Timer successfulRequests;
    private final Timer failedRequests;
    private final DistributionSummary requestLatency;

    // SLI Thresholds
    private static final Duration RESPONSE_TIME_SLI = Duration.ofMillis(500);
    private static final double AVAILABILITY_SLI = 99.9; // 99.9%
    private static final double ERROR_RATE_SLI = 0.1; // 0.1%

    public SLOMetricsService(MeterRegistry meterRegistry) {
        this.meterRegistry = meterRegistry;

        this.successfulRequests = Timer.builder("sli.requests.successful")
            .description("Successful requests within SLI threshold")
            .register(meterRegistry);

        this.failedRequests = Timer.builder("sli.requests.failed")
            .description("Failed requests or outside SLI threshold")

```

```
        .register(meterRegistry);

        this.requestLatency = DistributionSummary.builder("sli.latency")
            .description("Request latency for SLI calculation")
            .baseUnit("milliseconds")
            .register(meterRegistry);
    }

    public void recordRequest(Duration responseTime, boolean success, int
statusCode) {
        boolean withinSLI = responseTime.compareTo(RESPONSE_TIME_SLI) <= 0 &&
success;

        if (withinSLI) {
            successfulRequests.record(responseTime);
        } else {
            failedRequests.record(responseTime);
        }

        requestLatency.record(responseTime.toMillis());

        // Record detailed metrics
        meterRegistry.counter("sli.requests.total",
            Tags.of(
                "status", String.valueOf(statusCode),
                "sli_compliant", String.valueOf(withinSLI),
                "success", String.valueOf(success)
            )).increment();
    }

    public SLOReport generateSLOReport(Duration period) {
        Instant now = Instant.now();
        Instant start = now.minus(period);

        // Calculate availability SLI
        double totalRequests = getTotalRequests(start, now);
        double successfulRequests = getSuccessfulRequests(start, now);
        double availability = (successfulRequests / totalRequests) * 100;

        // Calculate latency SLI
        double p95Latency = getP95Latency(start, now);
        double latencySLI = (getRequestsUnderThreshold(start, now,
RESPONSE_TIME_SLI) / totalRequests) * 100;

        // Calculate error rate SLI
        double errorRate = (getErrorRequests(start, now) / totalRequests) * 100;

        return SLOReport.builder()
            .period(period)
            .availability(availability)
            .availabilitySLI(AVAILABILITY_SLI)
            .availabilityCompliant(availability >= AVAILABILITY_SLI)
            .latencyP95(p95Latency)
            .latencySLI(latencySLI)
```

```

        .latencyCompliant(p95Latency <= RESPONSE_TIME_SLI.toMillis())
        .errorRate(errorRate)
        .errorRateSLI(ERROR_RATE_SLI)
        .errorRateCompliant(errorRate <= ERROR_RATE_SLI)
        .totalRequests((long) totalRequests)
        .generatedAt(now)
        .build();
    }

    @Scheduled(fixedRate = 300000) // Every 5 minutes
    public void calculateAndReportSLO() {
        SLOReport hourlyReport = generateSLOReport(Duration.ofHours(1));
        SLOReport dailyReport = generateSLOReport(Duration.ofDays(1));

        // Log SLO compliance
        log.info("SLO Report (1h): Availability: {:.2f}% (SLI: {:.1f}%), " +
            "Latency P95: {}ms (SLI: {}ms), Error Rate: {:.3f}% (SLI: {:.1f}%)",
            hourlyReport.getAvailability(), AVAILABILITY_SLI,
            hourlyReport.getLatencyP95(), RESPONSE_TIME_SLI.toMillis(),
            hourlyReport.getErrorRate(), ERROR_RATE_SLI);

        // Send alerts if SLO violated
        checkSLOViolations(hourlyReport, dailyReport);

        // Export to external systems
        exportSLOMetrics(hourlyReport, dailyReport);
    }

    private void checkSLOViolations(SLOReport hourlyReport, SLOReport dailyReport)
    {
        if (!hourlyReport.isAvailabilityCompliant()) {
            alertService.sendAlert("SLO Violation - Availability",
                String.format("Availability SLO violated: %.2f%% (target: %.1f%%)",
                    hourlyReport.getAvailability(), AVAILABILITY_SLI),
                AlertSeverity.HIGH);
        }

        if (!hourlyReport.isLatencyCompliant()) {
            alertService.sendAlert("SLO Violation - Latency",
                String.format("Latency SLO violated: P95 = %dms (target: %dms)",
                    hourlyReport.getLatencyP95(),
                    RESPONSE_TIME_SLI.toMillis()),
                AlertSeverity.MEDIUM);
        }

        if (!hourlyReport.isErrorRateCompliant()) {
            alertService.sendAlert("SLO Violation - Error Rate",
                String.format("Error rate SLO violated: %.3f%% (target: %.1f%%)",
                    hourlyReport.getErrorRate(), ERROR_RATE_SLI),
                AlertSeverity.HIGH);
        }
    }

```

```
    }  
  }  
}
```

🚀 Prompts de IA para Módulo 10: Observabilidad

PROMPT GITHUB COPILOT - Observabilidad Completa:

Actúa como un SRE especializado en observabilidad para Spring Boot. Necesito implementar un stack completo de observabilidad para microservicios en producción.

Genera:

1. Logging Strategy completa:
 - Structured logging con Logback
 - MDC para correlation IDs
 - Async appenders para performance
 - Log aggregation con ELK/Loki
 - Security-safe logging patterns
2. Metrics Implementation:
 - Business metrics personalizados
 - SLI/SLO tracking automático
 - Custom Micrometer collectors
 - Prometheus alerting rules
 - Grafana dashboards automatizados
3. Distributed Tracing:
 - Spring Cloud Sleuth configuration
 - Cross-service correlation
 - Baggage propagation
 - Error span tagging
 - Performance bottleneck detection
4. Error Tracking:
 - Sentry/Bugsnag integration
 - Context-rich error reporting
 - Error rate monitoring
 - Alert fatigue prevention
 - Error trend analysis
5. Alerting Strategy:
 - Multi-channel notifications
 - Escalation policies
 - Alert correlation
 - Runbook automation
 - On-call rotation integration

Compara cada implementación con Node.js equivalents y explica las ventajas empresariales.

PROMPT CLAUDE - SLI/SLO Enterprise Implementation:

Eres un especialista en Site Reliability Engineering para Spring Boot. Necesito implementar SLI/SLO monitoring completo para garantizar service level objectives.

Implementa:

1. SLI Definition y Measurement:

- Availability SLI (uptime percentage)
- Latency SLI (response time percentiles)
- Quality SLI (error rate thresholds)
- Throughput SLI (requests per second)
- Business SLI (custom KPIs)

2. SLO Monitoring:

- Real-time SLO tracking
- Error budget calculations
- Burn rate alerting
- Multi-window SLO analysis
- Trend prediction algorithms

3. Alerting Framework:

- Priority-based alert routing
- Context-aware notifications
- Escalation policies
- Alert correlation engine
- Automated remediation triggers

4. Dashboard Strategy:

- Executive summary dashboards
- Technical deep-dive views
- Incident response dashboards
- Capacity planning views
- Business impact correlation

5. Incident Management:

- Automated incident creation
- Context gathering automation
- Runbook integration
- Post-incident analysis
- Learning loop implementation

6. Performance Optimization:

- Bottleneck identification
- Resource utilization tracking
- Scaling recommendations
- Cost optimization insights
- Predictive maintenance

Incluye automation scripts, best practices y integration con incident management tools.

🎯 Conclusiones del Curso

🚀 Resumen del Viaje de Aprendizaje

¡Felicitaciones! Has completado un viaje exhaustivo desde Node.js/TypeScript hacia el ecosistema empresarial de Java y Spring Boot. Durante estos 10 módulos, hemos cubierto:

📁 Módulos Completados:

1. **Fundamentos de Java** - Sintaxis, OOP, tipos y collections
2. **Spring Framework** - IoC, DI, configuración y arquitectura
3. **APIs REST** - Controllers, validation, error handling
4. **Base de Datos** - JPA, Hibernate, Spring Data, migraciones
5. **Seguridad** - Spring Security, JWT, autenticación/autorización
6. **Microservicios** - Service discovery, API Gateway, comunicación
7. **Testing** - Unit, Integration, TestContainers, estrategias
8. **DevOps** - Docker, CI/CD, Kubernetes, deployment
9. **Herramientas** - Maven, IDEs, profiles, documentation
10. **Observabilidad** - Logging, metrics, tracing, alerting

🎯 Habilidades Adquiridas:

- ✓ **Transición de sintaxis** JavaScript/TypeScript → Java
- ✓ **Arquitectura empresarial** con Spring Boot
- ✓ **Desarrollo de APIs** robustas y escalables
- ✓ **Persistencia de datos** con JPA/Hibernate
- ✓ **Seguridad enterprise** con Spring Security
- ✓ **Microservicios patterns** y distribución
- ✓ **Testing strategies** completas
- ✓ **DevOps practices** para producción
- ✓ **Observabilidad** y monitoring avanzado

📊 Principales Diferencias Aprendidas:

Aspecto	Node.js/TypeScript	Java/Spring Boot
Tipado	Opcional (TypeScript)	Fuerte y obligatorio
Runtime	V8 JavaScript Engine	JVM con optimizaciones
Ecosystem	npm/yarn, Express	Maven/Gradle, Spring
Concurrency	Event Loop, async/await	Threads, Virtual Threads, @Async
Memory	Garbage Collection básico	GC avanzado (G1, ZGC)
Enterprise	Frameworks externos	Ecosistema Spring integrado

🗺️ Próximos Pasos Recomendados:

1. **Práctica Hands-on:** Implementa un proyecto completo usando todos los módulos
2. **Spring Cloud:** Profundiza en microservicios con Config Server, Gateway
3. **Reactive Programming:** Explora Spring WebFlux para aplicaciones reactivas
4. **Cloud Native:** Aprende Spring Cloud y deployment en AWS/Azure/GCP
5. **Performance Tuning:** Optimización JVM y análisis de performance
6. **Event-Driven Architecture:** Kafka, RabbitMQ, Spring Cloud Stream

Recursos para Continuar:

- **Spring Official Guides:** <https://spring.io/guides>
- **Spring Boot Reference:** <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/>
- **Spring Cloud Documentation:** <https://spring.io/projects/spring-cloud>
- **Java Performance:** "Java Performance: The Definitive Guide"
- **Microservices Patterns:** "Microservices Patterns" by Chris Richardson

¡Has Completado la Transición!

Ahora tienes las herramientas y conocimientos para construir aplicaciones empresariales robustas con Java y Spring Boot. El ecosistema Spring te proporciona todo lo necesario para desarrollar, testear, desplegar y monitorear aplicaciones a escala empresarial.

¡Que disfrutes tu nueva carrera como Java/Spring Boot Developer! 🚀

Estadísticas del Curso:

- **Total de líneas:** 7,500+
 - **Módulos:** 10 completos
 - **Comparaciones Node.js:** 50+
 - **Ejemplos de código:** 100+
 - **Prompts de IA:** 20 especializados
 - **Tiempo estimado:** 40-60 horas de estudio
-